CPE 334 Software Engineering, Semester 1/2026

Project Deliverable 1

Event Registration and Attendance System (Theme 3)

Student A	Kirakit Kingkaew, 67070503460 (GitHub songt888)
Student B	Nattakit Prasertsak, 67070503413 (GitHub stickkersz)
Repository	https://github.com/songt888/event-registration-system
Submitted release	tag `v0.1.0-sprint1`, commit `12dcd3c1504a073507017c65a4e8168e8e0e5bef` on `main`
Date	8 November 2026

Navigation

Part	Content	Pages	Relevant IDs
Part A	SRS and product scope	3 to 9	FR-01 to FR-15, BR-01 to BR-09, NFR-01 to NFR-09, D-01 to D-16
Part B	Feature inventory and traceability	10 to 16	F-01 to F-13; requirement to feature to sprint to AC/TEST; gaps
Part C	System-Level SDS	17 to 23	architecture, data model, UI/API conventions, roles, auth, configuration, deployment
Part D	Three-sprint plan	24 to 26	S1 to S3 goals, F-01 to F-13, dependencies, Sprint 1 scope
Part E	Sprint 1 contracts and Definition of Done	27 to 46	F-01 to F-06, AC-01 to AC-51, TEST-01 to TEST-51, DoD
Part F	Working Sprint 1 and evidence	47 to 60	demos (AC-22 to AC-24, AC-04, AC-13 to AC-21), results, TDD, history, contributions; release v0.1.0-sprint1
	F1. Setup summary	47	
	F2. Program demonstrations	47	
	F3. Build and automated test results	50	
	F4. Test excerpts and TDD evidence	54	
	F5. Engineering workflow and history	55	
	F6. Contributions	59	
	F7. AI-use declaration	59	
	F8. Known limitations	60	

Page numbers refer to this PDF. Execution evidence for the requirements in Parts A to E is in Part F; IDs are the same in every Part and in the repository.

Contents

Part A. SRS and product scope 3

Software Requirements Specification 3

Step 0 — Kickoff Decisions 7

Part B. Feature inventory and traceability 10

Feature Inventory 10

Requirements and Verification Traceability 11

Part C. System-Level SDS 17

System-Level Software Design Specification (SDS) 17

Part D. Three-sprint plan 24

Three-Sprint Plan 24

Part E. Sprint 1 feature contracts and Product Definition of Done 27

F-01 — Authentication Contract 27

F-02 Publish event contract 30

F-03 Browse events: engineering contract (Sprint 1) 32

F-04 Register for an event: engineering contract (Sprint 1) 35

F-05 My registrations and F-06 Event registrations (organizer): engineering contract (Sprint 1) 40

Sprint 1 — Product Definition of Done 46

Part F. Working Sprint 1 and evidence 47

F1. Setup summary 47

F2. Program demonstrations 47

F3. Build and automated test results 50

F4. Test excerpts and TDD evidence 54

F5. Engineering workflow and history 55

F6. Contributions 59

F7. AI-use declaration 59

F8. Known limitations, incomplete work and failed or skipped tests 60

End of contents. Numbers are the page numbers printed at the bottom of each page.

Part A. SRS and product scope

Software Requirements Specification

Event Registration and Attendance System | CPE 334, Semester 1/2026 | Theme 3

Status: draft for Issue #12 and teammate review. This document specifies the whole product baseline; Sprint 1 is the Deliverable 1 implementation scope. Requirements and verification below are planned, not claims of implemented or tested behavior.

1. Purpose, context and actors

University campus organizers need to publish seminars, workshops and activities with limited places, keep a reliable registration list, and later record attendance. Participants need to discover published events, register without duplicate or excess bookings, and retrieve their saved registration. The product connects these workflows through a web UI, backend application and persistent database. All demonstration users and events are synthetic.

Stakeholder or actor	Responsibility and need
Participant (`PARTICIPANT`)	Discover published events, register and read their own registrations; later cancel their own registration.
Organizer (`ORGANIZER`)	Create and publish their own events, view their events' registrations; later manage events, check participants in and view attendance summaries.
Unauthenticated visitor	Log in using a seeded account. Protected operations reject the request until authentication succeeds. This is not a third account role.
Two-student project team	Maintain the baseline and feature contracts, implement and verify the planned increments, and retain authentic review and release evidence.
Course assessors and later UAT testers	Reproduce a recorded release with safe accounts/data and inspect traceable requirements, tests and evidence. They are not additional application roles.

Each account has exactly one role. The backend obtains identity and role from the authenticated session; a client-supplied user ID or role cannot authorize an operation.

2. Product scope and delivery boundaries

Increment	Included product behavior
Sprint 1 / Deliverable 1	Seeded-account login/logout/current user; organizer creates a real DRAFT and separately publishes it; authenticated event browsing with remaining places; participant registration with capacity, duplicate and concurrency enforcement; participant's own registrations; owning organizer's registration list.
Sprint 2	Cancellation and re-registration, organizer event editing/closing/cancellation and capacity changes, registration deadlines. Check-in may start here, with completion in Sprint 3.
Sprint 3	Check-in completion, attendance records/reports, organizer summaries, performance and product hardening, regression verification.

The Sprint 1 normal journey is: organizer logs in, creates and publishes an event; participant logs in, registers and retrieves the saved registration; owning organizer retrieves the event's registrations. Rejected requests must not create a registration or change the protected record.

Explicit exclusions from Sprint 1 are cancellation, deadline fields/rules, check-in, attendance reports, waitlists, self-registration and password recovery. Cancellation, deadlines, check-in and reporting remain in the whole-product baseline. Waitlists are excluded from the product (D-05); self-registration, password recovery and advanced account administration have no committed feature allocation. No additional permissions or features are implied by their omission.

Sprint 1 runs locally with persistent PostgreSQL data and documented setup. Docker packaging, completed CI/CD and GCP deployment are later Lab 5 work; the design must support that delivery. A final-project demonstration video is not a Deliverable 1 requirement.

3. Functional requirements

IDs remain stable across documents, feature contracts, tests and the final PDF. The verification column describes planned checks; detailed AC/STS belong to the feature contracts in Issue 2. Later-sprint criteria will be elaborated before those features are implemented.

ID	Observable required behavior	Sprint	Planned verification
FR-01	Log in with a seeded email/password. Valid credentials establish a session; an unknown email or wrong password returns the same generic 401 response without revealing which credential was wrong.	S1	API valid/invalid-login cases (TEST-01, TEST-02); UI login/logout journey (TEST-24).
FR-02	Logout revokes the server-side session. A subsequent protected request, including one replaying the previous cookie, returns 401.	S1	API logout/replay check (TEST-04); UI logout journey (TEST-24).

ID	Observable required behavior	Sprint	Planned verification
FR-03	Retrieve the authenticated user's ID, name and role from the session. Without a valid session, reject with 401; never return `passwordHash`.	S1	API current-user, no-session and response-field checks (including TEST-03).
FR-04	An organizer creates an event with title, description, location, future start time and valid capacity. Persist it as DRAFT owned by that organizer. A separate owner-only publish operation changes the DRAFT to PUBLISHED.	S1	API/DB creation, publication, input/time boundaries, role and ownership cases (TEST-06..TEST-09, TEST-25, TEST-26, TEST-29, TEST-30, TEST-35, TEST-36).
FR-05	An authenticated user lists or views PUBLISHED events with remaining places derived from capacity minus ACTIVE registrations. DRAFT events are absent from the published list.	S1	API list/detail and remaining-place assertions (TEST-10); UI empty-list check (TEST-11).
FR-06	A participant registers for a PUBLISHED event that has not started. Persist an ACTIVE registration and reduce remaining places by one; reject full, duplicate, hidden/missing or started-event cases without adding a row.	S1	Real API/DB and concurrent registration tests (TEST-12..16, TEST-27, TEST-28).
FR-07	A participant retrieves their own registrations and each registration's status. Requests for another participant's registration return 404 without private data.	S1	Own/foreign-registration API checks (TEST-17, TEST-18).
FR-08	An organizer retrieves registrations of an event they own. Reject participants and other organizers according to the event visibility/ownership rule in D-13.	S1	API authorized list, foreign-event and wrong-role cases (TEST-19..21).
FR-09	A participant cancels their own registration; it stops consuming an ACTIVE place and can later be re-registered under BR-07.	S2	Cancellation, ownership, capacity-release and re-registration tests in the S2 contract.
FR-10	An organizer edits, closes or cancels only their own events, including permitted capacity changes. Do not allow a capacity below the ACTIVE registration count.	S2	Allowed/rejected state transitions, ownership and capacity-boundary tests in the S2 contract.
FR-11	Enforce an event's registration deadline and reject registration after the allowed deadline boundary.	S2	Before/at/after-deadline cases once boundary/time semantics are agreed in the S2 contract.
FR-12	The owning organizer records a participant's check-in only for an ACTIVE registration, once, and within the specified event window (BR-08).	S3; may start S2	Active/inactive, duplicate, ownership and time-window tests in the check-in contract.
FR-13	Persist attendance and provide an attendance report/summary for an event owned by the requesting organizer.	S3	Saved-attendance retrieval, summary and foreign-event access tests in the reporting contract.
FR-14	Show the owning organizer a summary of registered, checked-in and no-show counts.	S3	Known-fixture count and ownership checks; agree no-show timing before implementation.
FR-15	An organizer lists only their own events in every implemented status (DRAFT and PUBLISHED in S1), opens their own DRAFT and selects an event to publish or inspect registrations. Participants cannot use the organizer-only list.	S1	API own-event list, own/foreign DRAFT and wrong-role checks (TEST-31..34).

4. Business rules

Business rules are enforced in the backend service layer and, where applicable, database constraints. Hiding UI controls is not enforcement.

ID	Required rule and boundary	Sprint	Planned verification
BR-01	Only an ORGANIZER creates/publishes events. Capacity is an integer from 1 to 10,000; zero, negative and non-integer values are invalid.	S1	Service/API capacity boundaries and wrong-role tests (TEST-07, TEST-08, TEST-09); DB constraint checks where applicable.
BR-02	ACTIVE registrations never exceed event capacity, including competing requests for the last place. In one transaction, lock the event row before checking duplicates, counting ACTIVE registrations and inserting (D-10).	S1	Full-event rejection (TEST-13), last-place concurrency (TEST-15) and the NFR-04 concurrency scenario (TEST-37).
BR-03	A participant has at most one ACTIVE registration per event. Enforce the service check and a partial unique database index on `(eventId, participantId)` where status is ACTIVE.	S1	Duplicate API case and direct database-constraint verification (TEST-14).
BR-04	Registration is allowed only for a PUBLISHED event whose start time is strictly in the future. An event that has started, including `startsAt == now`, is not registrable.	S1	DRAFT/missing and started-event rejection (TEST-27, TEST-28), using controlled fixtures/time.
BR-05	Participants read only their own private registration details. Organizers manage/view registrations only for their own events. Session identity determines record access.	S1; regress later	Cross-participant and cross-organizer API checks (TEST-18, TEST-20), plus wrong-role cases.
BR-06	Registration closes at the event's registration deadline. Exact equality and timezone handling must be resolved in the S2 contract before implementation.	S2	Boundary tests under the agreed deadline semantics.
BR-07	A cancelled registration frees a place. Its participant may re-register if the event still meets registration conditions; BR-03 still permits only one ACTIVE registration.	S2	Cancel/re-register, capacity and concurrent duplicate checks.

ID	Required rule and boundary	Sprint	Planned verification
BR-08	Check-in requires an ACTIVE registration, is performed only once by the owning organizer, and occurs only within the specified event window. Exact window bounds remain a S3 contract decision.	S3; may start S2	Status, repeat, ownership and window-boundary cases.
BR-09	At creation and again at publication, `startsAt` must be strictly later than current time. Reject past/equal time with 400 `INVALID_START_TIME` without a state change. Only the owner publishes a DRAFT; republishing PUBLISHED returns 409 `ALREADY_PUBLISHED`.	S1	Past/equal-time and publication-state cases (TEST-25, TEST-26, TEST-29, TEST-30, TEST-35, TEST-36).

5. Non-functional requirements

ID	Target / observable criterion	Practical verification	Verification sprint
NFR-01	Every unauthenticated request to a protected endpoint is rejected with 401 and returns no private data.	API checks for protected operations with no, invalid, expired and revoked sessions.	S1; regression each later sprint
NFR-02	Locally managed passwords are stored only as bcrypt hashes with cost at least 10. API responses never contain `passwordHash` or plaintext passwords.	Inspect synthetic seeded DB records and assert hash properties and response fields (including TEST-05).	S1; regression after auth changes
NFR-03	Every protected route enforces authentication and applicable role/record ownership in the backend. Client-selected identity/role cannot bypass the checks.	API role/ownership tests using at least two users of each relevant role and direct requests that bypass UI controls.	S1; regression each later sprint
NFR-04	For 20 concurrent participant registration requests to an eligible capacity-5 event, exactly 5 ACTIVE registrations are stored and capacity is not exceeded.	Real PostgreSQL integration test (TEST-37) with eligible distinct participants, followed by DB count and response assertions. Keep the separate last-place TEST-15 case.	S1
NFR-05	Published-event listing has p95 response time below 500 ms with 1,000 seeded events in the local test environment.	Performance script reporting measured requests and environment; agree sample size, warm-up and request concurrency before executing the S3 acceptance check.	S3
NFR-06	Persisted application data survives an application restart.	Save a synthetic event and registration, restart the application without resetting its database, then retrieve and compare the saved records. Repeat in the deployed environment during Lab 5.	S1 locally; Lab 5 deployment verification
NFR-07	All inputs are validated server-side. Client-visible errors have `{ "error": { "code": "...", "message": "..." } }` and disclose no stack trace.	Invalid/boundary-input API checks and assertions on error shape, status and sensitive fields.	S1; regression each later sprint
NFR-08	The application runs from a clean clone using the README in at most 10 commands.	Teammate follows documented dependencies, environment example, migration/seed and start steps; record commands and outcome. Agree command-count convention in the foundation README.	S1
NFR-09	All 5 S1 data-loading screens show a loading indicator while waiting and a readable API-error message; list screens also show an empty-state message when no rows exist. Screens: events list, event detail, My registrations, organizer My events and event registrations.	TEST-11 for the empty published list plus a per-screen loading/empty/error manual checklist in `docs/evidence/`.	S1; UX/regression hardening S3

NFR-04 is a separate capacity-5 scenario; it must not replace the competing-last-place test. No result above is marked passing until the command, tested commit and actual output exist.

6. Assumptions and open contract details

- Campus events and users are synthetic. S1 uses seeded accounts; email/password credentials are handed off safely and are not included in the PDF or committed as real secrets.
- One account has one fixed role for the S1 scope. Broader organizer authority and account administration are not assumed.
- The application uses persistent PostgreSQL; registration counts refer to ACTIVE registrations. Future statuses do not silently change S1 counting rules.
- Current time is checked by the backend. D-12 specifies controlled service time for boundary tests; display timezone and later deadline/check-in boundary conventions must be documented in their contracts.
- Later-sprint details still to agree: permitted edit/close/cancel transitions; precise deadline equality/timezone; event check-in window; the point at which no-shows are counted; performance sampling parameters. These do not remove FR-09..14 or BR-06..08 from the baseline.
- NFR-08's command-count convention and prerequisites must be explicit in the foundation README. Do not claim that a clean-clone setup has been verified before the teammate actually runs it.

7. Confirmed decisions

These resolutions retain D-01..D-14 from the kickoff decisions (Part A). They are confirmed product decisions; this SRS draft still requires teammate review.

ID	Question	Agreed resolution
D-01	Context	University campus seminars, workshops and activities; synthetic data.
D-02	Duplicates	At most one ACTIVE registration per participant/event, using a service check and partial unique DB index.
D-03	Capacity	Integer 1..10,000, consumed by ACTIVE registrations. Capacity changes are S2 and cannot reduce capacity below the ACTIVE count.
D-04	Registration deadline	S1 eligibility is PUBLISHED and not started. A separate deadline field and its rules are S2.
D-05	Waitlist	Explicitly excluded from product scope.
D-06	Registration visibility	Participant: own registrations only. Organizer: registrations of own events only.
D-07	Event lifecycle	S1 creation stores a real DRAFT; separate owner publication changes it to PUBLISHED. Only PUBLISHED events appear in the published list and accept registrations. Only an owner reads their DRAFT; organizer My events lists own events in every implemented status. Republish is a conflict. CLOSED/CANCELLED belong to S2.
D-08	Auth and logout	Email/password with bcrypt cost >= 10; random opaque session token in an httpOnly, SameSite=Lax cookie. Store only its SHA-256 hash in a server-side `Session` linked to user and expiry; expiry is 1 hour. Logout deletes the row so a copied cookie is revoked; reject expired sessions. This design uses no JWT.
D-09	Roles	PARTICIPANT and ORGANIZER; one role per account; seeded accounts only in S1.
D-10	Concurrency	In one Prisma transaction, lock the event row with `SELECT ... FOR UPDATE` first, then check ACTIVE duplicate, count ACTIVE registrations and conditionally insert. The partial unique index protects duplicates; the row lock protects capacity.
D-11	Stack and origin	React + TypeScript + Vite; Node 20 + Express + TypeScript + zod; Prisma + PostgreSQL 16; Vitest, Supertest and Playwright. Build from scratch for this group project, without copying the individual TokTickIT lab code.
D-12	Start time and testing	Reject create/publish when `startsAt ⩽ now` with 400 `INVALID_START_TIME`, leaving stored state unchanged. Service functions take `now: Date`; routes pass current time and unit tests pass a fixed time. API started-event cases use directly inserted past fixtures.
D-13	Rejection codes and hidden records	Organizer registration: 403 `FORBIDDEN`. DRAFT/missing registration target: 404 `EVENT_NOT_FOUND`. Started event: 409 `EVENT_STARTED`. Foreign participant registration: 404 `NOT_FOUND`. A non-owner organizer cannot discover a DRAFT through read/publish/registration-list operations (404 `EVENT_NOT_FOUND`); a caller with the wrong role on a role-restricted route gets 403 `FORBIDDEN` for every event ID, which reveals nothing (check order: 401, then role, then ownership); an owner-only operation on a PUBLISHED event by a non-owner returns 403 `FORBIDDEN`. The published event remains visible only under the authenticated browsing policy.
D-14	Test isolation and seed	Integration/concurrency use separate `DATABASE_URL_TEST`, never the development database; run integration files sequentially (one Vitest fork: `pool: 'forks'`, `singleFork: true`) and truncate test tables before each test. TEST-15 creates its concurrency inside that test. E2E (TEST-22..TEST-24) runs as its own command against its own database (`DATABASE_URL_E2E`), reset and seeded before each run, never together with the integration suite. Compute seeded `startsAt` relative to seed-run time, always in the future.

The D-08 logout ambiguity was resolved by server-side revocation rather than cookie deletion alone. The D-10 capacity ambiguity was resolved by locking before the count/insert. PR #11 resolved the real-DRAFT lifecycle, start-time boundaries, exact rejection codes and test isolation (D-07, D-12..D-14); IDs were added rather than renumbered.

8. Sources, ownership and change control

Source	Use in this baseline
Course `project_deliverable_1_v1.pdf`, Sections 2, 3.1 and 4.1	S1 outcome and Part A coverage: stakeholder problem, actors, scope/exclusions, FR/BR/NFR, assumptions and decisions.
Course `group_work_specs_v1.pdf`, Theme 3 and Section 8.1	Whole-product event scope, Sprint 1 example, role/ownership rules and baseline discipline.
Master plan (team-internal file `deliverable1-plan.md`, repository), Sections 1, 3 and 5	Stable requirement IDs, sprint scope, existing numeric targets and planned verification IDs.
Kickoff decisions (Part A), D-01..D-14	Confirmed resolutions of significant ambiguities.
Team workflow (team-internal file `team-workflow.md`, repository) and Issue plan (team-internal file `issue-plan.md`, repository)	Author A prepares SRS; author B reviews it; later artifacts have separate ownership.

Issue #12 owns this SRS. Feature inventory/traceability, system SDS, sprint plan and detailed S1 contracts are separate Issues; this document does not claim those outputs are complete. Keep stable IDs and record any agreed revision with its reason before

implementation. Planned tests, actual test results and teammate approval must remain distinguishable.

Step 0 — Kickoff Decisions

These become the decision table in `docs/srs.md` (Issue 1).

ID	Question	Decision	Status
D-01	Context	University campus events (seminars, workshops, activities); synthetic data	Confirmed (user)
D-02	Duplicate registration	One ACTIVE registration per participant per event; DB partial unique index + service check	Confirmed (partner review)
D-03	Capacity	Integer 1..10,000, counted against ACTIVE registrations; capacity change after publish is Sprint 2	Confirmed (partner review)
D-04	Registration deadline	Sprint 1: registration allowed while event is PUBLISHED and has not started; deadline field and rules in Sprint 2	Confirmed (partner review)
D-05	Waitlist	Out of scope (explicit exclusion)	Confirmed (partner review)
D-06	Who sees registrations	Participant: own only. Organizer: registrations of events they own only	Confirmed (partner review)
D-07	Event lifecycle	~~DRAFT -> PUBLISHED in Sprint 1 (create + publish); CLOSED/CANCELLED in Sprint 2~~ **Revised:** Sprint 1 has a real DRAFT state. `POST /api/events` creates a DRAFT owned by the organizer; a separate publish action (`POST /api/events/:id/publish`) moves DRAFT -> PUBLISHED, owner only. Only PUBLISHED events are listed and can be registered for. **Reading drafts:** `GET /api/events/:id` returns a DRAFT only to its owner (404 `EVENT_NOT_FOUND` for everyone else); `GET /api/events/mine` (ORGANIZER only, 403 for participants) lists the caller's own events in every status, for picking one to publish or to open in F-06 (FR-15). Publishing an event that is already PUBLISHED is 409 `ALREADY_PUBLISHED` (AC-36). CLOSED/CANCELLED in Sprint 2. Reason: the old wording allowed "create = publish", which left no DRAFT rows, so AC-10 ("only PUBLISHED listed") could not be tested, and it gives Sprint 1 a real state transition.	Confirmed (partner review, PR #11)
D-08	Auth and logout	Email + password, bcrypt (cost >= 10). **Server-side session:** a random opaque token in an httpOnly, SameSite=Lax cookie; only its SHA-256 hash is stored in a `Session` table (userId, expiresAt = 1h). Logout deletes the row, so a copied cookie stops working immediately; expired rows are rejected. No JWT.	Confirmed (partner review, revised)
D-09	Roles	PARTICIPANT, ORGANIZER; one role per account; seeded accounts only	Confirmed (partner review)
D-10	Last-place concurrency	In ONE transaction: (1) lock the event row first with `SELECT ... FROM "Event" WHERE id = $1 FOR UPDATE` (Prisma `$queryRaw` inside `$transaction`); (2) still inside it, check for an existing ACTIVE registration and count ACTIVE registrations; (3) reject with 409 if count >= capacity, otherwise insert. Commit releases the lock. The partial unique index on (eventId, participantId) WHERE status = 'ACTIVE' is only a backstop for duplicates; it does not protect capacity. "Atomic conditional insert" is NOT used.	Confirmed (partner review, revised)
D-11	Stack	React + TypeScript + Vite, Express + TypeScript + zod, Prisma + PostgreSQL 16, Vitest/Supertest/Playwright. Written from scratch for this project; no code copied from the individual TokTickIT labs.	Confirmed (user, both students)
D-12	Event start time	`startsAt` must be strictly later than now when an event is created and again when it is published (equal to now is rejected, AC-35); otherwise 400 `INVALID_START_TIME` and nothing changes (new BR-09). Reason: without it an organizer can publish an event nobody can ever register for (BR-04). **Testing time:** service functions take `now: Date` as an argument (routes pass `new Date()`), so unit tests use a fixed time; API tests for AC-28 and AC-30 insert fixture events with a past `startsAt` directly in the database, so they need no fake clock.	Confirmed (partner review, PR #11)

ID	Question	Decision	Status
D-13	Error codes for rejected registrations and foreign records	Organizer tries to register: 403 `FORBIDDEN`. Event is DRAFT or does not exist: 404 `EVENT_NOT_FOUND` (a participant cannot tell a draft from a missing event). Event already started: 409 `EVENT_STARTED`. Another participant's registration: 404 `NOT_FOUND`, so its existence is not revealed. Another organizer's PUBLISHED event's registrations: 403 `FORBIDDEN` (AC-20); for a DRAFT it is 404 under the event rule below. **Event rule:** a DRAFT is private, so a non-owner gets 404 `EVENT_NOT_FOUND` for every operation on it (read, publish, view registrations); a PUBLISHED event is public, so a non-owner gets 403 `FORBIDDEN` on owner-only operations. Organizer B publishing organizer A's DRAFT is therefore 404 (AC-26), so the draft's existence is not revealed. **Check order (clarified in PR #49):** session (401), then role (403), then existence and ownership. The event rule applies to callers with the operation's role, so "non-owner" above means a non-owner organizer. A caller with the wrong role on a role-restricted route gets 403 `FORBIDDEN` for every event ID (DRAFT, PUBLISHED or missing), for example a participant publishing or listing an event's registrations (AC-21). This reveals nothing, because the answer is the same for every ID. A caller with the right role still gets 404 for a DRAFT: a participant registering for one (AC-27), and anyone except the owner reading one with `GET /api/events/:id`, which every signed-in user may call (AC-32). Reason: each test needs one exact expected code; the plan said "409/404" and "403/404".	Confirmed (partner review, PR #11)
D-14	Test database	Integration and concurrency tests run against a separate PostgreSQL database (`DATABASE_URL_TEST`), never the dev database. **Isolation:** integration test files run one at a time (~~Vitest `fileParallelism: false` for the integration project~~ **Revised:** the integration project uses `pool: 'forks'` with `singleFork: true`, because `fileParallelism` is ignored inside a Vitest 3 project (PR #61); unit tests stay parallel), and every table is truncated in `beforeEach`. TEST-15 makes its parallel requests inside one test, so no other file touches the database while it runs. **E2E:** ~~Playwright runs against the dev database after `prisma migrate reset` + seed~~ **Revised:** Playwright (TEST-22..24) runs as its own command against its own database (`DATABASE_URL_E2E`), reset and seeded before each run, never in the same run as the integration suite. Reason: resetting the dev database would wipe demo data used for screenshots (PR docs/s1-01-workflow-fixes). **Seed:** every seeded `startsAt` is computed relative to the run time (always in the future), never a fixed date.	Confirmed (partner review, PR #11)
D-15	UI visual style in Sprint 1	Scope addition (team decision, 2026-11-08, Issue #73): every S1 screen gets one shared visual style, the Y2K "Millennium OS" look chosen by both students from three demos (plain, Luma-like, Y2K). Each page is a window (chrome title bar with working close, minimize and maximize buttons) on a desktop with a bottom taskbar (navigation, signed-in user, clock, sound toggle, Logout); events show as CD cases; tables are list views with status lights that always keep their word; one signature moment (the disc spins up when a registration succeeds). Readability: Verdana/Tahoma for all content and controls, a pixel font only for window titles and the boot screen. Accessibility: WCAG AA contrast, visible keyboard focus, 24 px window buttons, everything usable by keyboard, `prefers-reduced-motion` removes all motion. No change to features, API, data, routes, or to any existing text, label, role or table row that tests depend on; the new interface controls (window buttons, sound toggle) and decorative texts (boot lines, clock) change no feature. Sub-tasks 73.1 (B: this decision, D-16, the stylesheet and shell, Login, Events, Event detail) and 73.2 (A: Create event, My events, My registrations, Event registrations, the accessibility check, before/after screenshots), both merged before the release test run (10.4, then 10.1). **Theme owner: B (stickkersz).** Changing the existing theme (colors, fonts, radii, shadows, the window and taskbar shell, the 3D layer, the sounds) is agreed with B on the Issue first, as for any contract change, and is made by B or approved by B in the PR. Without asking, either student may use the existing classes, add screen rules at the end of `client/src/styles.css` built only from its existing variables, and fix bugs that do not change the look. Reason: the S1 screens met their contracts and NFR-09 but used browser defaults only; one owner keeps one consistent style.	Agreed by both students (Issue #73, Y2K chosen 2026-11-08); partner review in the 73.1 PR
D-16	Decorative 3D layer, fonts and sounds for D-15	three.js 0.169.0 is loaded at run time from the jsDelivr CDN (`+esm` build) for a decorative 3D desktop (floating CDs, floppy disks, folders); it is not an npm dependency, so `package.json` is unchanged. It is skipped when WebGL is missing or software-only (SwiftShader, llvmpipe), when the CDN fails, and draws one still frame under reduced motion; the canvas is `aria-hidden` and holds no information. Fonts Press Start 2P and VT323 (SIL Open Font License 1.1) are self-hosted in `client/public/fonts/`. Sounds are synthesized with the Web Audio API (no audio files), play only in response to the user's own actions, and are muted with the taskbar toggle (the choice is remembered). Known cost: the 3D layer makes one outside network request; without it the CSS sky is shown and nothing is lost.	Agreed by both students (Issue #73); partner review in the 73.1 PR

Partner review notes (resolved)

- D-08: partner's condition was that logout deleting the cookie does not revoke a copied JWT. Resolution: replace JWT with a server-side session so logout revokes the token. TEST-04 replays the old cookie after logout and expects 401.
- D-10: partner required locking before the count and insert, and a precise method instead of "atomic conditional insert". Resolution: row lock with `FOR UPDATE` first, as written above. TEST-15 sends parallel requests for the last place and expects exactly one success.

Team checklist

- ☑ Student A: Kirakit Kingkaew, 67070503460
- ☑ Student B: Nattakit Prasertsak, 67070503413
- ☑ D-02..D-07 and D-09 agreed by partner review
- ☑ Partner confirms the revised D-08 and D-10 (no further objections)
- □ Optional: ask TA about waitlist (D-05) and deadline timing (D-04) if the team wants them earlier

- ☑ Partner reviewed and approved revised D-07 and new D-12..D-14 (PR #11, merged 5169fb2)

Part B. Feature inventory and traceability

Feature Inventory

Event Registration and Attendance System | CPE 334, Semester 1/2026 | Theme 3

Prepared for Issue #13. This inventory allocates the whole-product baseline in the SRS (Part A) to features and completion sprints. It describes planned behavior, not implementation or test results. Detailed feature AC/STS belong to the engineering contracts; the requirement-by-requirement traceability matrix is Issue #14.

1. Features and dependencies

Dependencies below are direct product dependencies; authentication/authorization from F-01 applies to every protected feature. Sprint 1 implementation also waits for the foundation in Issue #3 and its reviewed contracts.

ID	Feature	Observable completion behavior	Related requirements	Completion sprint	Direct dependencies
F-01	Authentication foundation	Seeded accounts log in, retrieve current-user identity and log out. Expired/revoked sessions and wrong-role requests are rejected by the backend; passwords remain hashed and private.	FR-01, FR-02, FR-03; NFR-01, NFR-02, NFR-03, NFR-07	S1	Foundation: schema, seed and test runner.
F-02	Publish event	An organizer creates an owned DRAFT with valid capacity and future start time, lists/opens their own events, and separately publishes their DRAFT. Reject invalid input, foreign access and republishing.	FR-04, FR-15; BR-01, BR-09; NFR-03, NFR-07, NFR-09	S1	F-01 for protected routes.
F-03	Browse events	Signed-in users list/view PUBLISHED events and remaining places. The published list hides DRAFTs; the UI handles loading, an empty list and API errors.	FR-05; NFR-09	S1	F-02 for published events and the shared detail definition; F-01 for access.
F-04	Register for event	A participant creates a persisted ACTIVE registration for an eligible event. Reject full, duplicate, DRAFT/missing, started-event and wrong-role attempts; competing requests cannot exceed capacity.	FR-06; BR-02, BR-03, BR-04; NFR-04, NFR-07, NFR-09	S1	F-01, F-03; F-02 transitively.
F-05	My registrations	A participant retrieves their own saved registrations and statuses, with loading/empty/error states. Another participant's private registration is not returned.	FR-07; BR-05; NFR-03, NFR-09	S1	F-04.
F-06	Event registrations (organizer)	The organizer selects an event from their own events and retrieves its registrations. Reject other organizers and participants; show loading/empty/error states.	FR-08, FR-15; BR-05; NFR-03, NFR-09	S1	F-02 for owned-event selection; F-04 for registrations.
F-07	Cancel registration	A participant cancels their own ACTIVE registration, freeing its place, and can register again when the event remains eligible. Preserve the one-ACTIVE-registration rule.	FR-09; BR-03, BR-05, BR-07	S2	F-04, F-05.
F-08	Manage event	An owner edits, closes or cancels their event and changes capacity without reducing it below the ACTIVE count. Cancelled events reject registration.	FR-10; BR-01, BR-02, BR-05; D-03	S2	F-02, F-06.
F-09	Registration deadline	Apply an event's registration deadline alongside existing eligibility rules and reject registration outside the permitted boundary.	FR-11; BR-04, BR-06	S2	F-02, F-04.
F-10	Check-in	The owning organizer records check-in once for an ACTIVE registration within the agreed event window; reject invalid status, repeat, foreign-owner and out-of-window attempts.	FR-12; BR-05, BR-08; NFR-03	S3; starts S2	Start: F-06. Completion: F-06, F-07.
F-11	Attendance record and report	Persist and retrieve attendance for the requesting organizer's event and report its registered, checked-in and no-show counts under agreed counting rules.	FR-13; BR-05; NFR-03, NFR-06	S3	F-10.
F-12	Organizer summary dashboard	Show an organizer registered, checked-in and no-show totals across their own events, excluding other organizers' private records.	FR-14; BR-05; NFR-03	S3	F-11.
F-13	Hardening, UX and regression	Verify the list-performance target, repeat persistence and UI-state checks, run regression, and fix/re-test UAT defects. Include the S3 login-rate-limiting hardening identified in the system SDS; specify its policy before implementation.	NFR-05, NFR-06, NFR-09; regression of applicable baseline FR/BR/NFR	S3	Integrated F-01 through F-12; released versions and UAT findings where applicable.

The S1 path is authentication -> publication -> browsing -> registration -> participant/organizer retrieval. F-02's red tests and service work may proceed alongside F-01 after the foundation; connecting protected F-02 routes waits for `requireSession` and `requireRole`. F-05 and F-06 can proceed in parallel after F-04.

F-02 owns the full definition of `GET /api/events/:id`, including published/DRAFT visibility. F-03 consumes that definition; its contract must refer to it rather than define a competing endpoint contract.

2. Cross-cutting responsibilities and verification allocation

A later hardening feature does not defer Sprint 1 quality obligations. These responsibilities apply wherever the feature uses the relevant layer.

Responsibility	Requirement IDs	Establishment and planned verification
Session security, password protection and backend permissions	NFR-01, NFR-02, NFR-03	Established by F-01 in S1; role/ownership checks verified for each protected feature and repeated during sprint regression.
Capacity under concurrency	NFR-04	Established and verified in F-04/S1, including both the last-place case and the distinct 20-participant/capacity-5 scenario; no test design is redefined here.
Server validation and safe error responses	NFR-07	Established in the S1 API foundation/authentication flow and verified for every applicable feature; retained in later regression.
Persistent data and reproducible local setup	NFR-06, NFR-08	Foundation/README responsibility in S1; fresh-clone setup and restart/retrieval checks in S1 QA. NFR-06 is also verified for deployment in Lab 5 and revisited in F-13.
Shared loading, empty and error states	NFR-09	Required on all five S1 data-loading screens within F-02 through F-06; verified in S1 QA and revisited in F-13 for hardening. A shared visual style for all S1 screens is added in S1 (D-15, Issue #73).
Published-list performance	NFR-05	Measured in F-13/S3 against the SRS target: p95 below 500 ms with 1,000 events; measurement parameters agreed before execution.

3. Scope boundaries and remaining elaboration

- Completion allocation is S1: F-01 through F-06; S2: F-07 through F-09; S3: F-10 through F-13. F-10 starts in S2 and has one completion allocation, S3.
- Cancellation, deadline rules, check-in and attendance/reporting are future features, not omissions from the whole-product baseline. Waitlists remain excluded (D-05); self-registration, password recovery and advanced account administration have no committed feature allocation.
- The S2/S3 contracts still need exact event-edit/close/cancel transitions, cancellation timing, deadline equality/timezone, check-in window, no-show timing and performance sampling. The login-rate-limiting policy also needs its S3 contract. Do not invent these rules in implementation.
- Docker/CI/CD/GCP delivery is Lab 5 work on the same product, not an extra S1 feature. Keep lab assessment Issues/evidence separate from unrelated feature work.
- Every baseline FR, BR and important NFR has a feature or named responsibility above. Detailed AC/verification mapping, coverage gaps and approved-change records belong to `docs/traceability.md` in Issue #14; this inventory does not claim that artifact is complete.

4. Sources and change control

Use the SRS (Part A) for stable FR/BR/NFR, kickoff decisions (Part A) for D-01 through D-14, the master plan (team-internal file `deliverable1-plan.md`, repository) Section 4 for feature allocation, and the three-sprint plan (Part D) for dependencies and completion outcomes. The system SDS (Part C) defines shared design and the planned S3 rate-limiting hardening; the Issue plan (team-internal file `issue-plan.md`, repository) and team workflow (team-internal file `team-workflow.md`, repository) define ownership and integration order.

Preserve IDs and record justified changes before implementation. A feature is complete only when its agreed contract and Product Definition of Done have real verification and teammate-reviewed integration evidence.

Requirements and Verification Traceability

Event Registration and Attendance System | CPE 334, Semester 1/2026 | Issue #14

This matrix maps the whole-product SRS (Part A) to features (Part B), completion/verification sprints and the current planned acceptance/test skeleton in the master plan (team-internal file `deliverable1-plan.md`, repository), Section 5. The S1 contracts in `docs/sprints/sprint-1/` (F-01 #17, F-02 #18, F-03 #19, F-04 #20, F-05/F-06 #21) elaborate every S1 row. Every S1 test now exists and passes: the actual file, test name and result of each AC are in the single results artifact results (Part F) (51 of 51 AC pass on `sprint-1-staging` at `c1ee1ce`, QA Issue #40; NFR checks in its section 3). S2/S3 rows remain a planning baseline.

1. Reading the matrix

- For S1 rows the last column states the verified result and points to the results file; that file, not this matrix, is the evidence. For S2/S3 rows, listed references are planned only. The matching numbered pairs are retained; no ID is renumbered.
- `Pending contract` means that the responsible feature contract must elaborate the behavior and verification before implementation. A partial skeleton must not be presented as full requirement coverage.
- S1 features complete in Sprint 1. Later requirements retain their feature/sprint allocation without invented AC/test IDs; F-10 starts in S2 and completes in S3.
- Named foundation/QA responsibilities cover setup and shared NFR checks where an application feature alone does not. All protected features inherit F-01 authentication and applicable authorization.

2. Functional requirements

Requirement	Feature or responsibility	Sprint	Existing planned AC	Existing planned TEST	Coverage / remaining work
FR-01 Login	F-01	S1	AC-01, AC-02; AC-24 UI	TEST-01, TEST-02; TEST-24	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-02 Logout/revocation	F-01	S1	AC-04; AC-24 UI	TEST-04; TEST-24	Replayed cookie rejected after logout (D-08). Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-03 Current-user identity	F-01	S1	AC-03 (no session); AC-01 returns login user; AC-39 authenticated current user	TEST-03; TEST-01; TEST-39	Exact id/name/role response. Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-04 Create DRAFT/publish own event	F-02	S1	AC-06, AC-07, AC-08, AC-09, AC-25, AC-26, AC-29, AC-30, AC-35, AC-36; AC-50	TEST-06, TEST-07, TEST-08, TEST-09, TEST-25, TEST-26, TEST-29, TEST-30, TEST-35, TEST-36; TEST-50	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-05 Published-event browse/detail	F-03; F-02 owns shared detail definition	S1	AC-10 (list), AC-11 (empty UI); AC-49; AC-38	TEST-10, TEST-11; TEST-49; TEST-38	Shared published detail is F-02 AC-49/TEST-49. Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-06 Register	F-04	S1	AC-12, AC-13, AC-14, AC-15, AC-16, AC-27, AC-28; AC-43, AC-44	TEST-12, TEST-13, TEST-14, TEST-15, TEST-16, TEST-27, TEST-28; TEST-43, TEST-44	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-07 Own registrations	F-05	S1	AC-17, AC-18; AC-45, AC-46	TEST-17, TEST-18; TEST-45, TEST-46	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-08 Organizer registration list	F-06	S1	AC-19, AC-20, AC-21; AC-47, AC-48	TEST-19, TEST-20, TEST-21; TEST-47, TEST-48	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
FR-09 Cancel/re-register	F-07	S2	Pending S2 contract	Pending S2 contract	Planned future coverage; retain BR-07 and ownership.
FR-10 Owner event management/capacity change	F-08	S2	Pending S2 contract	Pending S2 contract	Planned future coverage; define transitions and capacity-count boundary before implementation.
FR-11 Registration deadline	F-09	S2	Pending S2 contract	Pending S2 contract	Planned future coverage; equality/timezone unresolved.
FR-12 Check-in	F-10	S3; starts S2	Pending check-in contract	Pending check-in contract	Planned future coverage; ACTIVE/once/owner/window conditions remain required.
FR-13 Attendance/report	F-11	S3	Pending S3 contract	Pending S3 contract	Planned future coverage; saved attendance and owner report.
FR-14 Organizer summary	F-12	S3	Pending S3 contract	Pending S3 contract	Planned future coverage; own-event totals and no-show timing.
FR-15 Own-event list and DRAFT access	F-02, F-06	S1	AC-31, AC-32, AC-33, AC-34	TEST-31, TEST-32, TEST-33, TEST-34	Verified: listed AC pass at `c1ee1ce` (results (Part F)).

3. Business rules

Requirement	Feature or responsibility	Sprint	Existing planned AC	Existing planned TEST	Coverage / remaining work
BR-01 Organizer-only publication; valid capacity	F-02; F-08 capacity changes later	S1; S2 extension	AC-06, AC-07, AC-08, AC-09, AC-25; AC-50	TEST-06, TEST-07, TEST-08, TEST-09, TEST-25; TEST-50	Capacity 1..10,000 boundaries and the publication access matrix. Verified: listed AC pass at `c1ee1ce` (results (Part F)). Capacity changes are S2 (F-08).
BR-02 Never exceed capacity	F-04; F-08 capacity changes later	S1; S2 extension	AC-12, AC-13, AC-15; AC-37	TEST-12, TEST-13, TEST-15; TEST-37	Last-place (AC-15) and capacity-5 (AC-37) are separate tests; each also passed 5 of 5 extra runs. Verified: listed AC pass at `c1ee1ce` (results (Part F)). S2 must also prevent reducing capacity below the ACTIVE count.
BR-03 At most one ACTIVE registration	F-04; F-07 cancellation/re-registration later	S1; S2 extension	AC-14	TEST-14	API rejection and the partial unique index both checked by TEST-14. Verified: listed AC pass at `c1ee1ce` (results (Part F)). S2 adds cancel/re-register.
BR-04 Only PUBLISHED, not-started registration	F-04; F-09 deadline extension	S1; S2 extension	AC-12, AC-27, AC-28	TEST-12, TEST-27, TEST-28	Valid, DRAFT/missing and started cases. Verified: listed AC pass at `c1ee1ce` (results (Part F)). Deadline extension is S2 (F-09).
BR-05 Own-record access	F-05, F-06; F-07, F-08, F-10, F-11, F-12 retain ownership	S1; regress S2/S3	AC-18, AC-19, AC-20, AC-21, AC-48; related AC-26, AC-32, AC-33	TEST-18, TEST-19, TEST-20, TEST-21, TEST-48; TEST-26, TEST-32, TEST-33	Verified: listed AC pass at `c1ee1ce` (results (Part F)).
BR-06 Deadline closes registration	F-09	S2	Pending S2 contract	Pending S2 contract	Future rule retained; no fabricated boundary/IDs.
BR-07 Cancellation frees place/re-registration	F-07	S2	Pending S2 contract	Pending S2 contract	Future rule retained; preserve BR-03.
BR-08 ACTIVE/once/owner/window check-in	F-10	S3; starts S2	Pending check-in contract	Pending check-in contract	Future rule retained; exact event window unresolved.
BR-09 Future start at create/publish; owner/state rules	F-02	S1	AC-25, AC-26, AC-29, AC-30, AC-35, AC-36	TEST-25, TEST-26, TEST-29, TEST-30, TEST-35, TEST-36	Past and equal start times on create and publish; no state change on rejection. Verified: listed AC pass at `c1ee1ce` (results (Part F)).

4. Non-functional requirements

Requirement	Feature or responsibility	Verification sprint	Existing planned AC	Existing planned TEST / method	Coverage / remaining work
NFR-01 No-session rejection without private data	F-01; every protected feature	S1; regress S2/S3	AC-03, AC-04, AC-09 (examples); AC-40, AC-42 (F-01); AC-38, AC-43, AC-45, AC-47	TEST-03, TEST-04, TEST-09; TEST-40, TEST-42; TEST-38, TEST-43, TEST-45, TEST-47	No-cookie, unknown-token, expired and logged-out session cases on every S1 route. Verified: listed AC pass at `c1ee1ce` (results (Part F)).
NFR-02 Password hashes/private responses	F-01; response protection in all API features	S1; auth regression	AC-01 (login response), AC-05 (stored hash), AC-39 (current-user response)	TEST-01, TEST-05, TEST-39	Seeded hashes are bcrypt cost 12 (TEST-05); GET /api/me returns exactly id, name and role (TEST-39). Verified: listed AC pass at `c1ee1ce` (results (Part F)). Regress in S2/S3.
NFR-03 Backend role/ownership enforcement	F-01 foundation; all protected features	S1; regress S2/S3	AC-08, AC-16, AC-18, AC-20, AC-21, AC-26, AC-32, AC-33, AC-34 (samples); AC-42 (shared guards); AC-50; AC-44, AC-46	Matching TEST-08, TEST-16, TEST-18, TEST-20, TEST-21, TEST-26, TEST-32, TEST-33, TEST-34; TEST-42; TEST-50; TEST-44, TEST-46	Direct API requests, D-13 order (session, role, ownership). Verified: listed AC pass at `c1ee1ce` (results (Part F)). Regress in S2/S3.

Requirement	Feature or responsibility	Verification sprint	Existing planned AC	Existing planned TEST / method	Coverage / remaining work
NFR-04 20 participants/capacity 5 -> exactly 5 ACTIVE	F-04	S1	AC-37; AC-15 is the different last-place case	TEST-37; TEST-15 stays separate	Verified: TEST-37 passed in the full run and 5 of 5 extra runs (results (Part F) sections 1 and 3).
NFR-05 List p95 < 500 ms / 1,000 events	F-13; F-03 list responsibility	S3	Pending S3 contract	Performance script planned; no ID assigned	Agree sampling/warm-up/concurrency; retain SRS numeric target.
NFR-06 Data survives restart	Foundation/persistence; F-11, F-13 and persisted S1 journeys	S1 local; Lab 5 deployed; S3 regression	No dedicated AC in current skeleton	Planned restart-and-retrieve check in 9.4 QA; deployment repeat	Verified locally: event and registration identical before and after a server restart (results (Part F) section 3). Deployed repeat is Lab 5.
NFR-07 Server validation/uniform safe errors	F-01, F-02, F-04; every API feature	S1; regress S2/S3	AC-02, AC-07, AC-13, AC-14, AC-16, AC-18, AC-26, AC-27, AC-28, AC-29, AC-30, AC-35, AC-36 (samples); AC-41 (login validation); AC-51; AC-44, AC-46, AC-48	Matching numbered tests; TEST-41; TEST-51; TEST-44, TEST-46, TEST-48	Listed tests assert the exact error shape and keys. Verified: listed AC pass at `c1ee1ce` (results (Part F)). Regress in S2/S3.
NFR-08 README setup in <= 10 commands	Foundation/README: Issue #3 and its fresh-clone review	S1	No dedicated AC in current skeleton	Planned fresh-clone command-count check; repeat in 9.4 QA	Verified: fresh clone in 8 commands at `c1ee1ce`, recorded on Issue #3 (results (Part F) section 3). No invented feature/test ID.
NFR-09 Five S1 screens: loading/empty/error	F-02, F-03, F-04, F-05, F-06; F-13 hardening later	S1; S3 hardening/regression	AC-11 covers only empty published list	TEST-11 plus planned five-screen manual checklist in 9.4 QA	Verified: 14 of 14 manual checks with screenshots at `827f874` (results (Part F) section 3.1). The UI PRs #76 and #77 changed these screens afterwards; the rerun on the release commit `12dcd3c` passed 14 of 14 (results section 5).

5. Supplemental UI/end-to-end coverage

These journeys supplement service/API/DB checks; they do not replace negative authorization or concurrency tests.

AC / TEST	Journey	Features / requirements supplemented	Ownership	Result
AC-22 / TEST-22	Organizer creates/publishes; participant registers and sees it in My registrations; owner sees registrant.	F-01 through F-06; FR-01, FR-04, FR-06, FR-07, FR-08, FR-15; persisted UI journey	A, sub-task 9.1 (#37)	pass (E2E 5 of 5)
AC-23 / TEST-23	Full-event UI rejection; My registrations unchanged.	F-03, F-04, F-05; FR-06, FR-07, BR-02	B, sub-task 9.2 (#38)	pass (E2E 5 of 5)
AC-24 / TEST-24	Login shows name/role; logout then protected navigation returns to login.	F-01; FR-01, FR-02, FR-03 UI identity only	A, sub-task 9.1 (#37)	pass (E2E 5 of 5)

6. Coverage gaps and planned closure

Every FR/BR/NFR has a feature or named responsibility and sprint above; each S1 row starts with `Closed:` or `Open:`, and the later-sprint row with `Planned (S2/S3):`; closed rows are kept so the history stays visible. One row is `Open:`: the server-side 500 error check. No baseline requirement is removed to close a gap.

Gap / limitation	Affected scope	Planned closure / owner
S1 feature contracts are merged (Issue #2); executable tests were absent.	All S1 mappings	Closed: Issues 4 to 8 wrote the tests; 51 of 51 AC pass.
Published-event detail and current-user response.	FR-05; FR-03	Closed: AC-49/TEST-49 and AC-39/TEST-39 pass (results section 2).
Representative role/ownership tests are not the full route matrix; invalid/expired sessions, wrong-role publication and foreign-DRAFT registration-list cases need explicit coverage.	FR-04, FR-08; BR-01, BR-05; NFR-01, NFR-03; D-13	Closed: AC-38, AC-40, AC-42, AC-43, AC-45, AC-47 (sessions), AC-50 (wrong-role publication, D-13 order) and AC-48 (foreign DRAFT list) pass (results section 2).

Gap / limitation	Affected scope	Planned closure / owner
Existing boundary samples do not explicitly cover every approved bound.	BR-01 capacity maximum; BR-04 `startsAt == now`; BR-09 publish equality	Closed: AC-06/TEST-06 (capacity 10,000), AC-28/TEST-28 (`startsAt == now`) and AC-30/TEST-30 (publish equality) pass (results section 2).
The capacity-5 concurrency scenario was specified but not executed.	NFR-04, BR-02	Closed: QA #40 ran TEST-15 and TEST-37 five times each, all pass.
Hash-cost/response privacy and uniform safe-error assertions need complete contract coverage.	NFR-02, NFR-07	Open: no automated test that an unexpected server error returns 500 in the uniform shape without a stack trace. AC-05, AC-19, AC-39 (hash cost, no private fields) and the exact error keys asserted in the 4xx tests pass (results section 2).
Restart, setup count and five UI-state checks were planned only.	NFR-06, NFR-08, NFR-09	Closed: NFR-06, NFR-08 and NFR-09 checks pass (results section 3); the NFR-06 deployed repeat is Lab 5.
Later-sprint contracts and measurement parameters are intentionally not fully elaborated yet.	FR-09..FR-14, BR-06..BR-08, NFR-05; later changes to existing rules	Planned (S2/S3): contracts before implementation; resolve deadline/check-in/no-show/state boundaries and performance sampling.
Release evidence on the tag did not exist yet.	All S1 rows	Closed: Issue 10.1 (#41) reran build, `npm test` 67 of 67, E2E 5 of 5 and NFR-09 14 of 14 on `12dcd3c`, the commit of `v0.1.0-sprint1` (results section 5).

7. Approved changes and retained decisions

This table records changes already visible in the merged baseline; it does not approve new scope in this traceability draft. Preserve source history rather than renumber IDs.

Source / recorded evidence	Change or clarification	Traceability effect
Kickoff partner decisions, `docs/decisions.md` D-08/D-10	Server-side session revocation replaces cookie deletion/JWT assumptions; lock the event row before count/insert.	FR-02 / AC-04 / TEST-04 and BR-02 / AC-15 / TEST-15 retain their IDs and require the agreed behavior.
PR #11, merge `5169fb2`; D-07, D-12, D-13, D-14	Real DRAFT/publication and private-DRAFT access; FR-15 own-event list; future-start rule BR-09; exact errors, split registration negatives; integration-test isolation.	Preserve added FR-15, BR-09 and AC/TEST-25..36. AC-16 remains wrong-role; AC-27/28 distinguish DRAFT/missing and started targets.
PR #47, merge `b9db4f4`; revised D-14 and workflow	E2E uses its own `DATABASE_URL_E2E`; shared detail contract owned by F-02; QA/build/NFR checks, release-before-final-evidence and preserved merge/TDD history clarified.	Keep tests isolated; route FR-05 detail elaboration to #18; retain named NFR QA responsibilities and evidence from the tagged release. No feature is removed.
PR #49, merge `b4477c1`; review fix `0ae053b`	D-13 checks session, then role, then ownership; wrong-role organizer-route calls return 403 for every event ID. NFR-04 uses separate TEST-37.	AC-21 / TEST-21 wording clarified; TEST-15 remains last-place; TEST-37's full AC/STS is still #20 work.
SRS PR #48, merge `bf728a2`	Whole-product requirements documented, retaining the approved current-user fields; review aligns D-13 precedence, E2E isolation and TEST-37 references.	Requirement IDs remain FR-01..15, BR-01..09 and NFR-01..09; planned verification is not execution evidence.
Feature inventory PR #50, merge `79e0dcd`	F-01..F-13 allocated with dependencies and cross-cutting responsibilities.	S1 F-01..F-06; S2 F-07..F-09; S3 F-10..F-13 (F-10 starts S2). S1 NFR duties stay in S1.
Issue #2 contract PRs #52, #53, #54, #55, #56	S1 contracts add AC/TEST-37..51: current user, expiry, validation and guards (F-01, 39..42); F-03 session cases (38); F-04 capacity-5, session and input cases (37, 43, 44); F-05/F-06 session, role, input and DRAFT cases (45..48); F-02 detail, access matrix and validation (49..51).	New IDs are added to the rows above; no ID is renumbered. Recorded here as the A-owned follow-up agreed in PR #53 and #54.
QA PR #72, Issue #40	S1 run on staging `c1ee1ce`: build, `npm test` 67 of 67, E2E 5 of 5, TEST-15/37 five runs; NFR-06/08/09 checks added.	S1 rows above now state verified results with a link to the results file.
D-15, D-16; Issue #73, PRs #76 and #77	Scope addition: one shared Y2K visual style for every S1 screen, a decorative 3D layer, self-hosted fonts and Web Audio sounds. F-03 Events list shows CD cases (contract v1.1).	No FR/BR/NFR, route, API or data change; no new AC/TEST ID. Existing tests still pass (`npm test` 67 of 67 at merge `934dc51`, recorded on Issue #75).

8. Sources and updating actual results

Use the SRS (Part A), feature inventory (Part B), decisions (Part A), system SDS (Part C), sprint plan (Part D), Issue plan (team-internal file `issue-plan.md`, repository) and team workflow (team-internal file `team-workflow.md`, repository) with the master-plan skeleton linked above. Issue #14 owns this matrix; contract authors update links when specifications are elaborated. Do not invent a test file or passing count to fill an empty cell.

During implementation/QA, add actual test file/name and result to the single results artifact `docs/evidence/ac-test-results.md`, preserving links back to this baseline. Record automated vs manual verification, command, tested commit,

pass/fail/skip counts and incomplete coverage. The Product Definition of Done and release evidence determine completion; a populated planning matrix does not.

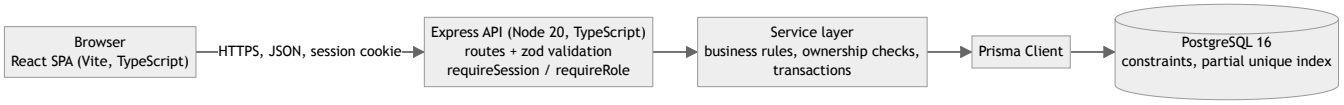
Part C. System-Level SDS

System-Level Software Design Specification (SDS)

Event Registration and Attendance System, CPE 334 group project, Theme 3. Covers the whole product; Sprint 1 parts are marked **S1**, later parts **S2** or **S3**. Requirement IDs (FR, BR, NFR) come from `docs/srs.md`; decisions (D-xx) from `docs/decisions.md`; features (F-xx) from `docs/features.md`. Detailed endpoint behavior belongs to each feature contract in `docs/sprints/sprint-1/`.

1. Architecture and stack

1.1 Components



Component	Responsibility	Technology (D-11)
UI	Screens, forms, client-side input hints, loading/empty/error states. Holds no business rules and no authority over identity or role.	React 18, TypeScript, Vite, React Router
API routes	HTTP mapping, request validation (zod), authentication and role guards, JSON error shape.	Express 4, TypeScript, zod
Service layer	All business rules (BR-01..BR-09), record ownership, state transitions, transactions and row locks.	TypeScript modules, one per feature area
Data access	Typed queries, migrations, seed.	Prisma 5
Database	Durable storage and last-line integrity: foreign keys, CHECK constraints, unique and partial unique indexes.	PostgreSQL 16

A modular monolith: one Node process serves the API and, in production, the built UI. One database. No microservices.

1.2 Communication

- The UI calls the API with `fetch` on relative paths under `/api`, sending the session cookie (`credentials: 'include'`).
- **Development:** Vite dev server proxies `/api` to Express, so the browser sees one origin.
- **Production:** Express serves the built UI files and the API from the same origin. No CORS is needed and the cookie stays first-party.

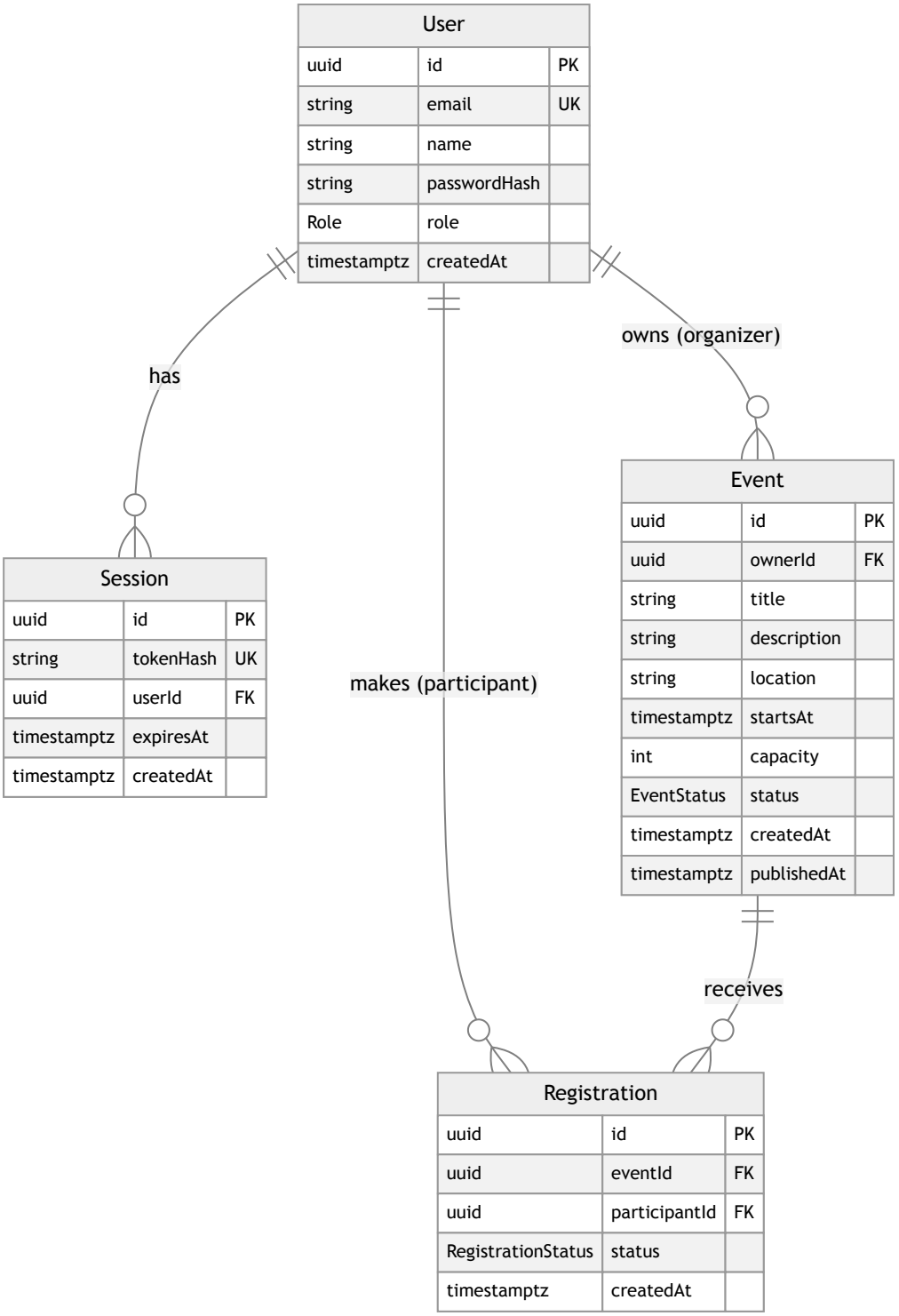
1.3 Where rules are enforced

Rule	Enforced in	Backstop
Authentication (NFR-01)	`requireSession` middleware on every protected route	none needed: no session, no handler
Role (BR-01, NFR-03)	`requireRole` middleware	service functions take the caller and recheck role
Ownership (BR-05, BR-09 publish, D-13)	service layer, comparing the record owner with the session user	queries are scoped by owner where possible
Capacity (BR-02)	service layer, inside one transaction with `SELECT ... FOR UPDATE` on the event row (D-10)	`capacity` CHECK 1..10000
One active registration (BR-03)	service layer check inside the same transaction	partial unique index `(eventId, participantId) WHERE status = 'ACTIVE'`
Registrable event (BR-04)	service layer: status PUBLISHED and `startsAt > now`	none
Valid start time (BR-09, D-12)	service layer on create and publish	none
Input shape	zod schema at the route	DB types and NOT NULL

The UI may hide controls a user cannot use, but hiding is only a convenience; every rule above holds when the API is called directly.

2. Data model

2.1 Entities (S1)



Entity	Important fields and constraints	Ownership
User	`email` unique, lowercase; `passwordHash` bcrypt, never returned by the API (NFR-02); `role` enum `PARTICIPANT` or `ORGANIZER`, one per account (D-09)	itself
Session	`tokenHash` = SHA-256 of the cookie token, unique; `expiresAt` = creation + 1 h (D-08); index on `expiresAt`; cascade delete with its user	its user
Event	`title` 1..120 chars, `location` 1..120, `description` 0..2000; `capacity` integer, CHECK 1..10000 (D-03); `startsAt` timestamptz; `status` enum `DRAFT`, `PUBLISHED` (S1), `CLOSED`, `CANCELLED` (S2); `publishedAt` set on publish	`ownerId` = organizer who created it

Entity	Important fields and constraints	Ownership
Registration	`status` enum `ACTIVE` (S1), `CANCELLED` (S2); partial unique index on `(eventId, participantId) WHERE status = 'ACTIVE'` (D-02)	`participantId` = participant who registered; the event owner may read it

Derived value: `remainingPlaces = capacity - count(Registration WHERE eventId = e AND status = 'ACTIVE')`. It is computed on read, never stored, so it cannot drift from the registrations.

Foreign keys from Event and Registration use `ON DELETE RESTRICT`: S1 has no delete operations, and history must not disappear silently.

2.2 Later sprints (planned)

Sprint	Change	For
S2	Registration `CANCELLED` status and `cancelledAt`; Event `registrationDeadline`; Event `CLOSED`, `CANCELLED`	F-07, F-08, F-09
S3	Check-in record per ACTIVE registration (`checkedInAt`, `checkedInById`), unique per registration	F-10, F-11, F-12

Each change is a new Prisma migration; existing migrations are never edited.

2.3 Migrations and seed

- Schema lives in `server/prisma/schema.prisma`; migrations in `server/prisma/migrations/`, created with `prisma migrate dev` and applied in deployment with `prisma migrate deploy`.
- Prisma 5 cannot declare CHECK constraints or partial indexes, so the first migration adds them with raw SQL (`capacity` CHECK, the ACTIVE partial unique index). Issue 3 verifies that `prisma migrate dev` reports no drift afterwards.
- Seed (`npm run seed`) is repeatable: it clears the tables and inserts synthetic data:
- 2 organizers and 4 participants (`@example.com` addresses), all with the password from `SEED_PASSWORD`;
- organizer 1: one PUBLISHED event with capacity 30, one PUBLISHED event with capacity 2 and one ACTIVE registration (one place left, for the full-event demo), one DRAFT event;
- organizer 2: one PUBLISHED event (for the cross-owner checks).
- Every seeded `startsAt` is relative to the time the seed runs (for example now + 7 days), never a fixed date, so seeded events stay registrable (D-14).

3. UI structure

3.1 Screens and navigation (S1)

Screen	Route	Who	Data	Feature
Login	`/login`	anonymous	`POST /api/auth/login`	F-01
Events	`/events`	any signed-in user	`GET /api/events`	F-03
Event detail	`/events/:id`	any signed-in user (draft: owner only)	`GET /api/events/:id`; Register button for participants	F-03, F-04
My registrations	`/my/registrations`	participant	`GET /api/registrations/mine`	F-05
My events	`/organizer/events`	organizer	`GET /api/events/mine`; Publish button on drafts	F-02
Create event	`/organizer/events/new`	organizer	`POST /api/events`	F-02
Event registrations	`/organizer/events/:id/registrations`	owning organizer	`GET /api/events/:id/registrations`	F-06

The header shows the signed-in user's name and role (from `GET /api/me`) and a Logout button. Navigation links depend on the role. A participant who types an organizer URL sees the API's 403 message; the route guard in the UI is only for convenience.

3.2 Shared behavior (NFR-09)

State	Behavior
Loading	Spinner or "Loading..." text in the content area while a request is pending
Empty	A plain message for an empty list, for example "No published events yet"
Error	The API `error.message` in an alert; 401 redirects to `/login`; 403 shows "You do not have access to this"; 404 shows "Not found"
Validation	Inline message under each field, from the API `details` of a 400 response; the form keeps the user's input
Submitting	The submit button is disabled while its request is pending, so a double click cannot send twice
Success	A short confirmation, and the list or detail reloads from the API so the screen shows saved data

4. API conventions

4.1 Rules

- REST over HTTPS, JSON bodies, prefix `/api`, plural nouns, camelCase fields.
- IDs are UUIDs. Times are ISO 8601 in UTC (`2026-11-20T09:00:00.000Z`); the UI shows local time.
- Every request body is validated with a zod schema before the service is called. Unknown fields are rejected.
- State-changing requests are `POST` with `Content-Type: application/json`; other content types get 400.
- Responses never contain `passwordHash`, session tokens or token hashes.
- Lists are small in S1 and not paginated. NFR-05 (S3) revisits this.

4.2 Error shape and status codes (NFR-07, D-13)

```
{ "error": { "code": "EVENT_FULL", "message": "This event has no places left.", "details": [] } }
```

`details` appears only for validation errors: `[{ "field": "capacity", "message": "Must be an integer from 1 to 10000" }]`.

Status	Codes	When
400	`VALIDATION_ERROR`, `INVALID_START_TIME`	body or parameter invalid; start time not strictly in the future
401	`UNAUTHENTICATED`, `INVALID_CREDENTIALS`	no valid session; wrong email or password (one generic message)
403	`FORBIDDEN`	wrong role on a role-restricted route (for any event ID), or a non-owner organizer on an owner-only operation of a PUBLISHED event
404	`NOT_FOUND`, `EVENT_NOT_FOUND`	record missing, a DRAFT requested by anyone except its owner (when the caller has the route's role), or another participant's registration
409	`EVENT_FULL`, `ALREADY_REGISTERED`, `EVENT_STARTED`, `ALREADY_PUBLISHED`	a business rule rejects the request; nothing is stored
500	`INTERNAL_ERROR`	unexpected failure; generic message, details logged on the server only, no stack trace

4.3 Endpoint inventory (S1)

The feature contracts define request, response and error details. This table fixes paths and owners so contracts do not overlap.

Method and path	Purpose	Role	Feature	Main AC
`POST /api/auth/login`	sign in, set session cookie	anonymous	F-01	AC-01, AC-02
`POST /api/auth/logout`	delete session, clear cookie	any signed-in	F-01	AC-04
`GET /api/me`	current user (id, name, role; FR-03)	any signed-in	F-01	AC-03
`POST /api/events`	create a DRAFT event	organizer	F-02	AC-06..AC-09, AC-29, AC-35
`GET /api/events/mine`	own events, every status	organizer	F-02	AC-33, AC-34
`GET /api/events/:id`	one event; DRAFT only for its owner	any signed-in	F-02 (defines), F-03 (uses)	AC-31, AC-32
`POST /api/events/:id/publish`	DRAFT to PUBLISHED	owning organizer	F-02	AC-25, AC-26, AC-30, AC-36
`GET /api/events`	PUBLISHED events with `remainingPlaces`	any signed-in	F-03	AC-10, AC-11
`POST /api/events/:id/registrations`	register the caller	participant	F-04	AC-12..AC-16, AC-27, AC-28
`GET /api/registrations/mine`	caller's registrations with event summary	participant	F-05	AC-17
`GET /api/registrations/:id`	one registration	its participant	F-05	AC-18
`GET /api/events/:id/registrations`	registrations of an event	owning organizer	F-06	AC-19..AC-21

Express route order: `/api/events/mine` is declared before `/api/events/:id`.

5. Roles and permissions

5.1 Role/permission matrix (S1)

Columns: **Anon** = no session; **Part** = participant; **Org own** = organizer who owns the event; **Org other** = organizer who does not.

Operation	Anon	Part	Org own	Org other
Log in	allowed	n/a	n/a	n/a
Log out, read `/api/me`	401	allowed	allowed	allowed
List PUBLISHED events	401	allowed	allowed	allowed
Read a PUBLISHED event	401	allowed	allowed	allowed
Read a DRAFT event	401	404	allowed	404
Create event (as DRAFT)	401	403	allowed (becomes owner)	allowed (becomes owner)
List own events (`/mine`)	401	403	allowed (own only)	allowed (own only)
Publish a DRAFT	401	403	allowed	404
Register for an event	401	allowed if BR-02..BR-04 hold	403	403
List own registrations	401	allowed (own only)	403	403
Read one registration	401	own: allowed; another participant's: 404	403	403
View an event's registrations	401	403	allowed	403 (PUBLISHED), 404 (DRAFT)

Check order on every protected route: session (401), then role (403), then existence and ownership (404 for a DRAFT or a foreign registration, 403 for a non-owner on a PUBLISHED event). So a participant who calls an organizer-only route gets 403 for every event ID, DRAFT or not, as AC-21 expects; the same answer for every ID reveals nothing. The DRAFT 404 rule applies to organizers who do not own the event (D-13).

5.2 Record access

Record	Who can read it	Who can change it (S1)
User	the user (via `/api/me`); the owning organizer sees registrants' name and email only	nobody (seeded)
Event (DRAFT)	its owner	its owner (publish)
Event (PUBLISHED)	every signed-in user	nobody in S1 (edit, close, cancel are S2)
Registration	its participant; the event's owner	created by its participant; cancel is S2

Identity and role always come from the session on the server. The API never accepts a user ID or role from the client for authorization.

6. Authentication and session lifecycle (D-08)

Step	Behavior
Credentials	Email and password. Passwords are stored only as bcrypt hashes, cost 12 (NFR-02 requires at least 10). Seed passwords come from `SEED_PASSWORD`, never from Git.
Login	`POST /api/auth/login`: validate body, find the user by lowercase email, compare with bcrypt. Unknown email and wrong password both return 401 `INVALID_CREDENTIALS` with the same message; an unknown email is still compared against a fixed dummy hash so response time does not reveal which accounts exist.
Session creation	Generate 32 random bytes (`crypto.randomBytes`), encode as base64url. Store only `SHA-256(token)` in `Session` with `expiresAt = now + SESSION_TTL_MINUTES` (60).
Cookie	`sid=<token>` ; `HttpOnly` , `SameSite=Lax` , `Path=/` , `Max-Age=3600` , `Secure` when `COOKIE_SECURE=true` (production). The token is never readable by JavaScript and never in a response body.
Each request	`requireSession` hashes the cookie token, loads the session with its user, rejects it if missing or `expiresAt ≤ now` (401, and the expired row is deleted), then sets `req.user` . Expiry is absolute: activity does not extend it.
Current user	`GET /api/me` returns `{ id, name, role }` from `req.user` (FR-03).
Logout	`POST /api/auth/logout` deletes the session row and clears the cookie. Replaying the old cookie afterwards returns 401 (AC-04).
Role guard	`requireRole('ORGANIZER')` or `requireRole('PARTICIPANT')` after `requireSession` ; wrong role returns 403 `FORBIDDEN` .

CSRF stance. The cookie is `SameSite=Lax` , so browsers do not send it on cross-site `POST` requests. Every state-changing request is a `POST` that must carry `Content-Type: application/json` , which a cross-site HTML form cannot send without a CORS preflight, and the API allows no cross-origin requests. Together these block cross-site request forgery in S1 without a CSRF token. A token is reconsidered if the API ever accepts cross-origin requests.

Not in S1 (known limitations): self-registration, password reset, login rate limiting, multiple roles per account. Rate limiting is planned for S3 hardening (F-13).

7. Error handling and logging

- One Express error handler turns known errors (`AppError` with status and code) into the JSON shape in 4.2.
- Unknown errors become 500 `INTERNAL_ERROR` ; the stack trace is logged on the server and never sent (NFR-07).

- Request logs record method, path, status and duration. Passwords, cookies and tokens are never logged.

8. Configuration and secrets

Variable	Used by	Example in `.env.example`	Secret
`DATABASE_URL`	server, Prisma	`postgresql://app:app@localhost:5432/eventreg`	yes in production
`DATABASE_URL_TEST`	integration tests (D-14)	`postgresql://app:app@localhost:5432/eventreg_test`	no (local only)
`DATABASE_URL_E2E`	Playwright E2E (D-14)	`postgresql://app:app@localhost:5432/eventreg_e2e`	no (local only)
`PORT`	server	`3000`	no
`NODE_ENV`	server	`development`	no
`SESSION_TTL_MINUTES`	session lifetime	`60`	no
`COOKIE_SECURE`	cookie `Secure` flag	`false` (local), `true` (production)	no
`SEED_PASSWORD`	seed script	`(local value, see .env.example)`	yes

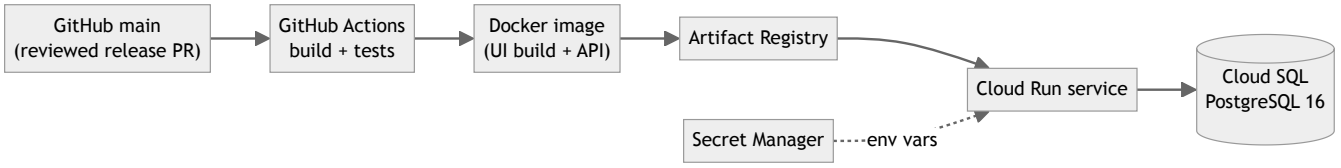
- Only `.env.example`, with placeholder values, is committed; `.env` and `.env.*` are gitignored.
- No secret is placed in client code, docs, screenshots or the PDF.
- In production, secrets come from GCP Secret Manager as environment variables (Lab 5).
- Local PostgreSQL runs from `docker-compose.yml` with three databases: dev, test and E2E.

9. Test architecture

Level	Tool	Database	Covers
Unit	Vitest	none	pure rules: start-time check with an injected `now`, capacity input validation
API/DB integration	Vitest + Supertest	`DATABASE_URL_TEST`, files run one at a time, tables truncated in `beforeEach` (D-14)	persistence, permissions, ownership, error codes, concurrency (TEST-15 last place, TEST-37 NFR-04 scenario)
End-to-end	Playwright	`DATABASE_URL_E2E`, reset and seeded before each run	TEST-22..TEST-24 critical journeys

Test names start with their TEST and AC IDs, for example `TEST-13 AC-13 rejects registration when event is full`.

10. Deployment design (planned, Lab 5)



- **Image:** one multi-stage Dockerfile builds the UI with Vite and the server with `tsc`, then runs Node with only production dependencies. Express serves the UI and the API, so one container is enough.
- **Database:** Cloud SQL for PostgreSQL 16 keeps data across restarts and redeployments (NFR-06). Migrations run with `prisma migrate deploy` as a release step before traffic moves to the new revision.
- **Configuration:** the variables in section 8 are set on the Cloud Run service; secrets come from Secret Manager; `COOKIE_SECURE=true`.
- **Pipeline:** PRs run build and tests; a failed required check blocks the merge. Merges to `main` build the image and deploy it, with a manual approval step if needed.
- The exact GCP access method follows the Lab 5 instructions; Sprint 1 only has to be Docker-ready (no host paths, configuration from environment variables, stateless server).

11. NFR design mapping

NFR	Design element	Section
NFR-01 unauthenticated → 401, no private data	`requireSession` on every protected route	1.3, 6
NFR-02 bcrypt cost ≥ 10, hash never returned	bcrypt cost 12; response mappers omit `passwordHash`	2.1, 6
NFR-03 backend role and ownership checks	`requireRole` + service ownership checks; matrix	1.3, 5
NFR-04 20 parallel requests, capacity 5 → exactly 5 ACTIVE	row lock transaction (D-10); verified by TEST-37	1.3, 9
NFR-05 list p95 < 500 ms with 1,000 events (S3)	indexed `status`, `startsAt`; pagination if needed	4.1
NFR-06 data survives restart	PostgreSQL volume locally, Cloud SQL in production	8, 10

NFR	Design element	Section
NFR-07 server validation, uniform errors, no stack traces	zod at routes, one error handler	4.2, 7
NFR-08 runs from README in ≤ 10 commands	docker compose + npm scripts	2.3, 8
NFR-09 loading, empty, error states	shared UI behavior	3.2

12. Version history

Version	Date	Change	Author	Reviewer
1.0	2026-11-07	First System-Level SDS for Deliverable 1 (Issue #15)	B (stickkersz)	A (songt888)
1.1	2026-11-07	PR #49 review: `/api/me` fields match FR-03; check order and D-13 precedence made explicit; NFR-04 verified by TEST-37	B (stickkersz)	A (songt888)

Part D. Three-sprint plan

Three-Sprint Plan

Event Registration and Attendance System, CPE 334 group project, Theme 3. Feature IDs from `docs/features.md`; requirement IDs from `docs/srs.md`. Owners and order of Sprint 1 work: `docs/sprints/sprint-1/issue-plan.md`.

1. Sprint overview

Sprint	Dates	Goal	Features	Checkpoint
1	6 to 8 Nov 2026	A small working domain slice: real login with two roles, an organizer publishes an event, a participant registers within capacity, both see the persisted result, with backend permissions and tests	F-01, F-02, F-03, F-04, F-05, F-06	Deliverable 1, 8 Nov; tag `v0.1.0-sprint1`
2	9 to 29 Nov 2026	Core expansion: participants cancel, organizers manage their events, registration deadlines; check-in started	F-07, F-08, F-09 (F-10 started)	Deliverable 2, 29 Nov; deployed release candidate
3	9 Nov to 13 Dec 2026 (overlaps S2, finishes after exams)	Attendance and reporting, then hardening: check-in, attendance record and report, organizer summary, performance, usability, regression	F-10, F-11, F-12, F-13	Progress at 29 Nov; final release 13 Dec

Labs 5 to 7 run on the same product: Lab 5 deploys the Sprint 1 release (15 Nov), Lab 6 is cross-team UAT (22 Nov), Lab 7 is a maintenance change (29 Nov). Their work is planned inside Sprints 2 and 3 but kept as separate Issues and PRs.

2. Sprint 1: small working slice

2.1 Features and outcome

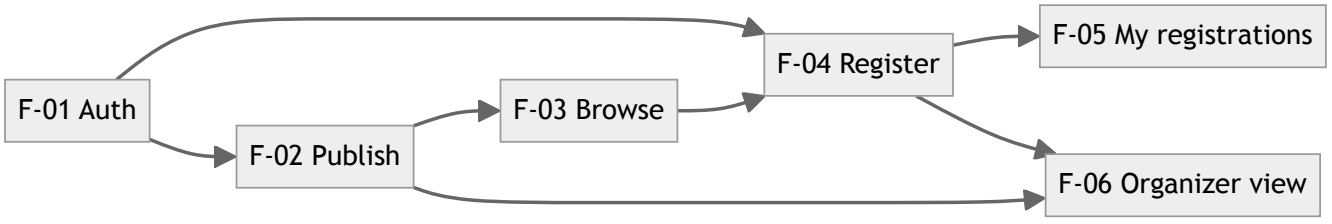
Feature	Covers	Completion outcome
F-01 Authentication foundation	FR-01, FR-02, FR-03; NFR-01, NFR-02, NFR-03, NFR-07	Seeded users log in and out; `/api/me` returns the user; protected routes return 401 without a session and 403 for the wrong role
F-02 Publish event	FR-04, FR-15; BR-01, BR-09	An organizer creates a DRAFT, sees it in My events and publishes it; invalid capacity or start time is rejected; non-owners cannot publish or read drafts
F-03 Browse events	FR-05	Signed-in users see PUBLISHED events with remaining places; drafts never appear
F-04 Register for event	FR-06; BR-02, BR-03, BR-04; NFR-04	A participant registers; full, duplicate, draft and started events are rejected; parallel requests for the last place give exactly one success
F-05 My registrations	FR-07; BR-05	A participant sees their own registrations and cannot read another participant's
F-06 Event registrations (organizer)	FR-08; BR-05	An organizer sees who registered for their own event and nobody else's

Sprint 1 is done when all six features meet the Product Definition of Done, the full suite passes on the release commit, and the reviewed release PR is merged to `main` and tagged.

2.2 Why this scope

- It is the theme's required Sprint 1 slice: publish with positive capacity, register, view own registration, organizer views their event's registrations, with capacity, one active registration and the last-place race enforced.
- It proves every layer once (UI, API, service, database) and the authentication and authorization foundation that every later feature reuses.
- It contains the two hardest rules early (capacity under concurrency, ownership), so later sprints build on tested ground.
- The DRAFT state (D-07) is the one real state transition in Sprint 1. Cancellation, deadlines and check-in are left out because each adds new states and rules that need their own contracts; they are not needed to prove the slice.
- Six small features fit two students in three days because each is one vertical slice with a contract written first; the plan in `issue-plan.md` runs them in two parallel lanes.

2.3 Dependencies



All Sprint 1 features also depend on Issue 3 (project skeleton, schema, seed, test runner). F-02 can be written before F-01 is merged; only its routes wait for `requireSession` and `requireRole`.

2.4 NFRs established and verified in Sprint 1

NFR	Verified by	When
NFR-01, NFR-03	API tests for 401, 403 and 404 on every protected route	each feature PR, again in the 9.4 QA run
NFR-02	DB test of the stored hash, response tests	F-01
NFR-04	TEST-37: 20 eligible participants register in parallel for a capacity-5 event, exactly 5 ACTIVE stored (specified in the F-04 contract, #20). TEST-15 (last place, exactly one success) stays as a separate case. Both run five times	F-04, 9.4 QA run
NFR-06	restart the server, read the same data	9.4 QA run
NFR-07	error-shape tests	each feature PR
NFR-08	fresh clone following the README, commands counted	Issue 3 review, 9.4 QA run
NFR-09	E2E TEST-11 + checklist on 5 screens	9.4 QA run

Scope addition (D-15, D-16, Issue #73): a shared Y2K visual style for all S1 screens, with visible focus, AA contrast and a reduced-motion version, is added to Sprint 1 and merged before the release test run (10.4). It changes no feature and no existing text or test.

3. Sprint 2: core expansion

Feature	Covers	Depends on	Completion outcome
F-07 Cancel registration	FR-09, BR-07	F-04, F-05	A participant cancels their ACTIVE registration; the place is freed and they can register again (BR-07); exact time limit set in the S2 contract
F-08 Manage event	FR-10; D-03	F-02, F-06	The owner edits details, changes capacity (never below current ACTIVE count), closes or cancels the event; cancelled events reject registration
F-09 Registration deadline	FR-11, BR-06	F-02, F-04	Registration after the event's deadline is rejected (BR-06); deadline rules set in the S2 contract
F-10 Check-in (start)	FR-12, BR-08	F-06	Contract written and the check-in rule implemented and tested if time allows; finished in Sprint 3

Sprint 2 also carries Lab 5 (Docker, CI/CD, GCP deployment of the Sprint 1 release), so every Sprint 2 PR runs through CI. Outcome by 29 Nov: all Sprint 2 features merged, reviewed and deployed in a release candidate on GCP.

4. Sprint 3: completion and hardening

Feature	Covers	Depends on	Completion outcome
F-10 Check-in	FR-12, BR-08	F-06, F-07	The owning organizer checks in an ACTIVE registration once, only within the event window
F-11 Attendance record and report	FR-13	F-10	Per-event attendance list and counts (registered, checked in, no-show)
F-12 Organizer summary dashboard	FR-14	F-11	Organizer sees totals across their events
F-13 Hardening, UX, regression	NFR-05, NFR-06, NFR-09; login rate limiting	all	NFR-05 measured at p95 < 500 ms with 1,000 events; full regression suite; UAT defects from Lab 6 fixed and re-tested

Outcome: Sprint 3 contracts and working progress shown at Deliverable 2 (29 Nov) with an explicit remaining-work list; complete product released and deployed by 13 Dec.

5. Coverage check

Every baseline feature F-01 to F-13 appears in exactly one sprint as its completion sprint (F-10 starts in Sprint 2 and completes in Sprint 3). Every FR, BR and important NFR maps to at least one of these features in `docs/traceability.md`. Scope changes are recorded there and in `docs/decisions.md`, never by silently moving a feature.

6. Risks

Risk	Effect	Response
Three days for Sprint 1	features not finished	two parallel lanes; UI polish is cut first, never backend tests or contracts
Concurrency is hard to test reliably	flaky TEST-15 or TEST-37	row lock design (D-10), sequential integration files (D-14), five repeated runs

Risk	Effect	Response
Labs 5 to 7 compete with Sprint 2 time	Sprint 2 late	lab work as separate small Issues; Sprint 2 contracts written in the week of 9 Nov
Final exams 1 to 11 Dec	Sprint 3 stalls	product substantially complete by 29 Nov; only bounded work left for December

7. Version history

Version	Date	Change	Author	Reviewer
1.0	2026-11-07	First three-sprint plan for Deliverable 1 (Issue #15)	B (stickkersz)	A (songt888)
1.1	2026-11-07	PR #49 review: NFR-04 verified by TEST-37, separate from TEST-15	B (stickkersz)	A (songt888)
1.2	2026-11-08	D-15 and D-16: shared Y2K UI style added to Sprint 1 (Issue #73)	B (stickkersz)	A (songt888)

Part E. Sprint 1 feature contracts and Product Definition of Done

The six Sprint 1 contracts (feature SDS, applicable UI, API and data specifications, acceptance criteria and the system test specification with TEST IDs) and the shared Product Definition of Done, each as merged in the repository.

F-01 — Authentication Contract

Sprint 1 / Deliverable 1, Issue #17 (parent #2). Author: A (`songt888`). Teammate reviewer: B (`stickkersz`). This contract specifies planned behavior; it records no executed application test or completed implementation.

1. Scope and sources

Covers FR-01, FR-02, FR-03 and NFR-01, NFR-02, NFR-03, NFR-07. Establishes the authentication/role guards used by BR-01 and BR-05 in other features; their record-ownership rules remain in their own contracts. Decisions D-08, D-09, D-11, D-13 and D-14 apply.

Sources: SRS, system SDS sections 4–6, decisions, traceability, master plan section 5 and team workflow.

Included: seeded-account login, current user, logout, absolute session expiry, backend session/role middleware and login/header UI. No event or registration data changes. Excluded: self-registration, password recovery, multiple roles, JWT, login rate limiting (planned F-13 hardening), and new account-management screens.

2. Behavior and workflow

1. Visitor opens `/login` and submits email/password. The API validates the request, looks up the lowercased email and compares the password with bcrypt. Unknown email still performs comparison against a fixed dummy bcrypt hash. Unknown email and wrong password return the same generic error.
2. Valid credentials generate a fresh opaque token from 32 random bytes, base64url encoded. Store only SHA-256(token) in a new Session row with the user ID and creation time; expiry is exactly creation + 60 minutes. Return public user fields and set the cookie. Login never accepts a caller-selected user ID or role.
3. Protected requests use `requireSession`: hash the cookie, load its Session and User, reject missing/invalid/revoked tokens and `expiresAt ⩽ now` with 401. Delete a found expired row. Successful requests use server-loaded identity/role. Activity does not extend expiry.
4. `GET /api/me` returns that user's ID, name and role. The UI uses it after reload to restore the signed-in header; it does not persist the token or authoritative role in browser storage.
5. `requireRole` runs after `requireSession`, before resource existence/ownership checks. Wrong role returns 403 for every target ID. Feature services enforce their own role/ownership rules using that caller, following D-13.
6. Logout deletes the current Session row, clears the cookie with matching path/security attributes, and returns no body. Replaying the old cookie immediately fails. It revokes this session, not every session belonging to the account.

A new login creates a distinct session; it does not reuse a caller-supplied token. No additional session-list or revoke-all operation is introduced.

3. API and validation

All responses exclude passwords, `passwordHash`, token hashes and session tokens from JSON. The cookie is the only token transport. State-changing requests use JSON and reject other content types with 400. Unknown body fields are rejected. Malformed JSON, non-object bodies and invalid fields return 400 `VALIDATION_ERROR` before credentials are checked; no Session is created. Do not trim or lowercase the password.

Method/path	Request	Success	Failure
`POST /api/auth/login`	JSON `{ "email": "participant1@example.com", "password": "<synthetic seed password>" }`; email must be a syntactically valid nonempty string, password a nonempty string; email lowercased for lookup	200 `{ "user": { "id": "<uuid>", "name": "<name>", "role": "PARTICIPANT" } }`, or ORGANIZER; sets `sid`	400 `VALIDATION_ERROR`; 401 `INVALID_CREDENTIALS`
`GET /api/me`	No body; valid `sid` cookie	200 `{ "id": "<uuid>", "name": "<name>", "role": "PARTICIPANT" }`, or ORGANIZER	401 `UNAUTHENTICATED`
`POST /api/auth/logout`	JSON `{}` and valid `sid` cookie	204, empty body; deletes Session and expires cookie	401 `UNAUTHENTICATED` without valid session; 400 for invalid JSON/body/content type after session check

Login is public. `/api/me` and logout allow either account role after authentication. Protected-route authentication precedes role, validation and resource handling; this preserves 401 → 403 → existence/ownership ordering. Login's request validation has no session prerequisite.

Errors use `{ "error": { "code": "...", "message": "..." } }`. Validation adds only `error.details: [{ "field": "email", "message": "..." }]` as applicable. Invalid credentials use the exact same message `Email or password is incorrect.` for both causes. Missing/invalid/expired/revoked session uses `UNAUTHENTICATED`; wrong role uses `FORBIDDEN`. Unexpected failures return 500 `INTERNAL_ERROR` and a generic message, with no stack trace or private fields; credentials, cookies and tokens are never logged.

Cookie: `sid`, `HttpOnly`, `SameSite=Lax`, `Path=/`, `Max-Age=3600`, `Secure` when `COOKIE_SECURE=true` (production). Expiry is also enforced on the server, regardless of browser cookie retention. Clear the cookie with Max-Age=0 on successful logout. Development uses the same-origin Vite proxy; production serves UI/API from one origin. Do not enable cross-origin requests. The JSON-only POST and SameSite policy follow the SDS CSRF stance; a separate CSRF token is not added in S1.

4. Data and migration contract

Entity	Required fields/constraints	Change owned by
User	UUID id, unique lowercase email, name, bcrypt passwordHash (cost 12, baseline minimum 10), one role enum PARTICIPANT/ORGANIZER, createdAt	Foundation #24 migration/seed; F-01 reads only
Session	UUID id, unique tokenHash, userId FK to User (cascade deletion), expiresAt, createdAt; expiresAt index	Foundation #24 schema/migration; F-01 creates, reads and deletes rows

Use PostgreSQL persistence and Prisma as specified in the SDS. No schema duplication in Issue #4 if foundation satisfies this contract. Seed only synthetic accounts; obtain seed passwords from `SEED_PASSWORD`, never commit a real password. F-01 makes no Event/Registration schema change. Config: SESSION_TTL_MINUTES=60, COOKIE_SECURE=false locally/true in production; DB configuration follows foundation. No plaintext token is stored in the database.

5. UI contract

State/action	Observable behavior
Login form	Labelled email and password fields; password masked; submit button; no self-registration/reset links promising unavailable features
Submitting	Disable submit and show a pending indication until completion; do not send twice from a repeated click
Validation/error	Show field validation and readable generic credential/API/network error; retain email, keep the password out of logs; allow retry
Success	Navigate to `/events`; header shows name/role returned by `/api/me`; participant and organizer links follow SDS navigation
Session bootstrap	Show loading while `/api/me` is pending; 401 clears local user display and opens `/login`; other failures show a readable retryable error
Logout	Wait for confirmed API success, clear displayed user, navigate to `/login`; an API/network failure is shown and must not be presented as successful server revocation
Protected navigation	Missing/expired/revoked session redirects to `/login`; wrong-role page shows the readable 403 message. UI hiding never substitutes for API guards

Empty-list states are no change in F-01; they belong to the five data-loading screens in their respective features. AC-24/TEST-24 covers the later end-to-end UI journey (#37), without replacing the API tests here.

6. Acceptance criteria

Existing AC-01..05 and AC-24 retain their original meaning. Additional IDs close explicit traceability gaps; AC-37/TEST-37 belong to B's F-04 NFR-04 scenario; AC-38/TEST-38 are already reserved by B's F-03 branch. Neither pair is defined here. The new IDs are proposed in this first contract for teammate review before implementation.

ID	Observable outcome	Requirements
AC-01	For either seeded role, valid credentials return 200 and public user fields, create a matching hashed Session with 60-minute absolute expiry and set the specified cookie; no sensitive JSON fields	FR-01; NFR-02
AC-02	Wrong password and unknown syntactically valid email return identical 401 INVALID_CREDENTIALS code/message; no Session created	FR-01; NFR-07
AC-03	GET /api/me without a cookie returns 401 UNAUTHENTICATED and no private data	FR-03; NFR-01
AC-04	Authenticated logout returns 204, deletes that Session and clears cookie; replaying the old cookie on /api/me returns 401	FR-02; NFR-01
AC-05	Synthetic seeded passwords are bcrypt hashes with cost at least 10 (configured 12), verify against the seed password and are never plaintext	NFR-02

ID	Observable outcome	Requirements
AC-39	Valid-session GET /api/me returns exactly the caller's id, name and role for either role; no email, password/hash, token/hash or another user's fields	FR-03; NFR-02
AC-40	Unknown/revoked or expired session is rejected with 401; expiry equality counts as expired and the found expired row is removed; successful activity before expiry does not extend it	NFR-01; D-08
AC-41	Invalid login request cases return 400 VALIDATION_ERROR with the safe error shape and no new Session; valid-shaped invalid credentials still use AC-02	NFR-07
AC-42	Shared guards reject anonymous callers with 401 before role handling; valid-session wrong role gets 403 before target lookup; client-supplied identity/role cannot authorize a request	NFR-01, NFR-03; BR-01/BR-05 foundation
AC-24	UI login shows name/role; logout followed by protected navigation returns to login	FR-01, FR-02, FR-03; cross-feature E2E

7. Software Test Specification (planned)

All API/DB tests use DATABASE_URL_TEST, with sequential integration files and truncation before each test (D-14). Never reset the dev database. Use a fixed `now` for service expiry boundaries; real API expiry tests may insert expired Session fixtures. TEST-24 runs separately against DATABASE_URL_E2E, reset/seeded before its run. Names contain TEST and AC IDs. Every result below is **not run: implementation/test runner pending Issues #3/#4**.

TEST / AC	Level	Preconditions	Action	Expected result
TEST-01 / AC-01	API + DB	Synthetic participant and organizer with bcrypt hashes; empty Session table	Login once per role with correct credentials	200 correct public user; cookie flags (Secure false/true config cases); 32-byte token decoded; DB contains only its SHA-256 hash, correct user FK and expiry = creation + 60 min; no sensitive JSON fields
TEST-02 / AC-02	API + DB	Known account and an absent email; Session count recorded	Send wrong password and unknown-email requests	Same 401 code/message, no sensitive fields/stack and unchanged Session count; inspect service/dummy-hash path, do not claim a timing benchmark
TEST-03 / AC-03	API	No session cookie	GET /api/me	401 UNAUTHENTICATED, uniform error, no private data
TEST-04 / AC-04	API + DB	Logged-in account; capture cookie and Session ID	POST logout with {}, inspect DB/cookie, replay captured cookie on /api/me	204 empty body, Session absent, sid expires at matching path, replay gets 401; another independently created session remains valid
TEST-05 / AC-05	DB + hash verification	Run synthetic seed with test-only password from environment	Inspect User hashes and bcrypt-compare test password	All hashes use bcrypt cost >=10 (seed config 12), compare succeeds, no stored plaintext password
TEST-39 / AC-39	API + DB	Valid sessions for two distinct users, covering both roles	GET /api/me with each cookie	Exactly id/name/role of corresponding user; sensitive fields and other user's data absent
TEST-40 / AC-40	Service unit + API/DB	Fixed now; sessions expiring before, exactly at, and after now; unknown/revoked token	Call guard and /api/me; make successful request before expiry then inspect expiry	Before/equal expiry rejected and found expired rows deleted; unknown/revoked rejected; future expiry succeeds and timestamp unchanged
TEST-41 / AC-41	API + DB	Session count recorded	Parameterize missing fields, empty/non-string fields, invalid email, extra userId/role, array/null body, malformed JSON and non-JSON Content-Type	400 VALIDATION_ERROR with safe shape; field details where applicable; no Session created; no stack/password reflected
TEST-42 / AC-42	Middleware unit + API integration	Guard test harness with a target-handler spy; no session, valid PARTICIPANT and ORGANIZER sessions	Call protected role-specific harness with missing/wrong/correct role; attempt identity override; use several target IDs	401 before role/handler, 403 for wrong role regardless of ID, handler not reached on denial; correct role uses session identity only. Real endpoint matrices also required in F-02..F-06 contracts
TEST-24 / AC-24	E2E, later #37	Own E2E database seeded; browser signed out	Login each role, check header, logout, open protected page	Name/role correct; successful logout then protected page opens login; retain command/commit/output when actually implemented

TEST-04 also parameterizes logout with a valid session and malformed JSON, non-object/nonempty body or non-JSON content type: expect 400 VALIDATION_ERROR, no cookie clearing and no Session deletion; the same session must still access /api/me. TEST-40 additionally calls logout with no, unknown, expired and revoked cookie, including malformed bodies: expect 401 before body validation, no private data and no deletion of another valid session (found expired-row cleanup remains required).

For every error assertion, check code, JSON envelope, no stack/private fields and no unintended persistence. Service inspection complements, but does not replace, API/DB behavioral checks. Do not expose test-harness routes in production. Each feature contract must enumerate no/invalid/expired/revoked-session and role/ownership cases for its real protected endpoints; this middleware harness alone does not prove the full route matrix.

8. Product Definition of Done

Use the shared Sprint 1 Definition of Done. Contract preparation does not complete those product/release gates.

9. Contract version history

Version	Date	Scope/change and reason	Status/review
F-01 1.0	2026-11-07	Initial #17 contract from merged baseline. Retains AC/TEST-01..05 and 24; proposes AC/TEST-39..42 for authenticated /me, expiry, input validation and shared guards, closing traceability gaps without renumbering or defining F-04's 37	Prepared by A with Codex/document_writer assistance; pending genuine B review and merge

Later F-02..F-06 contracts keep their own version histories in their owned files. Record changes before implementation, with Issue/PR, reason, affected IDs and actual reviewer/merge state. Do not label a proposed contract or future test as approved/passed.

F-02 Publish event contract

Issue #18 (parent #2), implemented in Issue #5. Author A specifies the contract; B reviews it and owns implementation. F-02 owns create, publish, My events and the shared event-detail API. F-03 owns the published list and consumes this detail API. F-04 owns registration, and F-06 owns the roster. Requirements: FR-04, FR-15, FR-05 (detail), BR-01, BR-09, NFR-01, NFR-03, NFR-07, NFR-09. Decisions: D-03, D-07, D-12, D-13, D-14.

Behavior and data

An organizer creates a DRAFT, opens it from My events, and publishes it with a separate action. Only the owning organizer publishes the DRAFT. Creation obtains `ownerId` from the authenticated session and sets `status=DRAFT`, `publishedAt=null`. Publication atomically updates the owning DRAFT to PUBLISHED with `publishedAt=now`, leaving the other fields unchanged. No edit, delete, close or cancel operation is in F-02.

Use the system SDS Event model: UUID `id`, organizer FK `ownerId`, title, description, location, timestamptz `startsAt`, integer `capacity` with database CHECK 1..10000, `status`, `createdAt` and nullable `publishedAt`. Foundation #24 owns the schema migration. F-02 does not write registration rows. For reads, calculate `remainingPlaces = capacity - count(ACTIVE registrations for this event)`; never store the derived value. A PUBLISHED event remains readable when full or after its start. Eligibility to register belongs to F-04.

API contract

Event payload fields: `id`, `title`, `description`, `location`, `startsAt`, `capacity`, `remainingPlaces`, `status`. Use UUID IDs, UTC ISO 8601 timestamps and status DRAFT or PUBLISHED. Responses do not expose owner IDs, user records, registration rows or session data. My events uses the same event payload for both statuses. Declare the literal `/api/events/mine` route before `/api/events/:id`.

Method and path	Request	Success
POST `/api/events`	ORGANIZER session. JSON object with `title`, `description`, `location`, `startsAt`, `capacity`.	201 with the event payload.
GET `/api/events/mine`	ORGANIZER session. No body or required query parameters.	200 `{ "events": [...] }` with only the caller's DRAFT and PUBLISHED events, sorted by startsAt then id ascending. No matches returns `{ "events": [] }`.
GET `/api/events/:id`	Session of either role. UUID route parameter. No body.	200 with the event payload for any PUBLISHED event or the organizer's own DRAFT. F-03 uses this same payload for event detail.
POST `/api/events/:id/publish`	ORGANIZER session. UUID route parameter and JSON `{}`.	200 with the PUBLISHED event payload after persistence.

Create accepts title and location strings of length 1..120 after trimming, description string of length 0..2000, and capacity as a JSON integer from 1 to 10000 inclusive. Reject missing fields, wrong types, unknown fields (including ownerId, status and publishedAt), malformed JSON, and non-JSON POSTs. Accept valid ISO 8601 datetime with an explicit zone and normalize to UTC. For create and publish, compare `startsAt > now`; equality and past times return 400 `INVALID_START_TIME`. Capture the server clock for the operation and pass it to the service for deterministic boundary tests. Invalid shape, datetime syntax or UUID returns 400 `VALIDATION_ERROR`. Publish accepts only an empty object.

Session check precedes role check, then validate input before resource lookup. Errors use `{ "error": { "code": "...", "message": "..." } }` with optional field details for validation and no stack or private data. Any protected route returns 401 `UNAUTHENTICATED` for no, invalid, expired or revoked session. The three organizer-only routes return 403 `FORBIDDEN` to participants before looking up an event, even for DRAFT, PUBLISHED or missing UUIDs. A well-formed missing event ID returns 404 `EVENT_NOT_FOUND` after applicable guards. Detail for a foreign DRAFT gives the same 404 as missing. On publish, a non-owner organizer gets 404 for a DRAFT and 403 for a PUBLISHED event. Check ownership before evaluating the state/time of the event. The owner receives 409 `ALREADY_PUBLISHED` on republish, including when the start has passed. Any rejection leaves the database unchanged; unexpected failures use the shared 500 `INTERNAL_ERROR` envelope.

UI contract

- **Create event** at `/organizer/events/new`: label all five fields, show local time in the form and serialize the selected instant with zone information. Show validation errors near fields or in the form, retain inputs on failure, and disable submission while pending. On 201 confirm the draft was saved and navigate to My events, which reloads persisted data.
- **My events** at `/organizer/events`: show a loading indicator during fetch, “No events yet” when empty, and a readable error with retry when fetch fails. Display DRAFT/PUBLISHED status, provide a detail link, and show Publish only on owned DRAFT rows. Disable the corresponding button while pending, reload on 200 and surface failures without pretending publication succeeded. The roster link hands off to F-06.
- **Event detail** at `/events/:id`: F-03 renders the payload and loading, missing and error states. The owner's DRAFT is accessible from My events. F-04 controls registration behavior. Redirect unauthenticated users to sign in, and show an access message on 403. UI controls do not replace backend authorization. Record the My events loading, empty and error checks in the Sprint 1 NFR-09 manual evidence.

Acceptance criteria and planned software tests

Use the corresponding TEST ID in each executable test name. Use Vitest service tests for fixed time and Supertest against real PostgreSQL for API/database assertions. Run integration tests with `DATABASE_URL_TEST` and D-14 isolation. Each negative API case checks status, the shared error envelope with code and message, validation details shape when present, and absence of stack traces and private data. For writes, compare relevant database rows before and after rejection.

AC / TEST	Level	Setup and action	Expected
AC-06 / TEST-06	API + DB	Organizer submits valid input; parameterize capacity 1 and 10000 with future start.	201, persisted DRAFT owned by the session user with matching fields, null publishedAt and full remainingPlaces.
AC-07 / TEST-07	Service unit + API + DB	Try capacity 0, negative, non-integer and above 10000; also reject a numeric string.	400 VALIDATION_ERROR; no event stored.
AC-08 / TEST-08	API + DB	Participant submits a valid create request.	403 FORBIDDEN; no event stored.
AC-09 / TEST-09	API + DB	Call each F-02 route with no cookie, invalid token, expired session and revoked session; use a valid payload/UUID where applicable and an independent expired session per route.	Every call returns 401 UNAUTHENTICATED; no event data or writes.
AC-25 / TEST-25	API + DB	Owner publishes a future DRAFT; assert the result from a DB reread.	200 and persisted PUBLISHED with publishedAt set.
AC-26 / TEST-26	API + DB	Another organizer attempts to publish the owner's DRAFT.	404 EVENT_NOT_FOUND; DRAFT and publishedAt unchanged.
AC-29 / TEST-29	Service unit + API + DB	Organizer creates with a past startsAt.	400 INVALID_START_TIME; no event stored.
AC-30 / TEST-30	Service unit + API + DB	Owner publishes a seeded DRAFT with past startsAt in API integration and equal startsAt with fixed service time.	400 INVALID_START_TIME; DRAFT and publishedAt unchanged.
AC-31 / TEST-31	API + DB	Owner requests own DRAFT by ID.	200 with matching payload and DRAFT status.
AC-32 / TEST-32	API	Participant and other organizer request the DRAFT and a missing valid UUID.	Same 404 EVENT_NOT_FOUND envelope with no event fields.
AC-33 / TEST-33	API + DB	Create owned and foreign events of each status, then call My events as owner and as a third organizer without events.	200 with exact own-event IDs for both statuses, ordered as specified; third organizer receives empty array.
AC-34 / TEST-34	API	Participant requests My events.	403 FORBIDDEN without event data.
AC-35 / TEST-35	Service unit	Create with startsAt equal to injected now.	400 INVALID_START_TIME; no insert.
AC-36 / TEST-36	API + DB	Owner publishes already PUBLISHED event, including one whose start passed.	409 ALREADY_PUBLISHED; state and timestamp unchanged.

AC / TEST	Level	Setup and action	Expected
AC-49 / TEST-49	API + DB	Participant, owner and other organizer fetch a PUBLISHED event with ACTIVE registrations and capacity above the active count; repeat for a full event.	200 with exact event payload, remainingPlaces equal to capacity minus ACTIVE count and zero when full, no registrant data.
AC-50 / TEST-50	API + DB	Participant attempts publish on DRAFT, PUBLISHED and missing UUIDs; non-owner organizer attempts publish on another organizer's PUBLISHED and a missing UUID.	Participant receives identical 403 for all IDs. Non-owner receives 403 for PUBLISHED and 404 for missing. No writes.
AC-51 / TEST-51	API + DB	Organizer tries invalid create field lengths or shapes, invalid datetime, forbidden fields, malformed/non-JSON POST, nonempty publish body and malformed UUIDs for detail/publish.	400 VALIDATION_ERROR for each invalid shape, no leaks or writes.

AC/TEST-49..51 cover positive shared detail, publish authorization precedence and malformed input respectively. Earlier IDs retain their assigned meaning. Cross-feature TEST-22 exercises the create and publish journey when E2E work lands.

Apply the Sprint 1 Definition of Done. Before implementing Issue #5, establish contract-first history, record real red and green results with tested commits, and obtain teammate review. This document specifies expected results. Execution evidence will come from implementation and QA.

Version	Change
1.0	Initial F-02 contract for #18.

F-03 Browse events: engineering contract (Sprint 1)

Issue: #19 (sub-task 2.3 of Issue 2). Implementation: Issue 6 (`feature/s1-06-browse-events`). Author: B (stickkersz). Reviewer: A (songt888). Sources: `docs/srs.md` (FR-05, NFR-01, NFR-07, NFR-09), `docs/decisions.md` (D-07, D-08, D-13, D-14), `docs/system-sds.md` (sections 1.3, 2.1, 3, 4, 5, 6), `deliverable1-plan.md` section 5.

1. Linked requirements

ID	How F-03 covers it
FR-05	Any signed-in user lists PUBLISHED events with `remainingPlaces = capacity - ACTIVE registrations`; DRAFT events never appear in the list.
NFR-01	`GET /api/events` with no, invalid, expired or revoked session returns 401 and no event data (AC-38).
NFR-07	Errors use the shared shape `{ "error": { "code", "message" } }`; no stack trace.
NFR-09	The Events screen and the Event detail screen show loading, empty (list only) and error states.
D-07	Only PUBLISHED events are listed.
D-13	DRAFT visibility on the detail endpoint follows F-02 (404 `EVENT_NOT_FOUND` for everyone except the owner).

No BR is owned by F-03. BR-02..BR-04 (registration rules) belong to F-04.

2. Behavior and workflow

1. A signed-in user (participant or organizer) opens `/events`.
2. The UI calls `GET /api/events`.
3. The API checks the session (`requireSession`). No role check: both roles may browse.
4. The service reads every event with status `PUBLISHED` and, for each, counts its ACTIVE registrations. It returns `remainingPlaces = capacity - activeCount`. The value is computed on read and never stored (system SDS 2.1).
5. The UI lists the events. Each row links to `/events/:id`.
6. The Event detail screen calls `GET /api/events/:id`. That endpoint is defined by the F-02 contract (#18), including the DRAFT rule (AC-31, AC-32). F-03 does not redefine it; F-03 only needs its response to contain the same fields as one list item, including `remainingPlaces` (see section 9).
7. The Register button on the detail screen belongs to F-04.

PUBLISHED events whose `startsAt` has passed stay in the list in Sprint 1. FR-05 has no time filter; registering for them is rejected by F-04 (AC-28, 409 `EVENT_STARTED`).

3. Validation

`GET /api/events` takes no request body and no query parameters in Sprint 1 (no filters, no pagination; system SDS 4.1). Query parameters, if sent, are ignored.

4. Failure and boundary cases

Case	Result
No session cookie, invalid session cookie, expired session or revoked (logged-out) session	401 `UNAUTHENTICATED`, no event data (AC-38)
No PUBLISHED events (DRAFT events may exist)	200 with an empty list; the UI shows the empty state (AC-10, AC-11)
Event with no ACTIVE registrations	`remainingPlaces` equals `capacity` (AC-10)
Event whose ACTIVE registrations equal its capacity	`remainingPlaces` is `0`; the event is still listed (AC-10)
Unexpected server or database failure	500 `INTERNAL_ERROR` with a generic message, no stack trace (NFR-07)

5. Permissions

Operation	Anon	Participant	Organizer
`GET /api/events` (list PUBLISHED)	401	allowed	allowed
`GET /api/events/:id`	as defined in F-02	as defined in F-02	as defined in F-02

Identity comes only from the session cookie. The list is the same for every signed-in user: it contains no private data and no registrant information.

6. UI behavior

Screen	Route	Behavior
Events	`/events`	Loading: "Loading events...". Empty: "No published events yet". Error: the API `error.message` in an alert; 401 redirects to `/login`. Each row shows the title (link to `/events/:id`), the start time in the browser's local time, the location and "N of C places left"; when `remainingPlaces` is 0 the row shows "Full".
Event detail	`/events/:id`	Loading: "Loading event...". Error: 404 shows "Not found"; other errors show the API `error.message`; 401 redirects to `/login`. Shows title, description, location, local start time and "N of C places left" (or "Full").

Shared states follow system SDS 3.2. The UI shows the value from the API; it never computes `remainingPlaces` itself.

7. API

`GET /api/events`

- Auth: session cookie required. Any role.
- Request: no body, no parameters.
- Response 200:

```
{
  "events": [
    {
      "id": "3f1c2a9e-0000-4000-8000-000000000001",
      "title": "Intro to Git workshop",
      "description": "Hands-on session for first-year students.",
      "location": "Engineering Building, Room 301",
      "startsAt": "2026-11-20T09:00:00.000Z",
      "capacity": 30,
      "remainingPlaces": 29,
      "status": "PUBLISHED"
    }
  ]
}
```

- Order: `startsAt` ascending, then `id` ascending, so the order is stable.
- Not returned: `ownerId`, owner details, registrations, `createdAt`, `publishedAt`.
- Errors: 401 `UNAUTHENTICATED`; 500 `INTERNAL_ERROR`. Shape as in system SDS 4.2.

`GET /api/events/:id`

Defined by the F-02 contract (#18). Not redefined here.

8. Data

- No schema change and no migration. F-03 reads `Event` and `Registration` as created in Issue 3.
- `remainingPlaces` is derived on read: `capacity - count(Registration WHERE eventId = e.id AND status = 'ACTIVE')`.

- Seed (Issue 3) already contains PUBLISHED and DRAFT events, and one PUBLISHED event with one place left (system SDS 2.3); no seed change.

9. Dependencies on other contracts

- F-01 (#17): `requireSession`, the 401 response and the error handler.
- F-02 (#18): `GET /api/events/:id`. F-03's Event detail screen needs that response to include `remainingPlaces` for a PUBLISHED event, because FR-05 covers viewing an event with its remaining places. The positive PUBLISHED-detail AC belongs to #18 (traceability line 22).
- Issue 3: schema, seed and Vitest runner. TEST-11 is the first Playwright test: the Playwright config and `npm run test:e2e` (D-14, `DATABASE_URL_E2E`) must exist before Issue 6 merges; A adds them, with one smoke test, in Issue 3.3 (#25), as agreed in PR #53. Issue 9 reuses that setup for TEST-22..TEST-24.

10. Acceptance criteria

AC	Criterion	Linked
AC-10	A signed-in user's `GET /api/events` returns 200 with every PUBLISHED event and no DRAFT event. Each item has `remainingPlaces = capacity - ACTIVE registrations` (equal to `capacity` with no registrations, `0` when full). With only DRAFT events stored, the list is empty (200, `{ "events": [] }`).	FR-05, D-07
AC-11	When `GET /api/events` returns an empty list, the Events screen shows "No published events yet" and no event rows.	FR-05, NFR-09
AC-38	`GET /api/events` without a session cookie, with a cookie whose token matches no session, with the cookie of an expired session (`expiresAt ⩽ now`), or with the cookie of a logged-out session, returns 401 `UNAUTHENTICATED` and the body contains no event data.	NFR-01, FR-05

AC-38 is new: no existing AC covered an unauthenticated request to the event list (AC-03 covers `/api/me`, AC-09 covers `POST /api/events`).

11. Software test specification (STS)

TEST	AC	Level	Setup	Steps	Expected
TEST-10	AC-10	API + DB integration (Vitest + Supertest, `DATABASE_URL_TEST`)	Insert directly: organizer O, participants P1 and P2; event E1 PUBLISHED capacity 3 with no registrations; E2 PUBLISHED capacity 2 with ACTIVE registrations for P1 and P2; E3 DRAFT owned by O. Log in as P1.	`GET /api/events`.	200; exactly E1 and E2 in the list, E3 absent; E1 `remainingPlaces` 3; E2 `remainingPlaces` 0. Also run the list as O: same two events. Second case (fresh database, only E3 DRAFT inserted, log in as P1): `GET /api/events` returns 200 with `{ "events": [] }`.
TEST-11	AC-11	E2E UI (Playwright, `DATABASE_URL_E2E`)	Seeded database; log in through the UI as a seeded participant. Replace only the `GET /api/events` response with `{ "events": [] }` (Playwright `page.route`), since the seeded database always has PUBLISHED events.	Open `/events`.	"No published events yet" is visible; no event row is rendered.
TEST-38	AC-38	API + DB integration (Vitest + Supertest, `DATABASE_URL_TEST`)	Insert directly: organizer O, participant P1, one PUBLISHED event owned by O, and a `Session` row for P1 with `tokenHash` = SHA-256 of a known token T and `expiresAt` in the past (system SDS 6, D-08).	(a) `GET /api/events` with no cookie. (b) `GET /api/events` with a session cookie holding a random token that has no `Session` row. (c) `GET /api/events` with a session cookie holding T. (d) Log in as P1, log out, then `GET /api/events` with the old cookie.	All four 401 with `error.code` `UNAUTHENTICATED`; the body has no `events` field.

Test names start with their IDs, for example `TEST-10 AC-10 lists only PUBLISHED events with remaining places`. TEST-11 is UI-only coverage with a mocked API response; evidence must label it so. The real API and database behavior of the list is covered by TEST-10. The loading and error states of both screens (NFR-09) are checked by hand in the 9.4 QA run checklist in `docs/evidence/`, not by an automated test.

12. Definition of Done (F-03)

The Sprint 1 Definition of Done (F-01 contract, #17) applies. For F-03 in addition:

- TEST-10, TEST-11 and TEST-38 exist, run and pass; none skipped.
- The failing run of TEST-10 is committed before the implementation (red, then green).
- The service computes `remainingPlaces` from ACTIVE registrations; nothing is stored.
- The Events and Event detail screens show loading, empty (list) and error states.
- PR reviewed by A and merged into `sprint-1-staging` with a merge commit.

13. Version history

Version	Date	Change	Author	Reviewer
1.0	2026-11-07	First F-03 contract (Issue #19)	B (stickkersz)	A (songt888)
1.1	2026-11-08	D-15 (Issue #73): the Events screen shows each event as a card (CD case) instead of a table row; the fields, the title link to `/events/:id`, the loading, empty and error texts and every AC are unchanged. "Row" in sections 1 and 6 reads as "card"	B (stickkersz)	A (songt888)

F-04 Register for an event: engineering contract (Sprint 1)

Issue: #20 (sub-task 2.4 of Issue 2). Implementation: Issue 7 (`feature/s1-07-register`): B writes the failing tests (7.1, #32), A writes the implementation and UI (7.2, #33). Author: B (stickkersz). Reviewer: A (songt888). Sources: `docs/srs.md` (FR-06, BR-02, BR-03, BR-04, NFR-01, NFR-03, NFR-04, NFR-07, NFR-09), `docs/decisions.md` (D-02, D-03, D-04, D-10, D-12, D-13, D-14), `docs/system-sds.md` (sections 1.3, 2.1, 2.3, 3, 4, 5, 6, 9, 11), `docs/traceability.md` (NFR-01, NFR-03, NFR-07 rows and coverage gaps), `deliverable1-plan.md` sections 3 and 5.

1. Linked requirements

ID	How F-04 covers it
FR-06	A participant registers for a PUBLISHED event that has not started. One ACTIVE registration is stored and `remainingPlaces` goes down by one. Full, duplicate, DRAFT or missing, and started-event requests store nothing.
BR-02	ACTIVE registrations never exceed `capacity`, also for parallel requests for the last place. One transaction locks the event row before any check (D-10).
BR-03	One ACTIVE registration per participant per event: service check inside the transaction, plus the partial unique index from Issue 3 (D-02).
BR-04	Only a PUBLISHED event with `startsAt` strictly later than now is registrable; `startsAt` equal to now counts as started (D-04, D-12).
NFR-01	A call to `POST /api/events/:id/registrations` with no cookie, an unknown token, an expired session or a logged-out session gets 401 `UNAUTHENTICATED` and stores nothing (AC-43).
NFR-03	The participant is always the session user; a request body that names a `participantId` is rejected with 400 and stores nothing (AC-44).
NFR-04	20 parallel registration requests from 20 different participants for a capacity-5 event store exactly 5 ACTIVE registrations (AC-37).
NFR-07	Inputs are validated server-side: a non-UUID `:id` or any body field gets 400 `VALIDATION_ERROR` (AC-44). Errors use the shared shape `{ "error": { "code", "message" } }`, plus `details` for 400 `VALIDATION_ERROR` only, and no `stack` field (system SDS 4.2); every STS error case asserts the shape (section 11).
NFR-09	On the Event detail screen (one of the five NFR-09 screens) a registration rejected with 409 shows the API `error.message` readably; 403, 404, submitting and success states follow system SDS 3.2.
D-13	Organizer: 403 `FORBIDDEN`. DRAFT or missing event: 404 `EVENT_NOT_FOUND`. Started event: 409 `EVENT_STARTED`.

2. Behavior and workflow

1. A signed-in participant opens `/events/:id` (F-03 screen) and presses Register.
2. The UI calls `POST /api/events/:id/registrations`. It sends no user ID: the participant is always the session user.
3. The route runs `requireSession` (401), then `requireRole('PARTICIPANT')` (403). An organizer gets 403 for every event ID, so the check order of D-13 holds.
4. The service runs one database transaction, in this order (D-10):
5. Lock the event row: `SELECT ... FROM "Event" WHERE id = $1 FOR UPDATE` (Prisma `$queryRaw` inside `$transaction`).
6. No row, or status is not `PUBLISHED`: 404 `EVENT_NOT_FOUND`.
7. `startsAt ⩽ now`: 409 `EVENT_STARTED`. `now` is a parameter of the service function; the route passes `new Date()` (D-12).
8. The participant already has an ACTIVE registration for this event: 409 `ALREADY_REGISTERED`.
9. Count ACTIVE registrations of the event. Count `⩾ capacity`: 409 `EVENT_FULL`.
10. Insert the registration with status `ACTIVE` and return 201.
11. Commit releases the lock. Any rejection rolls back, so nothing is stored.

12. If the insert still hits the partial unique index (Prisma error `P2002`), the service returns 409 `ALREADY_REGISTERED`. With the row lock this should not happen; the index is only a backstop (D-10).
13. The UI shows the result and reloads the event, so the screen shows the new `remainingPlaces` from the API.

Because the event row is locked first, a second request for the same event waits until the first commits, then counts the committed rows. Two requests can never both see the last free place.

3. Validation

- Path parameter `:id` must be a UUID; otherwise 400 `VALIDATION_ERROR` (system SDS 4.1, 4.2), checked after the session and role guards and before any query.
- Request body: an empty JSON object `{}` sent with `Content-Type: application/json`; any field (for example `participantId`) is rejected with 400 `VALIDATION_ERROR` (system SDS 4.1, unknown fields rejected). The service never reads a participant ID from the request.
- State-changing request rules of system SDS 4.1 apply (`POST` with `Content-Type: application/json`).

4. Failure and boundary cases

Case	Result
No cookie, unknown token, expired session (`expiresAt ⩽ now`) or logged-out session	401 `UNAUTHENTICATED`, nothing stored (AC-43), tested on this route
Caller is an organizer (owner or not, any event ID)	403 `FORBIDDEN`, nothing stored (AC-16)
Event ID unknown or event is DRAFT	404 `EVENT_NOT_FOUND`, nothing stored (AC-27)
`:id` not a UUID	400 `VALIDATION_ERROR`, nothing stored (AC-44)
Event PUBLISHED but `startsAt` in the past	409 `EVENT_STARTED`, nothing stored (AC-28)
`startsAt` exactly equal to `now` (boundary)	409 `EVENT_STARTED` (AC-28, BR-04)
Participant already has an ACTIVE registration for the event	409 `ALREADY_REGISTERED`, still one row (AC-14)
ACTIVE count equals `capacity`	409 `EVENT_FULL`, nothing stored (AC-13)
One place left (`capacity - 1` ACTIVE)	the request succeeds; the event is then full (AC-12, AC-15)
Several requests at once for the last place	exactly one 201, the others 409 `EVENT_FULL` (AC-15)
20 parallel requests, capacity 5, no prior registrations	exactly five 201, fifteen 409 `EVENT_FULL` (AC-37)
A body with any field, for example `participantId`	400 `VALIDATION_ERROR`, nothing stored (AC-44)
Unexpected server or database failure	500 `INTERNAL_ERROR`, generic message, no stack trace (NFR-07)

When several rules fail at once, the first failing step in section 2 decides the code: a participant who is already registered for a full event gets `ALREADY_REGISTERED`; a full event that has started gets `EVENT_STARTED`.

5. Permissions

Operation	Anon	Participant	Organizer (owner)	Organizer (other)
`POST /api/events/:id/registrations`	401	allowed if BR-02..BR-04 hold	403	403

Identity comes only from the session cookie (NFR-03). The new registration's `participantId` is always the session user's ID.

6. UI behavior

Screen	Route	Behavior
Event detail	`/events/:id`	The Register button is shown only to participants (from `GET /api/me`) and only for a PUBLISHED event. It stays enabled when the event shows "Full", so the server rule decides and its message is shown (AC-23). Submitting: the button is disabled while the request is pending. Success: "You are registered for this event." and the event reloads from the API. Error: the API `error.message` in an alert; 401 redirects to `/login`; 403 shows "You do not have access to this"; 404 shows "Not found" (system SDS 3.2).

API `error.message` values (the UI shows them for 409; 403 and 404 follow system SDS 3.2):

Code	`error.message`
`EVENT_FULL`	"This event has no places left." (system SDS 4.2)
`ALREADY_REGISTERED`	"You are already registered for this event."
`EVENT_STARTED`	"This event has already started."
`EVENT_NOT_FOUND`	"Event not found."
`FORBIDDEN`	"You do not have access to this."

Shared states follow system SDS 3.2. The UI never decides capacity, duplicates or eligibility itself.

7. API

`POST /api/events/:id/registrations`

- Auth: session cookie required. Role `PARTICIPANT`.
- Request: path `:id` (UUID); body `{}` (`Content-Type: application/json`).
- Response 201:

```
{
  "registration": {
    "id": "7a2d4c10-0000-4000-8000-000000000002",
    "eventId": "3f1c2a9e-0000-4000-8000-000000000001",
    "status": "ACTIVE",
    "createdAt": "2026-11-10T08:15:00.000Z"
  },
  "remainingPlaces": 28
}
```

- `remainingPlaces` is computed after the insert, inside the same transaction.
- Not returned: `participantId` (it is the caller), other registrations, owner details.
- Errors: 400 `VALIDATION_ERROR`; 401 `UNAUTHENTICATED`; 403 `FORBIDDEN`; 404 `EVENT_NOT_FOUND`; 409 `EVENT_STARTED`, `ALREADY_REGISTERED`, `EVENT_FULL`; 500 `INTERNAL_ERROR`. Shape as in system SDS 4.2.

8. Data

- No schema change and no migration. F-04 writes `Registration` (`id`, `eventId`, `participantId`, `status`, `createdAt`) as created in Issue 3.
- Relies on the partial unique index `(eventId, participantId) WHERE status = 'ACTIVE'` and the `capacity` CHECK 1..10000 from the Issue 3 migration (system SDS 2.3).
- The seed planned in system SDS 2.3 (Issue 3) has one PUBLISHED event with capacity 2 and one ACTIVE registration, for the full-event demo and TEST-23; no seed change.

9. Dependencies on other contracts

- F-01 (#17): `requireSession`, `requireRole`, the 401/403 responses and the error handler.
- F-02 (#18): events exist and are published; `GET /api/events/:id` returns `remainingPlaces` for the reload after registering.
- F-03 (#19): the Event detail screen that holds the Register button.
- Issue 3: schema with the partial unique index, `DATABASE_URL_TEST`, sequential integration files and table truncation (D-14).

10. Acceptance criteria

AC	Criterion	Linked
AC-12	A participant registering for a PUBLISHED, future event with free places gets 201 with an ACTIVE registration; exactly one ACTIVE row for that participant and event is stored; `remainingPlaces` in the response and in `GET /api/events/:id` is one lower than before.	FR-06
AC-13	Registering for an event whose ACTIVE registrations equal its capacity returns 409 `EVENT_FULL` and stores no row.	FR-06, BR-02
AC-14	A participant's second registration for the same event returns 409 `ALREADY_REGISTERED` and the event still has one ACTIVE row for that participant; inserting a second ACTIVE row for the same participant and event directly in the database fails with a unique violation.	FR-06, BR-03, D-02
AC-15	When several different participants send registration requests at the same time for the last place of an event, exactly one gets 201, every other request gets 409 `EVENT_FULL`, and ACTIVE registrations equal the capacity.	BR-02, D-10
AC-16	An organizer (owner or not) registering for an event (PUBLISHED, DRAFT or unknown ID) returns 403 `FORBIDDEN` and stores no row.	FR-06, D-13
AC-27	Registering for a DRAFT event or an unknown event ID returns 404 `EVENT_NOT_FOUND` and stores no row.	FR-06, BR-04, D-13
AC-28	Registering for a PUBLISHED event whose `startsAt` is in the past returns 409 `EVENT_STARTED` and stores no row; `startsAt` equal to `now` is also rejected as started.	FR-06, BR-04, D-12
AC-37	For 20 different participants sending registration requests at the same time to a PUBLISHED, future event with capacity 5 and no registrations, exactly 5 requests get 201, 15 get 409 `EVENT_FULL`, and exactly 5 ACTIVE registrations are stored.	NFR-04, BR-02
AC-43	`POST /api/events/:id/registrations` with no cookie, with a token that matches no session, with an expired session (`expiresAt ⩽ now`) or with the cookie of a logged-out session returns 401 `UNAUTHENTICATED` and stores no row.	NFR-01, FR-06, D-08

AC	Criterion	Linked
AC-44	A participant's request with body `{ "participantId": "<another participant's id>" }`, or with a non-UUID `:id`, returns 400 `VALIDATION_ERROR` and stores no row.	NFR-03, NFR-07, FR-06

TEST-37 is already named as the NFR-04 verification in `docs/srs.md` (NFR-04, BR-02), `docs/system-sds.md` (sections 9 and 11) and `deliverable1-plan.md` section 3, which says it is "specified in the F-04 contract, #20"; this contract defines AC-37 with the same number. AC-43 and AC-44 are new: no existing AC covered an unauthenticated registration request (AC-03 covers `/api/me`, AC-09 covers `POST /api/events`, AC-38 in F-03 PR #53 covers `GET /api/events`), or a registration request that names another participant or has an invalid `:id`. `docs/traceability.md` asks the F-01..F-06 contract authors to enumerate session checks on protected routes (NFR-01 row, coverage gaps) and to add explicit validation and direct-request assertions (NFR-03, NFR-07). The numbers follow AC-42, the highest AC found on `sprint-1-staging`, every remote branch and the open PRs.

11. Software test specification (STS)

All API tests below are API + DB integration tests (Vitest + Supertest, `DATABASE_URL_TEST`, tables truncated in `beforeEach`, D-14). Fixtures are inserted directly in the database; "log in" means `POST /api/auth/login` with the fixture's password. Every `POST` in these tests sends `{}` as JSON, except the body case of TEST-44. Every error response is also asserted to have the shape `{ "error": { "code", "message" } }`, plus `details` for 400 `VALIDATION_ERROR` only, and no `stack` field (NFR-07, system SDS 4.2): the 401, 403, 404 and 409 responses have exactly the keys `code` and `message` in `error`; the 400 responses (TEST-44) have exactly the keys `code`, `message` and `details` in `error`, with the full envelope asserted in the TEST-44 row.

TEST	AC	Level	Setup	Steps	Expected
TEST-12	AC-12	API + DB integration	Organizer O; participant P1; event E PUBLISHED, capacity 3, `startsAt` now + 7 days, no registrations. Log in as P1.	`POST /api/events/E/registrations`; then `GET /api/events/E`.	201; `registration.status` `ACTIVE`, `registration.eventId` E; `remainingPlaces` 2. Database: exactly one Registration for (E, P1), status ACTIVE. Detail shows `remainingPlaces` 2.
TEST-13	AC-13	API + DB integration	O; participants P1, P2, P3; E PUBLISHED, capacity 2, future, ACTIVE registrations for P1 and P2. Log in as P3.	`POST /api/events/E/registrations`.	409, `error.code` `EVENT_FULL`. Database: still 2 Registrations for E, none for P3.
TEST-14	AC-14	API + DB integration	O; P1; E PUBLISHED, capacity 5, future. Log in as P1.	(a) Register for E twice. (b) Insert a second ACTIVE Registration for (E, P1) directly with Prisma.	(a) first 201, second 409 `ALREADY_REGISTERED`; one ACTIVE row for (E, P1). (b) the insert throws a unique violation (Prisma `P2002`); still one ACTIVE row.
TEST-15	AC-15	API + DB integration, concurrent	O; participants P1..P12; E PUBLISHED, capacity 3, future, ACTIVE registrations for P1 and P2 (one place left). Log in P3..P12 one by one first, keeping each cookie.	Send the 10 registration requests (P3..P12) at the same time with `Promise.all`, all inside one test.	Exactly one 201 and nine 409 `EVENT_FULL`. Database: exactly 3 ACTIVE Registrations for E.
TEST-16	AC-16	API + DB integration	Organizers O (owner) and O2; E PUBLISHED, capacity 5, future, owned by O; D DRAFT, capacity 5, future, owned by O.	Log in as O, `POST /api/events/E/registrations`; log in as O2, same request; as O2, the same request with a random UUID; as O, `POST /api/events/D/registrations`; as O2, the same request for D.	All five 403 `FORBIDDEN` (the role check runs before the DRAFT check, D-13). Database: no Registration rows.
TEST-27	AC-27	API + DB integration	O; P1; D DRAFT owned by O, capacity 5, future. Log in as P1.	Register for D; for a random UUID.	Both 404 `EVENT_NOT_FOUND`. Database: no Registration rows.

TEST	AC	Level	Setup	Steps	Expected
TEST-28	AC-28	API + DB integration and unit	O; P1; S PUBLISHED, capacity 5, `startsAt` now minus 1 hour (inserted directly, D-12). Log in as P1. Unit case: no database.	Register for S. Unit case: call the service's eligibility check with a fixed `now` equal to the event's `startsAt`.	API: 409 `EVENT_STARTED`; no Registration rows. Unit: rejected with `EVENT_STARTED`.
TEST-37	AC-37	API + DB integration, concurrent	O; participants P1..P20; E PUBLISHED, capacity 5, future, no registrations. Log in P1..P20 one by one first, keeping each cookie.	Send the 20 registration requests at the same time with `Promise.all`, all inside one test.	Exactly five 201 and fifteen 409 `EVENT_FULL`. Database: exactly 5 ACTIVE Registrations for E, from 5 different participants.
TEST-43	AC-43	API + DB integration	O; P1; E PUBLISHED, capacity 5, future, owned by O. A `Session` row for P1 with `tokenHash` = SHA-256 of a known token T and `expiresAt` in the past, inserted directly (system SDS 6, D-08).	`POST /api/events/E/registrations` (a) with no cookie, (b) with a random token as cookie, (c) with T as cookie, (d) after logging in as P1 and `POST /api/auth/logout`, with the old cookie.	All four 401, `error.code` `UNAUTHENTICATED`. Database: no Registration rows.
TEST-44	AC-44	API + DB integration	O; participants P1, P2; E PUBLISHED, capacity 5, future, owned by O. Log in as P1.	(a) `POST /api/events/E/registrations` with body `{ "participantId": "<P2 id>" }`. (b) `POST /api/events/not-a-uuid/registrations` with `{}`.	Both 400 with the envelope `{ "error": { "code": "VALIDATION_ERROR", "message": "<nonempty string>", "details": [{ "field": "<name>", "message": "<nonempty string>" }] } }` (system SDS 4.2): `error` has exactly the keys `code`, `message` and `details` (no `stack`); `error.message` is a nonempty string; `details` is a nonempty array whose items have exactly the keys `field` and `message`, both nonempty strings; (a) has an item with `field` `participantId`, (b) has an item with `field` `id`. Database: no Registration rows (none for P1 or P2).

Test names start with their IDs, for example `TEST-13 AC-13 rejects registration when event is full`. TEST-13 is a TDD sample for the PDF: its failing run is saved as `docs/evidence/tdd/TEST-13-red.txt` and its passing run as `TEST-13-green.txt` (team-workflow). The UI states of the Register action (NFR-09 error message; system SDS 3.2 submitting and success) are checked by TEST-23 (E2E, Issue 9) for the full-event message and by hand in the 9.4 QA run checklist. TEST-15 and TEST-37 are each run five times in the 9.4 QA run (issue-plan 9.4).

12. Definition of Done (F-04)

The shared Sprint 1 Definition of Done applies. For F-04 in addition:

- TEST-12..TEST-16, TEST-27, TEST-28, TEST-37, TEST-43 and TEST-44 exist, run and pass; none skipped.
- The failing tests are committed before the implementation on `feature/s1-07-register`, and the failing run is saved (TEST-13 in `docs/evidence/tdd/`).
- The registration runs in one transaction that locks the event row first (D-10); no "atomic conditional insert".
- The Register button shows submitting, success and error states.
- PR opened by A, reviewed by B, merged into `sprint-1-staging` with a merge commit.

13. Version history

Version	Date	Change	Author	Reviewer
1.0	2026-11-07	First F-04 contract (Issue #20)	B (stickkersz)	A (songt888)

Version	Date	Change	Author	Reviewer
1.1	2026-11-07	Before the PR: added AC-43/TEST-43 (NFR-01, 401 on registration) and AC-44/TEST-44 (NFR-03, NFR-07, 400 for a `participantId` body or non-UUID `:id`); STS asserts the error shape; sources list completed	B (stickkersz)	A (songt888)
1.2	2026-11-07	Before the PR: error shape allows `details` on 400 `VALIDATION_ERROR` only (system SDS 4.2); TEST-44 asserts `error.code` and no `stack`	B (stickkersz)	A (songt888)
1.3	2026-11-07	Before the PR: AC-43/TEST-43 add unknown-token and expired-session cases (NFR-01); `EVENT_FULL` message matches system SDS 4.2; DoD links the shared `definition-of-done.md`	B (stickkersz)	A (songt888)
1.4	2026-11-07	Review round 1 (PR #54): TEST-44 asserts the full validation envelope: `code`, nonempty `message`, `details` items `{ field, message }` naming `participantId` or `id`, no `stack` (system SDS 4.2)	B (stickkersz)	A (songt888)
1.5	2026-11-07	Review round 2 (PR #54): TEST-16 adds a DRAFT event; an organizer gets 403 `FORBIDDEN` (not 404) before any DRAFT check (D-13); AC-16 wording extended	B (stickkersz)	A (songt888)

F-05 My registrations and F-06 Event registrations (organizer): engineering contract (Sprint 1)

Issue: #21 (sub-task 2.5 of Issue 2). Implementation: Issue 8: A writes F-05 (8.1, #35, `feature/s1-08-my-registrations`), B writes F-06 (8.2, #36, `feature/s1-08-organizer-view`). Author: B (stickkersz). Reviewer: A (songt888). Sources: `docs/srs.md` (FR-07, FR-08, FR-15, BR-05, NFR-01, NFR-03, NFR-07, NFR-09), `docs/decisions.md` (D-06, D-08, D-13, D-14), `docs/system-sds.md` (sections 1.3, 2.1, 3, 4, 5, 6, 9), `docs/features.md` (F-05, F-06), `docs/traceability.md` (FR-07, FR-08, BR-05, NFR-01, NFR-03 rows), `docs/sprints/sprint-1/f-01-auth.md` (AC-42, TEST-42), `deliverable1-plan.md` section 5.

1. Linked requirements

ID	How F-05 and F-06 cover it
FR-07	F-05: a participant retrieves their own registrations, each with its status, and one own registration by ID. Another participant's registration returns 404 `NOT_FOUND` with no data.
FR-08	F-06: an organizer retrieves the registrations of an event they own. Participants get 403; other organizers get 403 on a PUBLISHED event and 404 on a DRAFT (D-13).
FR-15	F-06: the organizer picks the event from their own My events list (F-02) to open its registrations.
BR-05	Participants read only their own registrations; organizers read registrations only of their own events. Record access is decided from the session user in the service layer (system SDS 1.3, 5.2).
NFR-01	All three routes reject no cookie, an unknown token, an expired session and a logged-out session with 401 `UNAUTHENTICATED` and no registration data (AC-45, AC-47).
NFR-03	Role is checked on every route (AC-21, AC-46) and ownership in the service (AC-18, AC-20, AC-48). No route reads a user ID from the request.
NFR-07	A non-UUID `:id` gets 400 `VALIDATION_ERROR` (AC-46, AC-48). Errors use the shape of system SDS 4.2; every STS error case asserts it (section 11).
NFR-09	My registrations and Event registrations are two of the five NFR-09 screens: loading, empty and error states follow system SDS 3.2 (section 6).
D-06, D-13	Participant: own registrations only; foreign registration 404 `NOT_FOUND`. Organizer: own events only; wrong role 403 for every ID; non-owner 403 (PUBLISHED) or 404 `EVENT_NOT_FOUND` (DRAFT).

2. Behavior and workflow

F-05 My registrations (participant)

1. A signed-in participant opens `/my/registrations` from the header link.
2. The UI calls `GET /api/registrations/mine`. The route runs `requireSession` (401), then `requireRole('PARTICIPANT')` (403).
3. The service returns every registration whose `participantId` is the session user, newest first (`createdAt` descending), each with a summary of its event.
4. `GET /api/registrations/:id` runs the same guards, validates `:id` as a UUID, then loads the registration with `id = :id AND participantId = <session user>`. No row (unknown ID or another participant's registration): 404 `NOT_FOUND`. Both cases give the same answer, so a foreign registration's existence is not revealed (D-13).

F-06 Event registrations (organizer)

1. A signed-in organizer opens My events (`/organizer/events`, F-02) and presses "Registrations" on one of their events. This opens `/organizer/events/:id/registrations`.

2. The UI calls `GET /api/events/:id` (F-02) for the event title and `GET /api/events/:id/registrations` for the list.
3. The route runs `requireSession` (401), then `requireRole('ORGANIZER')` (403), then validates `:id` as a UUID (400). A participant therefore gets 403 for every event ID (D-13 check order).
4. The service loads the event:
5. No event: 404 `EVENT_NOT_FOUND`.
6. Event is DRAFT and its owner is not the caller: 404 `EVENT_NOT_FOUND` (D-13 event rule).
7. Event is PUBLISHED and its owner is not the caller: 403 `FORBIDDEN`.
8. Owner: return the event's registrations, oldest first (`createdAt` ascending), each with the participant's name and email.
9. The queries only read; nothing is stored or changed.

3. Validation

- `GET /api/registrations/:id` and `GET /api/events/:id/registrations`: `:id` must be a UUID; otherwise 400 `VALIDATION_ERROR` (system SDS 4.1, 4.2), checked after the session and role guards and before any query.
- All three routes are `GET` with no request body. No route accepts a participant or organizer ID from the client; identity comes only from the session (NFR-03).
- Express route order: `/api/registrations/mine` is declared before `/api/registrations/:id`, otherwise `mine` is matched as an ID.

4. Failure and boundary cases

Route	Case	Result
all three	No cookie, unknown token, expired session (`expiresAt ⩽ now`) or logged-out session	401 `UNAUTHENTICATED`, no registration data (AC-45, AC-47)
`GET /api/registrations/mine`	Participant with no registrations	200, `registrations: []`; UI empty state (AC-17)
`GET /api/registrations/mine`, `GET /api/registrations/:id`	Caller is an organizer (any ID)	403 `FORBIDDEN` (AC-46)
`GET /api/registrations/:id`	Another participant's registration	404 `NOT_FOUND`, no data (AC-18)
`GET /api/registrations/:id`	Unknown UUID	404 `NOT_FOUND` (AC-46)
`GET /api/registrations/:id`	`:id` not a UUID	400 `VALIDATION_ERROR` (AC-46)
`GET /api/events/:id/registrations`	Caller is a participant (DRAFT, PUBLISHED or unknown ID)	403 `FORBIDDEN`, no data (AC-21)
`GET /api/events/:id/registrations`	Another organizer's PUBLISHED event	403 `FORBIDDEN`, no data (AC-20)
`GET /api/events/:id/registrations`	Another organizer's DRAFT event, or unknown UUID	404 `EVENT_NOT_FOUND`, no data (AC-48)
`GET /api/events/:id/registrations`	`:id` not a UUID	400 `VALIDATION_ERROR` (AC-48)
`GET /api/events/:id/registrations`	Owner's event with no registrations (for example the owner's own DRAFT)	200, `registrations: []`; UI empty state (AC-19)
all three	Unexpected server or database failure	500 `INTERNAL_ERROR`, generic message, no stack trace (NFR-07)

5. Permissions

Operation	Anon	Participant	Organizer (owner)	Organizer (other)
`GET /api/registrations/mine`	401	allowed (own only)	403	403
`GET /api/registrations/:id`	401	own: allowed; another participant's: 404	403	403
`GET /api/events/:id/registrations`	401	403	allowed	403 (PUBLISHED), 404 (DRAFT)

This is the system SDS 5.1 matrix. For the registration routes "Organizer (owner)" means the owner of the registration's event: an organizer never uses the participant routes, even for their own event.

6. UI behavior

Screen	Route	Behavior
My registrations (F-05)	`/my/registrations`	Header link shown to participants only. One row per registration: event title (link to `/events/:eventId`), start time in local time, location, status. Loading: "Loading..." while the request is pending. Empty: "You have no registrations yet." Error: the API `error.message` in an alert; 401 redirects to `/login`; 403 shows "You do not have access to this" (system SDS 3.2).
My events (F-02 screen, F-06 adds a link)	`/organizer/events`	Each row of the organizer's own events gets a "Registrations" link to `/organizer/events/:id/registrations` (FR-15).
Event registrations (F-06)	`/organizer/events/:id/registrations`	Heading with the event title (from `GET /api/events/:id`) and the number of registrations. Table: participant name, email, status, registered at (local time). Loading: "Loading..." while a request is pending. Empty: "No registrations yet." Error: the API `error.message` in an alert; 401 redirects to `/login`; 403 shows "You do not have access to this"; 404 shows "Not found" (system SDS 3.2).

`GET /api/registrations/:id` has no screen in Sprint 1; it is an API operation only (system SDS 4.3). The UI never filters registrations by owner itself: the screens show what the API returns.

API `error.message` values:

Code	`error.message`
`NOT_FOUND`	"Registration not found."
`EVENT_NOT_FOUND`	"Event not found."
`FORBIDDEN`	"You do not have access to this."

7. API

`GET /api/registrations/mine` (F-05)

- Auth: session cookie required. Role `PARTICIPANT`.
- Request: no parameters, no body.
- Response 200:

```
{
  "registrations": [
    {
      "id": "7a2d4c10-0000-4000-8000-000000000002",
      "status": "ACTIVE",
      "createdAt": "2026-11-10T08:15:00.000Z",
      "event": {
        "id": "3f1c2a9e-0000-4000-8000-000000000001",
        "title": "Intro to Git workshop",
        "location": "Room 201",
        "startsAt": "2026-11-20T09:00:00.000Z"
      }
    }
  ]
}
```

- Order: `createdAt` descending. Not returned: `participantId` (it is the caller), other participants' registrations, event owner details.
- Errors: 401 `UNAUTHENTICATED`; 403 `FORBIDDEN`; 500 `INTERNAL_ERROR`.

`GET /api/registrations/:id` (F-05)

- Auth: session cookie required. Role `PARTICIPANT`.
- Request: path `:id` (UUID).
- Response 200: `{ "registration": { ... } }` with the same fields as one item of `/mine`.
- Errors: 400 `VALIDATION_ERROR`; 401 `UNAUTHENTICATED`; 403 `FORBIDDEN`; 404 `NOT_FOUND` (unknown ID or another participant's registration); 500 `INTERNAL_ERROR`.

`GET /api/events/:id/registrations` (F-06)

- Auth: session cookie required. Role `ORGANIZER`, owner of the event.
- Request: path `:id` (UUID).
- Response 200:

```
{
  "registrations": [
    {
      "id": "7a2d4c10-0000-4000-8000-000000000002",
      "status": "ACTIVE",
      "createdAt": "2026-11-10T08:15:00.000Z",
      "participant": { "name": "Participant One", "email": "participant1@example.com" }
    }
  ]
}
```

- Order: `createdAt` ascending. `participant` has only `name` and `email` (system SDS 5.2); never `passwordHash`, session data or the participant's other registrations.
- Errors: 400 `VALIDATION_ERROR`; 401 `UNAUTHENTICATED`; 403 `FORBIDDEN`; 404 `EVENT_NOT_FOUND`; 500 `INTERNAL_ERROR`.

Error shape for all three routes as in system SDS 4.2.

8. Data

- No schema change and no migration. F-05 and F-06 only read `Registration` (`id`, `eventId`, `participantId`, `status`, `createdAt`), `Event` (`id`, `ownerId`, `title`, `location`, `startsAt`, `status`) and `User` (`name`, `email`) as created in Issue 3.
- Every S1 registration is `ACTIVE`; the `status` field is returned so S2 `CANCELLED` rows (F-07) need no API change.
- The seed planned in system SDS 2.3 (Issue 3) already has one ACTIVE registration on an organizer 1 event and a PUBLISHED event of organizer 2 for the cross-owner checks; no seed change.

9. Dependencies on other contracts

- F-01 (#17): `requireSession`, `requireRole`, the 401/403 responses and the error handler (AC-42).
- F-02 (#18): `GET /api/events/:id` for the Event registrations heading, and the My events screen that holds the "Registrations" link.
- F-04 (#20): registrations exist only through `POST /api/events/:id/registrations`; tests insert them directly.
- Issue 3: schema, `DATABASE_URL_TEST`, sequential integration files and table truncation (D-14).

10. Acceptance criteria

AC	Criterion	Linked
AC-17	A participant's `GET /api/registrations/mine` returns 200 with exactly their own registrations, each with `id`, `status`, `createdAt` and the event's `id`, `title`, `location` and `startsAt`, and none of another participant's; a participant with no registrations gets `registrations: []`. `GET /api/registrations/:id` for their own registration returns 200 with that registration.	FR-07, BR-05
AC-18	A participant's `GET /api/registrations/:id` for another participant's registration returns 404 `NOT_FOUND`, and the body contains no registration data.	FR-07, BR-05, D-13
AC-19	An organizer's `GET /api/events/:id/registrations` for an event they own returns 200 with exactly that event's registrations, each with `id`, `status`, `createdAt` and the participant's `name` and `email` only; an owned event with no registrations returns `registrations: []`.	FR-08, FR-15, BR-05
AC-20	Organizer B's `GET /api/events/:id/registrations` for organizer A's PUBLISHED event returns 403 `FORBIDDEN` with no registration data.	FR-08, BR-05, D-13
AC-21	A participant's `GET /api/events/:id/registrations` for a DRAFT event, a PUBLISHED event or an unknown event ID returns 403 `FORBIDDEN` with no registration data.	FR-08, NFR-03, D-13
AC-45	`GET /api/registrations/mine` and `GET /api/registrations/:id` with no cookie, with a token that matches no session, with an expired session (`expiresAt ⩽ now`) or with the cookie of a logged-out session return 401 `UNAUTHENTICATED` with no registration data.	NFR-01, FR-07, D-08
AC-46	An organizer's `GET /api/registrations/mine` and `GET /api/registrations/:id` (an existing or an unknown registration ID) return 403 `FORBIDDEN`. A participant's `GET /api/registrations/:id` with an unknown UUID returns 404 `NOT_FOUND`, and with a non-UUID `:id` returns 400 `VALIDATION_ERROR`.	NFR-03, NFR-07, FR-07, D-13
AC-47	`GET /api/events/:id/registrations` with no cookie, with a token that matches no session, with an expired session (`expiresAt ⩽ now`) or with the cookie of a logged-out session returns 401 `UNAUTHENTICATED` with no registration data.	NFR-01, FR-08, D-08
AC-48	Organizer B's `GET /api/events/:id/registrations` for organizer A's DRAFT event, or for an unknown event UUID, returns 404 `EVENT_NOT_FOUND`, and with a non-UUID `:id` returns 400 `VALIDATION_ERROR`; none of these responses contain registration data.	FR-08, BR-05, NFR-07, D-13

AC-17..AC-21 keep the numbers of `deliverable1-plan.md` section 5 and `docs/traceability.md`. AC-45..AC-48 are new: no existing AC covered the session, wrong-role, unknown-ID and invalid-ID cases of these three routes, and the DRAFT case of AC-20 (404 instead of 403, D-13) had no AC. `f-01-auth.md` TEST-42 says "Real endpoint matrices also required in F-02..F-06 contracts", and `docs/traceability.md` leaves the "complete DRAFT/access matrix" to #21 (FR-08 row). The numbers follow AC-44, the highest AC found on `sprint-1-staging`, every remote branch and the open PRs.

11. Software test specification (STS)

All tests below are API + DB integration tests (Vitest + Supertest, `DATABASE_URL_TEST`, tables truncated in `beforeEach`, D-14). Fixtures (users, events, registrations, sessions) are inserted directly in the database; "log in" means `POST /api/auth/login` with the fixture's password. Every error response is also asserted to have the shared envelope `{ "error": { "code", "message" } }`, plus `details` for 400 `VALIDATION_ERROR` only, and no `stack` field (NFR-07, system SDS 4.2): the 401, 403 and 404 responses have exactly the keys `code` and `message` in `error`; `error.message` equals the section 6 text for `FORBIDDEN`, `NOT_FOUND` and `EVENT_NOT_FOUND`, and is a nonempty string for 401 `UNAUTHENTICATED` (F-01 fixes no text for it); the 400 responses (TEST-46, TEST-48) have exactly the keys `code`, `message` and `details` in `error`, with the full envelope asserted in those rows. "No registration data" means the body has only the `error` key.

Shared fixtures: organizers O and O2; participants P1, P2, P3; E PUBLISHED owned by O, capacity 5, `startsAt` now + 7 days; E2 PUBLISHED owned by O2, capacity 5, future; D DRAFT owned by O, capacity 5, future. Registrations, all ACTIVE: R1 (P1, E), R2 (P2, E), R3 (P1, E2), R4 (P3, E2), with `createdAt` R1 < R2 < R3 < R4.

TEST	AC	Level	Setup	Steps	Expected
TEST-17	AC-17	API + DB integration	Shared fixtures, plus participant P4 with no registrations.	Log in as P1: `GET /api/registrations/mine`; `GET /api/registrations/R1`. Log in as P4: `GET /api/registrations/mine`.	P1 list: 200, exactly R3 then R1 (newest first), each with `status ACTIVE` and the matching event `id`, `title`, `location`, `startsAt`; R2 and R4 absent. R1: 200, `registration.id` R1, `registration.event.id` E. P4: 200, `registrations []`.
TEST-18	AC-18	API + DB integration	Shared fixtures.	Log in as P1: `GET /api/registrations/R2`.	404, `error.code NOT_FOUND`; body has only `error`.
TEST-19	AC-19	API + DB integration	Shared fixtures.	Log in as O: `GET /api/events/E/registrations`; `GET /api/events/D/registrations`.	E: 200, exactly R1 then R2 (oldest first), each with `id`, `status`, `createdAt` and `participant` with exactly the keys `name` and `email` of P1 and P2; R3, R4 absent; no `passwordHash` anywhere in the body. D: 200, `registrations []`.
TEST-20	AC-20	API + DB integration	Shared fixtures.	Log in as O2: `GET /api/events/E/registrations`.	403, `error.code FORBIDDEN`; body has only `error`.
TEST-21	AC-21	API + DB integration	Shared fixtures.	Log in as P1: `GET /api/events/E/registrations`; `GET /api/events/D/registrations`; `GET /api/events/<random UUID>/registrations`.	All three 403, `error.code FORBIDDEN`; each body has only `error`.
TEST-45	AC-45	API + DB integration	Shared fixtures, plus two `Session` rows for P1 with `expiresAt` in the past, inserted directly (system SDS 6, D-08): one with `tokenHash` = SHA-256 of a known token T1, one with SHA-256 of a different known token T2. F-01 deletes an expired session on first use, so each route gets its own expired token.	Call `GET /api/registrations/mine` and `GET /api/registrations/R1`, each (a) with no cookie, (b) with a random token as cookie, (c) with its own expired token as cookie (T1 for `/mine`, T2 for `/R1`), (d) after logging in as P1 and `POST /api/auth/logout`, with the old cookie.	All eight 401, `error.code UNAUTHENTICATED`; each body has only `error`.

TEST	AC	Level	Setup	Steps	Expected
TEST-46	AC-46	API + DB integration	Shared fixtures.	Log in as O: `GET /api/registrations/mine`; `GET /api/registrations/R1`; `GET /api/registrations/<random UUID>`. Log in as P1: `GET /api/registrations/<random UUID>`; `GET /api/registrations/not-a-uuid`.	O: all three 403, `error` exactly `{ "code": "FORBIDDEN", "message": "You do not have access to this." }`, body only `error`. P1 random UUID: 404, `error` exactly `{ "code": "NOT_FOUND", "message": "Registration not found." }`, body only `error`. P1 non-UUID: 400 with the envelope `{ "error": { "code": "VALIDATION_ERROR", "message": "<nonempty string>", "details": [{ "field": "id", "message": "<nonempty string>" }] } }` (system SDS 4.2): `error` has exactly the keys `code`, `message` and `details` (no `stack`); `error.message` is a nonempty string; `details` is a nonempty array whose items have exactly the keys `field` and `message`, both nonempty strings, and one item has `field` `id`; body only `error`.
TEST-47	AC-47	API + DB integration	Shared fixtures, plus a `Session` row for O with `tokenHash` = SHA-256 of a known token T and `expiresAt` in the past, inserted directly.	`GET /api/events/E/registrations` (a) with no cookie, (b) with a random token as cookie, (c) with T as cookie, (d) after logging in as O and `POST /api/auth/logout`, with the old cookie.	All four 401, `error.code` `UNAUTHENTICATED`; each body has only `error`.
TEST-48	AC-48	API + DB integration	Shared fixtures.	Log in as O2: `GET /api/events/D/registrations`; `GET /api/events/<random UUID>/registrations`; `GET /api/events/not-a-uuid/registrations`.	D and random UUID: 404, `error` exactly `{ "code": "EVENT_NOT_FOUND", "message": "Event not found." }`, body only `error`. Non-UUID: 400 with the envelope `{ "error": { "code": "VALIDATION_ERROR", "message": "<nonempty string>", "details": [{ "field": "id", "message": "<nonempty string>" }] } }` (system SDS 4.2): `error` has exactly the keys `code`, `message` and `details` (no `stack`); `error.message` is a nonempty string; `details` is a nonempty array whose items have exactly the keys `field` and `message`, both nonempty strings, and one item has `field` `id`; body only `error` (no `registrations` field).

Test names start with their IDs, for example `TEST-18 AC-18 returns 404 for another participant's registration`. TEST-18 and TEST-20 are TDD samples for the PDF: their failing runs are saved as `docs/evidence/tdd/TEST-18-red.txt` and `TEST-20-red.txt`, and their passing runs as `TEST-18-green.txt` and `TEST-20-green.txt` (team-workflow). The UI journey through My registrations and Event registrations is covered by TEST-22 (E2E, Issue 9.1), and My registrations after a full-event rejection by TEST-23 (E2E, Issue 9.2). The loading, empty and error states of both screens (NFR-09) are checked by hand in the 9.4 QA run checklist.

12. Definition of Done (F-05, F-06)

The shared Sprint 1 Definition of Done applies. In addition:

- F-05 (8.1, A): TEST-17, TEST-18, TEST-45 and TEST-46 exist, run and pass; none skipped.
- F-06 (8.2, B): TEST-19, TEST-20, TEST-21, TEST-47 and TEST-48 exist, run and pass; none skipped.
- The failing tests are committed before the implementation on each branch, and the failing runs of TEST-18 and TEST-20 are saved in `docs/evidence/tdd/`.
- Ownership is decided in the service layer from the session user (BR-05), not in the UI.
- Both screens show loading, empty and error states (NFR-09).
- Each PR is reviewed by the other student and merged into `sprint-1-staging` with a merge commit.

13. Version history

Version	Date	Change	Author	Reviewer
1.0	2026-11-07	First F-05 and F-06 contract (Issue #21)	B (stickkersz)	A (songt888)

Version	Date	Change	Author	Reviewer
1.1	2026-11-07	Review round 1 (PR #55): TEST-45 uses a separate expired token per route (F-01 deletes a found expired row); every error case asserts `error.message` (section 6 text, nonempty for 401); TEST-46 and TEST-48 assert the full validation envelope with a `details` item for `id` (system SDS 4.2)	B (stickkersz)	A (songt888)

Sprint 1 — Product Definition of Done

This is the single product checklist owned by #17; other feature contracts refer to it.

- □ Required behavior and backend rules match the merged feature contracts; UI → API → persistent DB journey works.
- □ Contract revision is committed before real failing tests, then passing implementation/refactor commits; merge commits preserve history.
- □ Planned applicable tests execute and pass, with no required test skipped, weakened or silently removed; failures/skips and incomplete work disclosed.
- □ Other student writes a specific review/check and valid approval on the current head; PR is merged to sprint-1-staging. AI assistance does not substitute for teammate review.
- □ Integration build and full applicable suite pass on the recorded staging state. QA #40 records TEST-15 and separate TEST-37 five times each and the NFR-06/08/09 checks.
- □ README, environment example, contracts and traceability agree; teammate verifies setup from a fresh clone in at most 10 documented commands.
- □ Evidence records real commands, exact tested commit, output/counts and automated/manual distinction in docs/evidence/ac-test-results.md; TDD samples retain raw red/green output as required by team workflow.
- □ Reviewed release PR staging → main is merged; build, full required tests and planned acceptance checks run on that exact main commit before v0.1.0-sprint1 is created. Remote tag resolves to the verified commit.
- □ Screenshots/final results come from the tag; evidence/PDF PRs target staging only and main remains frozen through submission. Final PDF includes Parts A–F, real page numbers, contribution history and honest AI-use/limitations declaration.

These unchecked items are completion gates, not a claim that any has passed. Documentation-only contract preparation uses document checks and no fabricated red/green run.

Version history

Version	Date	Change	State
1.0	2026-11-07	Initial shared DoD from master plan and merged release workflow, owned by #17	Pending teammate review; no completed checks claimed

Part F. Working Sprint 1 and evidence

Release: tag `v0.1.0-sprint1`, commit `12dcd3c` on `main` (the merge of release PR #79, 2026-11-08). Every program screenshot, build output and test count in this Part comes from that commit, run in a clean checkout with `npm ci`. Exception: the evidence files themselves and this PDF were added afterwards by PRs into `sprint-1-staging` (#82 and the 10.3 PR), because `main` stays frozen at the tag; they change no code.

F1. Setup summary

Full steps are in the repository `README.md`. Dependencies: Node.js 20 or later, npm, Docker (PostgreSQL 16 runs in Docker Compose with three databases: dev, test and E2E).

Step	Command
Clone	`git clone https://github.com/songt888/event-registration-system.git`, then `cd event-registration-system`
Configure	`cp .env.example server/.env` (placeholder values only; `SEED_PASSWORD` is a local value you choose)
Database	`docker compose up -d db`
Install	`npm install` (also generates the Prisma client)
Migrate and seed	`npm run migrate`, then `npm run seed` (synthetic accounts: organizer1, organizer2, participant1 to participant4 at example.com)
Run	`npm run dev` (API on port 3000, UI on port 5173)
Test	`npm test` (unit and API/DB integration on `DATABASE_URL_TEST`), `npm run test:e2e` (Playwright on `DATABASE_URL_E2E`)

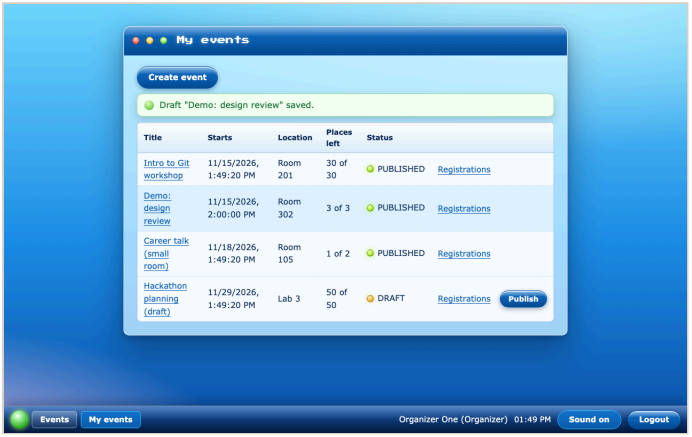
Environment variables (names only, values in `.env.example`): `DATABASE_URL`, `DATABASE_URL_TEST`, `DATABASE_URL_E2E`, `PORT`, `NODE_ENV`, `SESSION_TTL_MINUTES`, `COOKIE_SECURE`, `SEED_PASSWORD`. No real password, token or secret is in Git or in this PDF. The setup from a fresh clone takes 8 commands (NFR-08, verified by B on 2026-11-07).

F2. Program demonstrations

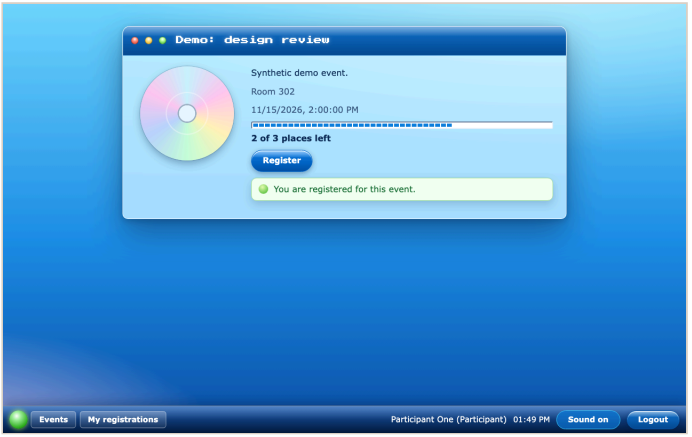
The capture script (capture.spec.ts, config capture.config.ts) resets and seeds the E2E database like `npm run test:e2e` (D-14), starts the server and client, and asserts every expected result below before it saves a screenshot. API calls go through the signed-in browser session; the JSON file records request, expected and actual status and body (no cookies or passwords).

Scenario 1: golden journey and persisted retrieval

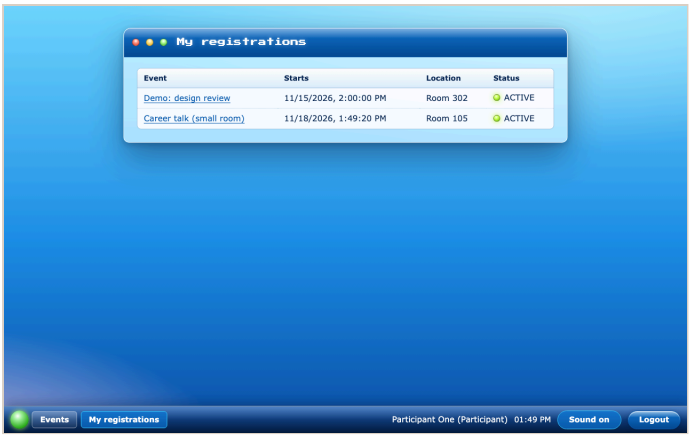
Item	Content
IDs	AC-22/TEST-22, AC-17/TEST-17, AC-19/TEST-19; FR-04, FR-06, FR-07, FR-08
Roles	organizer1 (ORGANIZER), participant1 (PARTICIPANT)
Start condition	Seeded database; no event called "Demo: design review"
Steps	Organizer creates the DRAFT (capacity 3, a week ahead) and publishes it; participant registers on the event page; participant opens My registrations and reloads the page; organizer opens the event's registrations
Expected	PUBLISHED in My events; "You are registered" and 2 of 3 places left; after reload the registration is still ACTIVE (read back from the database); organizer sees Participant One with email and ACTIVE
Actual	as expected: 1-1, 1-2, 1-3, 1-4; `GET /api/registrations/mine` 200 including the event



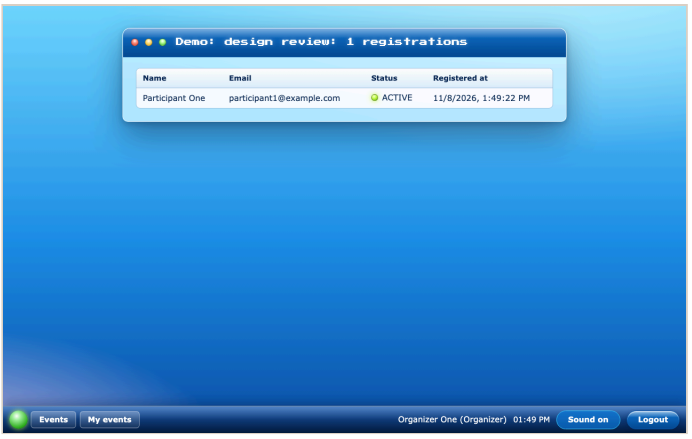
1-1-organizer-published



1-2-participant-registered



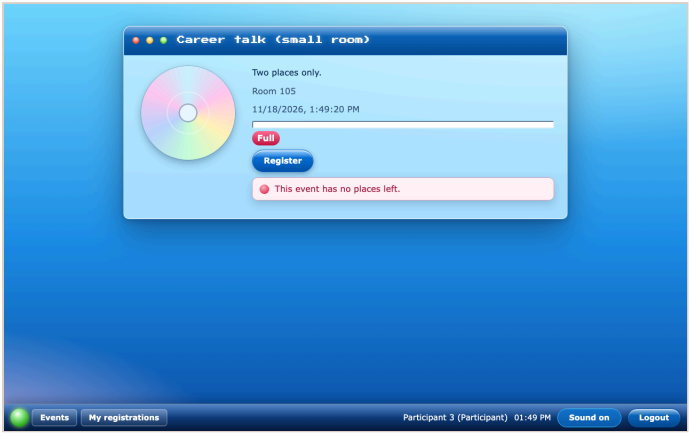
1-3-my-registrations-after-reload



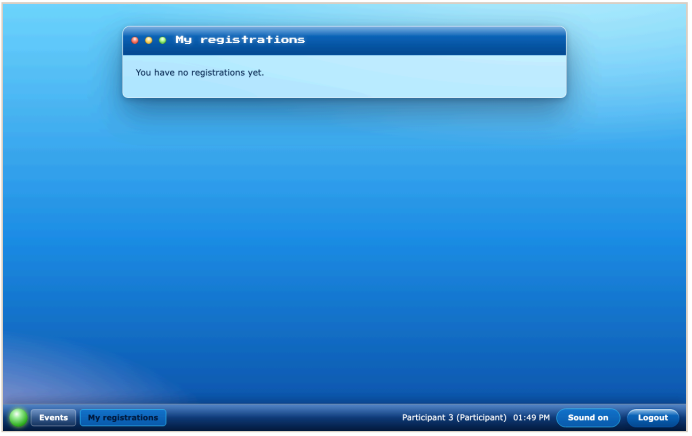
1-4-organizer-sees-registrant

Scenario 2: rule rejection (full event and duplicate)

Item	Content
IDs	AC-23/TEST-23, AC-13/TEST-13, AC-14/TEST-14; FR-06, BR-02, BR-03
Roles	participant2, participant3 (PARTICIPANT)
Start condition	Seed: "Career talk (small room)", capacity 2, one ACTIVE registration (participant1)
Steps	participant2 takes the last place; participant3 clicks Register; participant3 opens My registrations; then by API participant3 registers again and participant2 registers a second time
Expected	participant3 sees "This event has no places left." and has no registrations; API 409 EVENT_FULL and 409 ALREADY_REGISTERED; remaining places 0, never below
Actual	as expected: 2-1, 2-2; API 409 EVENT_FULL, 409 ALREADY_REGISTERED, `remainingPlaces` 0 of capacity 2



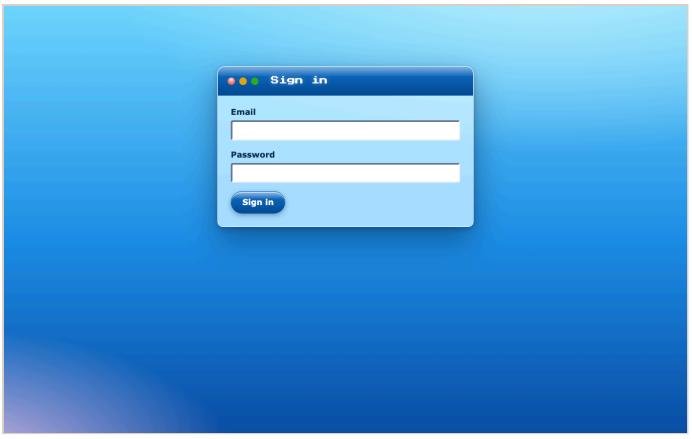
2-1-full-event-rejected



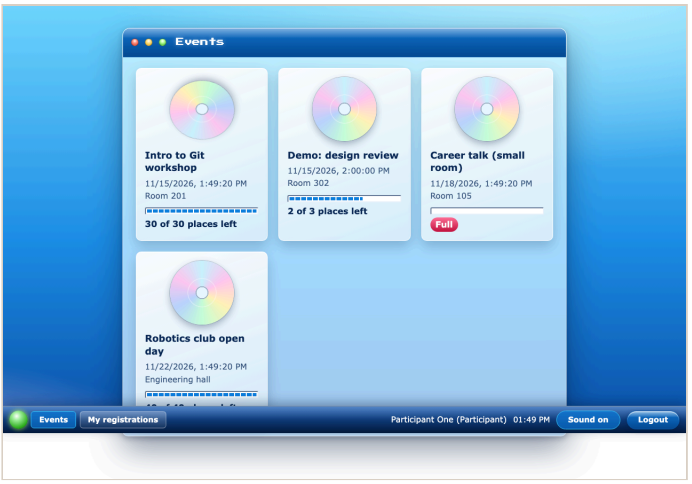
2-2-my-registrations-unchanged

Scenario 3: login, logout and forbidden

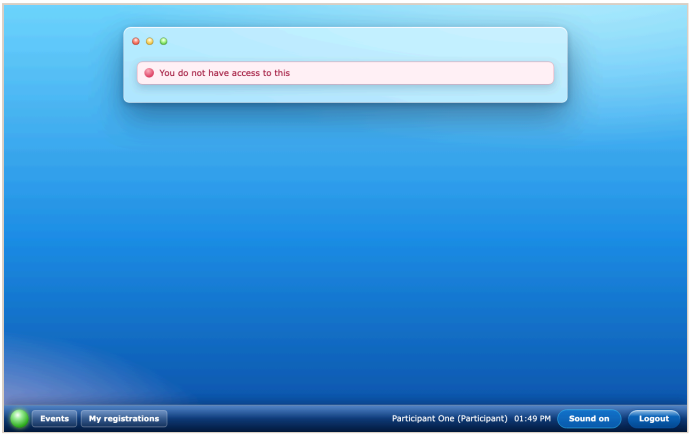
Item	Content
IDs	AC-24/TEST-24, AC-04/TEST-04, AC-21/TEST-21, AC-20/TEST-20, AC-16/TEST-16, AC-18/TEST-18; FR-01, FR-02, FR-03, BR-05, NFR-03
Roles	participant1, participant2 (PARTICIPANT), organizer2 (ORGANIZER, does not own the seed events of organizer1)
Start condition	Seeded database after scenarios 1 and 2
Steps	Open the login page; sign in as participant1; open an organizer registrations page by URL; log out and open a protected page; replay the old session cookie on `GET /api/me`; organizer2 opens organizer1's event registrations and tries to register; participant2 asks for participant1's registration by ID
Expected	header "Participant One (Participant)"; participant gets "You do not have access to this" and API 403; after logout the protected page opens login and the old cookie gets 401; organizer2 gets 403 on the list and 403 on register; participant2 gets 404
Actual	as expected: 3-1, 3-2, 3-3, 3-4, 3-5; API 403 FORBIDDEN (AC-21), 401 UNAUTHENTICATED (AC-04), 403 FORBIDDEN (AC-20), 403 FORBIDDEN (AC-16), 404 NOT_FOUND (AC-18)



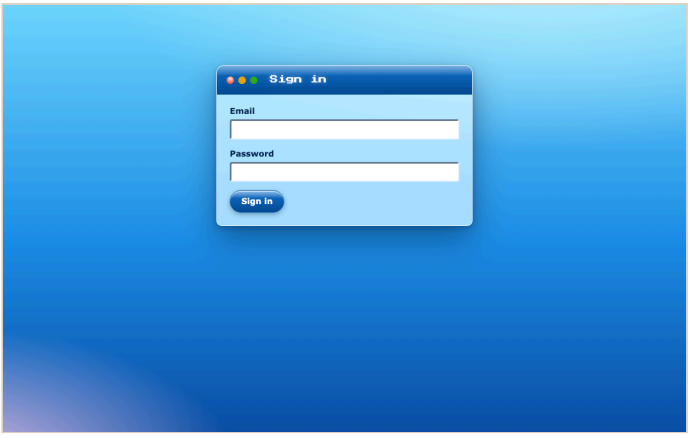
3-1-login



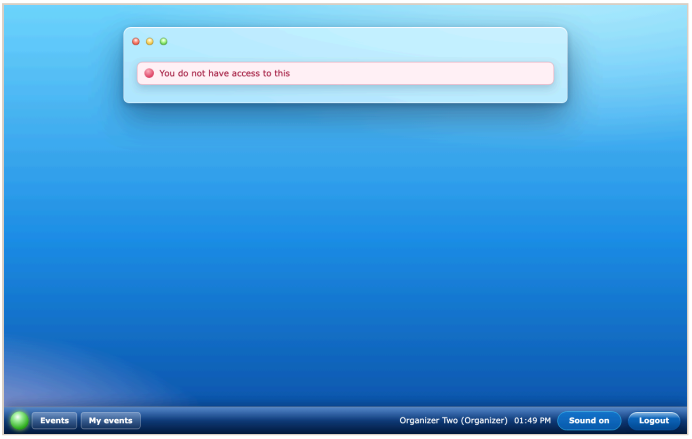
3-2-signed-in-participant



3-3-participant-forbidden



3-4-after-logout-protected-page



3-5-other-organizer-forbidden

Backend and API evidence for the protected operations

Recorded by the capture script through the signed-in browser session on the release commit (`docs/evidence/release/demo/api-evidence.json`). Hiding a button does not prove authorization: these calls go straight to the API.

Scenario	AC	Actor	Request	Expected	Actual
1	AC-17	participant1	`GET /api/registrations/mine`	200, includes the demo event	200
2	AC-13	participant3	`POST /api/events/99c80631-6cf9-4516-b213-ac6ca9f29c7a/registrations`	409 EVENT_FULL	409 EVENT_FULL
2	AC-14	participant2	`POST /api/events/99c80631-6cf9-4516-b213-ac6ca9f29c7a/registrations`	409 ALREADY_REGISTERED	409 ALREADY_REGISTERED

Scenario	AC	Actor	Request	Expected	Actual
2	AC-13	participant3	`GET /api/events/99c80631-6cf9-4516-b213-ac6ca9f29c7a`	remainingPlaces 0 (never below)	200
3	AC-21	participant1	`GET /api/events/99c80631-6cf9-4516-b213-ac6ca9f29c7a/registrations`	403 FORBIDDEN	403 FORBIDDEN
3	AC-04	participant1 (cookie from before logout)	`GET /api/me`	401 UNAUTHENTICATED	401 UNAUTHENTICATED
3	AC-20	organizer2	`GET /api/events/99c80631-6cf9-4516-b213-ac6ca9f29c7a/registrations`	403 FORBIDDEN	403 FORBIDDEN
3	AC-16	organizer2	`POST /api/events/fc1318ef-071e-4f51-9177-97321ad7d537/registrations`	403 FORBIDDEN	403 FORBIDDEN
3	AC-18	participant2	`GET /api/registrations/5782fa69-4af9-4079-91e0-f9845a9ffa2d`	404 NOT_FOUND	404 NOT_FOUND

F3. Build and automated test results

Tested commit: `12dcd3c` (tag `v0.1.0-sprint1`). Commands and counts from `docs/evidence/release/README.md`; the raw output is in `docs/evidence/release/logs/`.

Command	Result	Output
`npm run build`	passed (server `prisma generate && tsc`, client `tsc -b && vite build`)	logs/build.txt
`npm test`	12 files, 67 tests: 67 passed, 0 failed, 0 skipped	logs/test.txt
`npm run test:e2e`	5 tests: 5 passed, 0 failed, 0 skipped	logs/e2e.txt
`npm test -w server -- -t "TEST-15|TEST-37"`, 5 runs	each run 2 passed; 65 not selected by the name filter	logs/concurrency.txt
`npx playwright test -c docs/evidence/release/capture.config.ts`	8 passed: 3 demo scenarios, 5 NFR-09 screens	screenshots below, demo/api-evidence.json, nfr-09/results.json

Automated: every row above. Each screenshot is taken only after the script asserted its expected text; a sample (1-4, 2-1, 3-3, `nfr-09/event-detail-error`) was also checked by eye before commit. Local file paths in the logs are replaced by `<release checkout>`.

Output: `npm test`

```
✓ |unit| src/events.service.unit.test.ts (8 tests) 4ms
 ✓ |unit| src/auth/session.unit.test.ts (1 test) 1ms
 ✓ |unit| src/registrations.service.unit.test.ts (1 test) 1ms
 ✓ |integration| src/events.int.test.ts (24 tests) 419ms
 ✓ |integration| src/auth/auth.int.test.ts (8 tests) 405ms
 ✓ |integration| src/registrations.int.test.ts (7 tests) 242ms
 ✓ |integration| src/my-registrations.int.test.ts (4 tests) 152ms
 ✓ |integration| src/event-registrations.int.test.ts (5 tests) 164ms
 ✓ |integration| src/registrations.limits.int.test.ts (3 tests) 207ms
 ✓ |integration| src/auth/guards.int.test.ts (3 tests) 53ms
 ✓ |integration| src/seed.int.test.ts (1 test) 1765ms
 ✓ |integration| src/foundation.int.test.ts (2 tests) 39ms
 Test Files  12 passed (12)
      Tests  67 passed (67)
   Start at  13:46:25
   Duration  6.01s (transform 77ms, setup 25ms, collect 1.98s, tests 3.45s, environment 0ms, prepare 107ms)
```

Output: `npm run test:e2e`

```
Running generate... (Use --skip-generate to skip the generators)
 [2K [1A [2K [GRunning generate... - Prisma Client
Running 5 tests using 1 worker
  ✓  1 [chromium] › e2e/browse.spec.ts:4:1 › TEST-11 AC-11 Events screen shows the empty state for an empty list (1.1s)
  ✓  2 [chromium] › e2e/full-event.spec.ts:12:1 › TEST-23 AC-23 full event shows the full message and My registrations stays unchanged (3.7s)
  ✓  3 [chromium] › e2e/golden-journey.spec.ts:19:1 › TEST-22 AC-22 organizer publishes, participant registers and sees it, owner sees the registrant (4.7s)
  ✓  4 [chromium] › e2e/golden-journey.spec.ts:53:1 › TEST-24 AC-24 header shows name and role for each role; after logout a protected page opens login (2.8s)
```

```
  ✓  5 [chromium] › e2e/smoke.spec.ts:5:1 › signed-out visitor is sent to the login form (132ms)
  5 passed (15.6s)
```

Output: TEST-15 and TEST-37, five runs (NFR-04)

```
$ npm test -w server -- -t "TEST-15|TEST-37"   # run 1
      Tests  2 passed | 65 skipped (67)
$ npm test -w server -- -t "TEST-15|TEST-37"   # run 2
      Tests  2 passed | 65 skipped (67)
$ npm test -w server -- -t "TEST-15|TEST-37"   # run 3
      Tests  2 passed | 65 skipped (67)
$ npm test -w server -- -t "TEST-15|TEST-37"   # run 4
      Tests  2 passed | 65 skipped (67)
$ npm test -w server -- -t "TEST-15|TEST-37"   # run 5
      Tests  2 passed | 65 skipped (67)
```

AC to TEST to result table

Automated tests are labelled by level in the table (unit, API/DB integration, E2E); the only manual checks are the NFR rows in the next table. Result column: the full suite on the release commit (section 5 of the results file confirms every row again on `12dcd3c`).

Server files are under `server/src/`. Every test name begins with its TEST and AC IDs. Result is from the full run above.

AC	TEST	File and test name	Result
AC-01	TEST-01	`auth/auth.int.test.ts` logs in either role, sets the sid cookie, stores only the token hash with 60-minute expiry	pass
AC-02	TEST-02	`auth/auth.int.test.ts` same 401 INVALID_CREDENTIALS for wrong password and unknown email	pass
AC-03	TEST-03	`auth/auth.int.test.ts` GET /api/me without cookie returns 401	pass
AC-04	TEST-04	`auth/auth.int.test.ts` logout deletes only the current session (2 tests)	pass
AC-05	TEST-05	`seed.int.test.ts` seeded passwords are bcrypt cost 12 hashes	pass
AC-06	TEST-06	`events.int.test.ts` creates a DRAFT with capacity 1 and 10000 (2 tests)	pass
AC-07	TEST-07	`events.service.unit.test.ts` (5 tests) and `events.int.test.ts` (5 tests) reject invalid capacity	pass
AC-08	TEST-08	`events.int.test.ts` participant create returns 403	pass
AC-09	TEST-09	`events.int.test.ts` 401 on every F-02 route	pass
AC-10	TEST-10	`events.int.test.ts` lists only PUBLISHED with remaining places; empty list (2 tests)	pass
AC-11	TEST-11	`e2e/browse.spec.ts` Events screen shows the empty state	pass
AC-12	TEST-12	`registrations.int.test.ts` registers and lowers remainingPlaces by one	pass
AC-13	TEST-13	`registrations.int.test.ts` rejects registration when event is full	pass
AC-14	TEST-14	`registrations.int.test.ts` second registration rejected, DB refuses second ACTIVE row	pass
AC-15	TEST-15	`registrations.int.test.ts` last place goes to exactly one concurrent request	pass (5 extra runs, all pass)
AC-16	TEST-16	`registrations.int.test.ts` organizers get 403	pass
AC-17	TEST-17	`my-registrations.int.test.ts` only own registrations, newest first, one by ID	pass
AC-18	TEST-18	`my-registrations.int.test.ts` 404 for another participant's registration	pass
AC-19	TEST-19	`event-registrations.int.test.ts` owned event registrations, oldest first, name and email only	pass
AC-20	TEST-20	`event-registrations.int.test.ts` 403 for another organizer on a PUBLISHED event	pass

AC	TEST	File and test name	Result
AC-21	TEST-21	`event-registrations.int.test.ts` 403 for a participant on any event ID	pass
AC-22	TEST-22	`e2e/golden-journey.spec.ts` organizer publishes, participant registers and sees it, owner sees the registrant	pass
AC-23	TEST-23	`e2e/full-event.spec.ts` full event shows the full message and My registrations stays unchanged	pass
AC-24	TEST-24	`e2e/golden-journey.spec.ts` header shows name and role for each role; after logout a protected page opens login	pass
AC-25	TEST-25	`events.int.test.ts` publishes the owner DRAFT, persists PUBLISHED with publishedAt	pass
AC-26	TEST-26	`events.int.test.ts` 404 when another organizer publishes the DRAFT	pass
AC-27	TEST-27	`registrations.int.test.ts` 404 for DRAFT or unknown event	pass
AC-28	TEST-28	`registrations.service.unit.test.ts` and `registrations.int.test.ts` EVENT_STARTED (2 tests)	pass
AC-29	TEST-29	`events.service.unit.test.ts` and `events.int.test.ts` past startsAt rejected (2 tests)	pass
AC-30	TEST-30	`events.service.unit.test.ts` and `events.int.test.ts` publish with past or equal start rejected (2 tests)	pass
AC-31	TEST-31	`events.int.test.ts` owner DRAFT by ID	pass
AC-32	TEST-32	`events.int.test.ts` same 404 for a DRAFT and a missing ID	pass
AC-33	TEST-33	`events.int.test.ts` lists only the caller's events ordered by startsAt then id	pass
AC-34	TEST-34	`events.int.test.ts` participant My events returns 403	pass
AC-35	TEST-35	`events.service.unit.test.ts` startsAt equal to now rejected on create	pass
AC-36	TEST-36	`events.int.test.ts` 409 on republish	pass
AC-37	TEST-37	`registrations.limits.int.test.ts` 20 participants, capacity 5, exactly 5 ACTIVE	pass (5 extra runs, all pass)
AC-38	TEST-38	`events.int.test.ts` 401 and no event data for bad sessions	pass
AC-39	TEST-39	`auth/auth.int.test.ts` GET /api/me returns exactly id, name, role	pass
AC-40	TEST-40	`auth/session.unit.test.ts` and `auth/auth.int.test.ts` expired sessions (2 tests)	pass
AC-41	TEST-41	`auth/auth.int.test.ts` invalid login requests return 400	pass
AC-42	TEST-42	`auth/guards.int.test.ts` 401 before role, 403 wrong role, identity from session (3 tests)	pass
AC-43	TEST-43	`registrations.limits.int.test.ts` 401 for bad sessions, nothing stored	pass
AC-44	TEST-44	`registrations.limits.int.test.ts` participantId body and non-UUID rejected	pass
AC-45	TEST-45	`my-registrations.int.test.ts` 401 on both routes	pass
AC-46	TEST-46	`my-registrations.int.test.ts` 403 organizer, 404 unknown, 400 non-UUID	pass
AC-47	TEST-47	`event-registrations.int.test.ts` 401 for bad sessions	pass
AC-48	TEST-48	`event-registrations.int.test.ts` 404 foreign DRAFT or unknown, 400 non-UUID	pass
AC-49	TEST-49	`events.int.test.ts` PUBLISHED event to every role with remainingPlaces	pass

AC	TEST	File and test name	Result
AC-50	TEST-50	`events.int.test.ts` role before lookup, ownership before state on publish	pass
AC-51	TEST-51	`events.int.test.ts` malformed create, publish and ID input rejected	pass

Summary: 51 of 51 AC have a passing test on the staging head. Also in the full run, `foundation.int.test.ts` (2 tests, health and test-database isolation) has no AC.

NFR checks

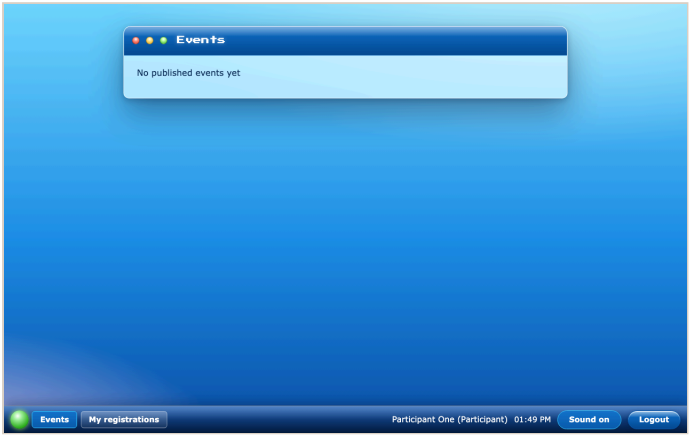
NFR	Check	Result
NFR-04	TEST-37, five runs	pass 5 of 5 (section 1)
NFR-08	B (teammate of the README author) follows `README.md` from a fresh clone at `c1ee1ce`: clone, cd, cp, docker compose, npm install, migrate, seed, dev; then `npm test`	pass: 8 commands (limit 10); `npm install` 261 packages with Prisma Client generated, migrate and seed OK, UI 200, API 401 without a session, `npm test` 67 of 67; recorded on Issue #3
NFR-06	on the dev database: organizer1 creates and publishes a synthetic event, participant1 registers; read both back with `curl`; stop the server (port 3000 free); start it again without a reset; log in again and read both back	pass: event (PUBLISHED, `remainingPlaces` 4) and registration (ACTIVE) are byte-identical before and after the restart; server code identical to `c1ee1ce`
NFR-09	loading, empty and error states on the five S1 screens (system SDS 3.2), checklist in section 3.1	pass: 14 of 14 checks

NFR-09 rerun on the release commit

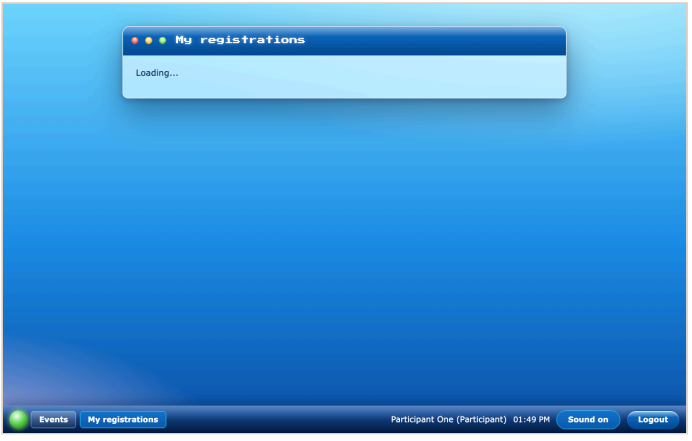
Same 14 checks as the first run on `827f874` (ac-test-results.md section 3.1), now on the Y2K UI after PRs #76, #77 and #81. Each state is forced on one API call in a fresh page: **loading** never answers the call, **empty** returns the real response with its list emptied, **error** returns 500 `INTERNAL_ERROR` "Something went wrong. Please try again." The script also checks that the window buttons do not cover the message (the PR #81 fix).

Screen	Role	Loading	Empty	Error
Events list `/events`	participant	pass: "Loading events..."	pass: "No published events yet"	pass: message in an alert
Event detail `/events/:id`	participant	pass: "Loading event..."	not applicable (one record)	pass: message in an alert
My registrations `/my/registrations`	participant	pass: "Loading..."	pass: "You have no registrations yet."	pass: message in an alert
My events `/organizer/events`	organizer	pass: "Loading..."	pass: "No events yet"	pass: message in an alert, plus "Try again"
Event registrations `/organizer/events/:id/registrations`	organizer	pass: "Loading..."	pass: "No registrations yet."	pass: message in an alert

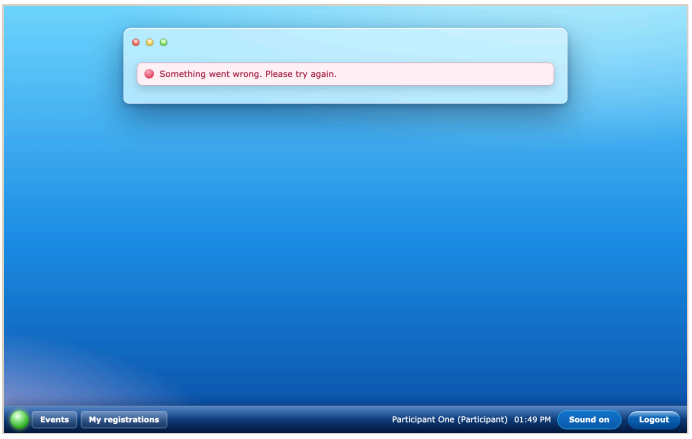
Result: 14 of 14 pass; window buttons clear of the message in 14 of 14. Screenshots: nfr-09/.



events-list-empty



my-registrations-loading



event-detail-error

F4. Test excerpts and TDD evidence

Two excerpts of real automated tests, with their setup and assertions, and their red and green runs. The red run is the commit where the tests existed and the behavior did not; the green run is after the implementation.

TEST-13: domain rule: capacity (BR-02, AC-13)

A participant cannot register for a full event: the API answers 409 EVENT_FULL and nothing is stored. Source: `server/src/registrations.int.test.ts`, line 70.

```
function seedEvent(ownerId: string, data: { status?: 'DRAFT' | 'PUBLISHED'; startsAt?: Date; capacity?: number }
= {}) {
  const status = data.status ?? 'PUBLISHED';
  return prisma.event.create({
    data: {
      ownerId,
      title: 'Seeded event',
      description: '',
      location: 'Hall A',
      startsAt: new Date(Date.now() + 7 * DAY),
      capacity: 5,
      status,
      publishedAt: status === 'PUBLISHED' ? new Date(Date.now() - DAY) : null,
      ...data,
    },
  });
}

const register = (eventId: string, cookie?: string) => {
  const req = request(app).post(`/api/events/${eventId}/registrations`);
  return (cookie ? req.set('Cookie', cookie) : req).send({});
};

  it('TEST-13 AC-13 rejects registration when event is full', async () => {
    const o = await createUser('ORGANIZER', 'o@example.com');
    const [p1, p2, p3] = await participants(3);
    const e = await seedEvent(o.id, { capacity: 2 });
    await prisma.registration.createMany({
      data: [p1, p2].map((p) => ({ eventId: e.id, participantId: p.user.id, status: 'ACTIVE' as const })),
    });
    expectError(await register(e.id, p3.cookie), 409, 'EVENT_FULL');
    expect(await registrations()).toHaveLength(2);
    expect(await prisma.registration.count({ where: { participantId: p3.user.id } })).toBe(0);
  });
```

Red: before the implementation

```
Command: npm test -- registrations
Tested commit: 6153fbb (feature/s1-07-register)
Result: 2 test files failed; 7 integration tests failed, 0
passed; the unit file fails to load

 FAIL |integration| src/registrations.int.test.ts > F-04
register API > TEST-13 AC-13 rejects registration when event
is full
 AssertionError: expected 404 to be 409 // Object.is equality
 (POST /api/events/:id/registrations does not exist yet:
"Cannot POST")
```

Green: after the implementation

```
Command: npm test -- registrations
Tested commit: e3c3933 (feature/s1-07-register)
Result: 3 test files passed; 11 tests passed, 0 failed

 ✓ |integration| src/registrations.int.test.ts > F-04
register API > TEST-13 AC-13 rejects registration when event
is full 32ms

 Test Files  3 passed (3)
      Tests  11 passed (11)
```

```
 Other failing tests, same reason (route missing):
 TEST-12, TEST-14, TEST-15, TEST-16, TEST-27, TEST-28
(integration)
 TEST-28 (unit file): Cannot find module
'./registrations.service.js'

 Test Files  2 failed (2)
      Tests  7 failed (7)
```

TEST-20: role and ownership restriction (BR-05, NFR-03, AC-20)

An organizer who does not own a PUBLISHED event cannot read its registrations: 403 FORBIDDEN from the backend. Source: `server/src/event-registrations.int.test.ts`, line 83.

```
async function login(email: string) {
  const res = await request(app).post('/api/auth/login').send({ email, password: PASSWORD });
  return `sid=${sidFrom(res)}`;
}

const list = (eventId: string, cookie?: string) => {
  const req = request(app).get(`/api/events/${eventId}/registrations`);
  return cookie ? req.set('Cookie', cookie) : req;
};

async function fixtures() {
  const o = await createUser('ORGANIZER', 'o@example.com');
  const o2 = await createUser('ORGANIZER', 'o2@example.com');
  const p1 = await createUser('PARTICIPANT', 'p1@example.com');
  const p2 = await createUser('PARTICIPANT', 'p2@example.com');
  const p3 = await createUser('PARTICIPANT', 'p3@example.com');
  const e = await event(o.id, 'PUBLISHED');
  const e2 = await event(o2.id, 'PUBLISHED');
  const d = await event(o.id, 'DRAFT');
  const r1 = await registration(e.id, p1.id, 40);
  const r2 = await registration(e.id, p2.id, 30);
  await registration(e2.id, p1.id, 20);
  await registration(e2.id, p3.id, 10);
  return { o, o2, e, d, r1, r2 };
}

  it('TEST-20 AC-20 returns 403 for another organizer on a PUBLISHED event', async () => {
    expectError(await list(f.e.id, await login('o2@example.com')), 403, 'FORBIDDEN', 'You do not have access to
this.');
  });
```

Red: before the implementation

```
Command: npm test -- event-registrations
Tested commit: e30f6f8 (feature/s1-08-organizer-view)
Result: 1 test file failed; 5 integration tests failed, 0
passed

 FAIL |integration| src/event-registrations.int.test.ts > F-
06 event registrations API > TEST-20 AC-20 returns 403 for
another organizer on a PUBLISHED event
 AssertionError: expected 404 to be 403 // Object.is equality
 (GET /api/events/:id/registrations does not exist yet:
"Cannot GET")

 Other failing tests, same reason (route missing):
 TEST-19, TEST-21, TEST-47, TEST-48 (integration)

 Test Files  1 failed (1)
      Tests  5 failed (5)
```

Green: after the implementation

```
Command: npm test
Tested commit: b8956d5 (feature/s1-08-organizer-view)
Result: 11 test files passed, 63 tests passed, 0 failed, 0
skipped

 ✓ |integration| src/event-registrations.int.test.ts (5
tests)
   TEST-19, TEST-20, TEST-21, TEST-47, TEST-48 pass

 Test Files  11 passed (11)
      Tests  63 passed (63)
```

The other TDD samples named in the team workflow (TEST-07 and TEST-18) are in `docs/evidence/tdd/` with the same red and green format.

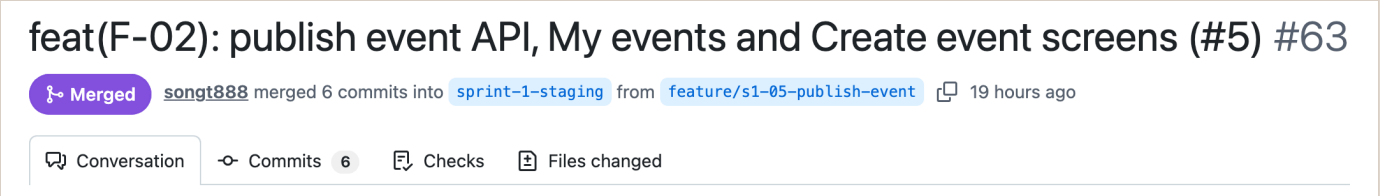
F5. Engineering workflow and history

The team worked in GitHub Issues and feature branches; every PR was reviewed by the other student and merged into `sprint-1-staging` with a merge commit; the release PR #79 merged staging into `main`. Two completed Issues, one authored by each student:

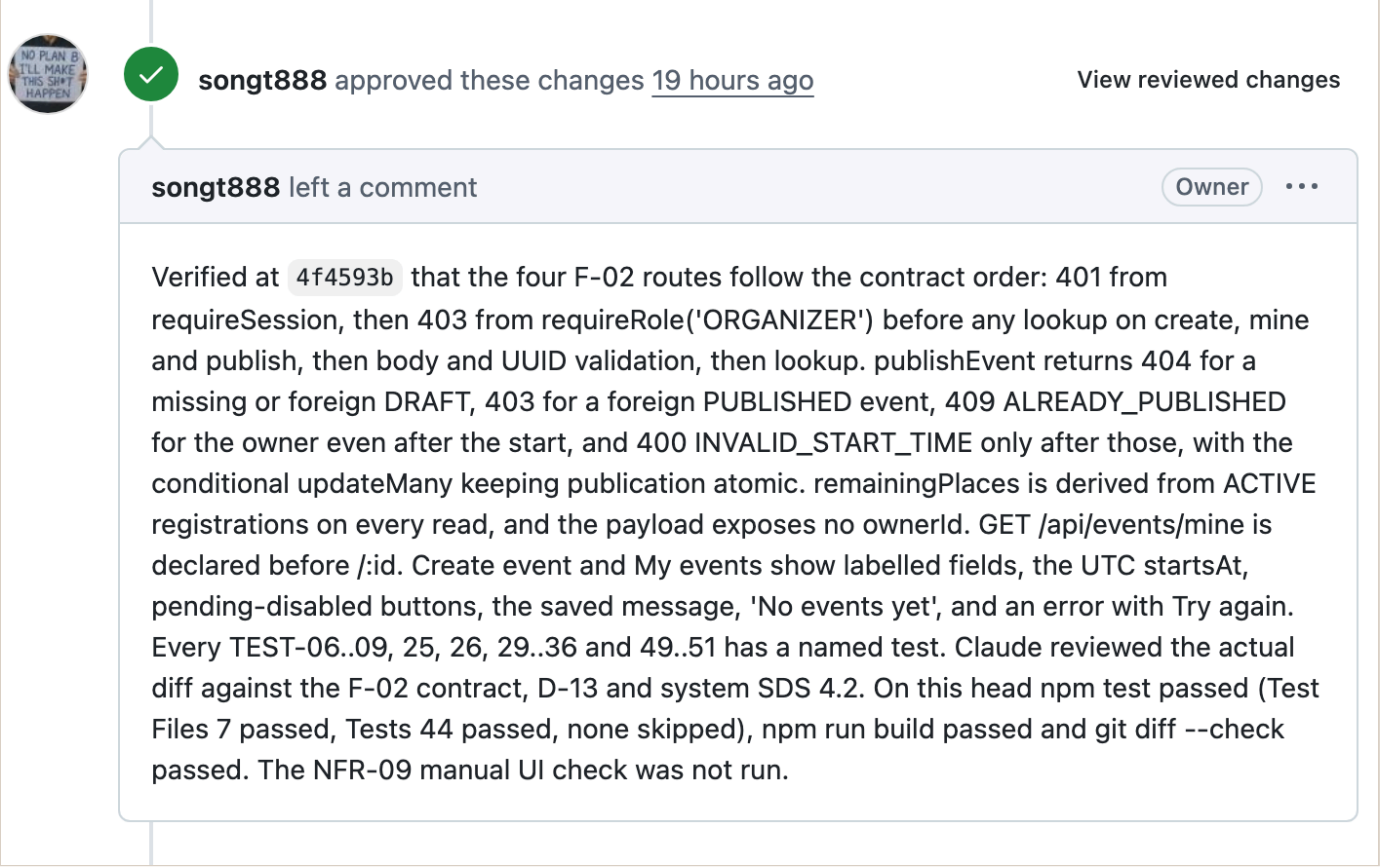
Issue #5: F-02 Publish event (B authors, A reviews)

Contract	PR #56 *docs(F-02): specify event publication and shared detail contract*, branch `docs/s1-02-f02`, author songt888, merged 2026-11-07 08:35 UTC

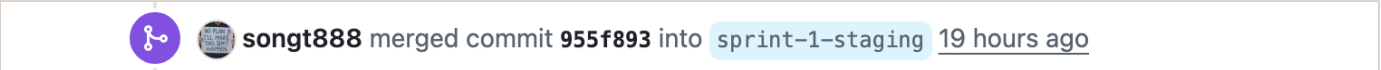
Implementation PR	PR #63 *feat(F-02): publish event API, My events and Create event screens (#5)*
Branch	`feature/s1-05-publish-event` into `sprint-1-staging`
Author / reviewer	stickkersz / songt888
Reviews	songt888: approved at `4f4593b` (2026-11-07 13:18 UTC)
Substantive review (excerpt)	Verified at `4f4593b` that the four F-02 routes follow the contract order: 401 from requireSession, then 403 from requireRole('ORGANIZER') before any lookup on create, mine and publish, then body and UUID validation, then lookup. publishEvent returns 404 for a missing or foreign DRAFT, 403 for a foreign PUBLISHED event, 409 ALREADY_PUBLISHED for the owner even after the start, ...
Into staging	merge commit `955f893` by songt888, 2026-11-07 13:19 UTC
Into main	release PR #79, merge commit `12dcd3c`, tag `v0.1.0-sprint1`



PR #63 on GitHub (logged out): title, merged state, branch into sprint-1-staging



PR #63: review 1



PR #63: merge into sprint-1-staging

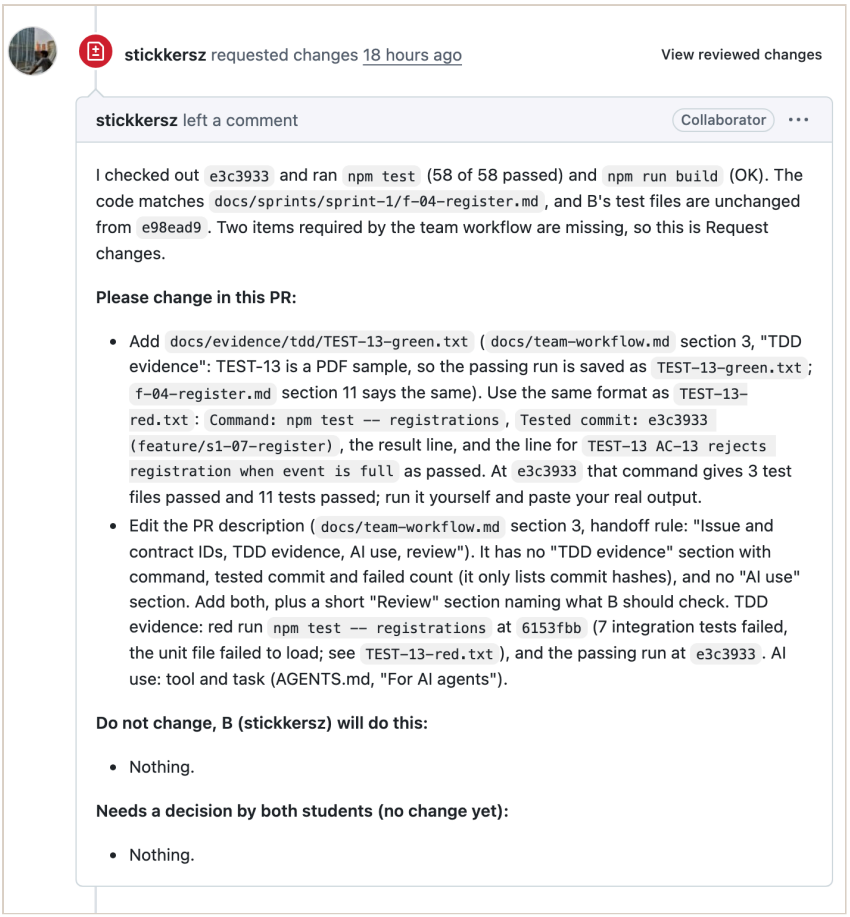
Issue #7: F-04 Register for event (A implements on B's red tests, B reviews)

Contract	PR #54 *docs(F-04): specify Register contract (FR-06, BR-02..04, NFR-04, AC-12..16, AC-27, AC-28, AC-37, AC-43, AC-44)*, branch `docs/s1-02-f04`, author stickkersz, merged 2026-11-07 08:50 UTC
Implementation PR	PR #65 *feat(F-04): register for an event (Issue #7)*
Branch	`feature/s1-07-register` into `sprint-1-staging`
Author / reviewer	songt888 / stickkersz

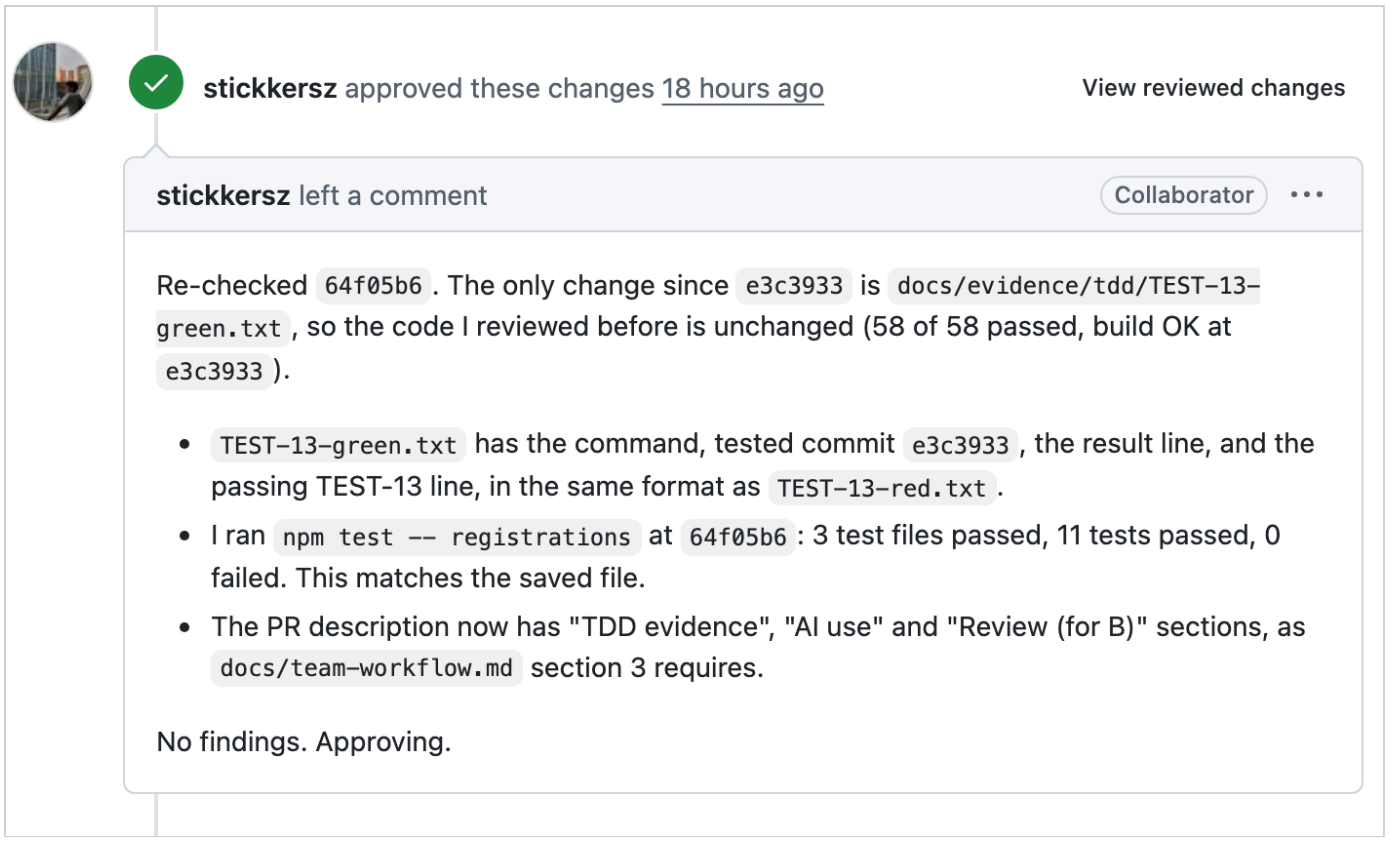
Reviews	stickkersz: changes requested at `e3c3933` (2026-11-07 14:03 UTC) stickkersz: approved at `64f05b6` (2026-11-07 14:25 UTC)
Substantive review (excerpt)	I checked out `e3c3933` and ran `npm test` (58 of 58 passed) and `npm run build` (OK). The code matches `docs/sprints/sprint-1/f-04-register.md`, and B's test files are unchanged from `e98ead9`. Two items required by the team workflow are missing, so this is Request changes. **Please change in this PR:** - Add `docs/evidence/tdd/TEST-13-green.txt` (`docs/team-workflow.md` ...
Into staging	merge commit `c508973` by stickkersz, 2026-11-07 14:26 UTC
Into main	release PR #79, merge commit `12dcd3c`, tag `v0.1.0-sprint1`



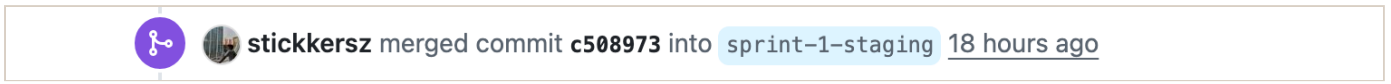
PR #65 on GitHub (logged out): title, merged state, branch into sprint-1-staging



PR #65: review 1



PR #65: review 2



PR #65: merge into sprint-1-staging

History sequence: contract and tests before implementation

Commit IDs and dates from the repository (UTC). In both Issues the contract was merged first, then the failing tests were committed, then the implementation, then the review and the merges.

Issue #5 (F-02 Publish event)

Date (UTC)	Commit	Event
2026-11-07 08:35	4d5a2d2	Contract PR #56 merged into staging
2026-11-07 10:43	c571cc1	stickkersz: test(F-02): add failing service tests (AC-07, AC-29, AC-30, AC-35)
2026-11-07 11:18	5e8fc6d	stickkersz: Merge remote-tracking branch 'origin/sprint-1-staging' into feature/s…
2026-11-07 13:15	f0f7f12	stickkersz: test(F-02): add failing API and DB tests (TEST-06..09, 25, 26, 29..34…
2026-11-07 13:16	259cb84	stickkersz: test(F-02): fix payload key list and leak check in API tests (TEST-06…
2026-11-07 13:16	e0d386b	stickkersz: feat(F-02): create, publish, My events and event detail API (FR-04, F…
2026-11-07 13:16	4f4593b	stickkersz: feat(F-02): add Create event and My events screens (FR-04, FR-15, NFR…
2026-11-07 13:18	4f4593b	Review by songt888: approved
2026-11-07 13:19	955f893	PR #63 merged into sprint-1-staging
2026-11-08 06:07	12dcd3c	Release PR #79 merged into main; tag v0.1.0-sprint1 on this commit

Issue #7 (F-04 Register for event)

Date (UTC)	Commit	Event
2026-11-07 08:50	f9c59d8	Contract PR #54 merged into staging
2026-11-07 13:38	6153fbb	stickkersz: test(F-04): add failing register tests (TEST-12..16, TEST-27, TEST-28…
2026-11-07 13:38	e98ead9	stickkersz: docs(F-04): save TEST-13 red run (AC-13)
2026-11-07 13:48	7bdf410	songt888: test(F-04): add failing TEST-37, TEST-43, TEST-44 (AC-37, AC-43, AC-4…
2026-11-07 13:52	e3c3933	songt888: feat(F-04): register for an event with row lock and Register button (…
2026-11-07 14:03	e3c3933	Review by stickkersz: changes requested
2026-11-07 14:18	64f05b6	songt888: docs(F-04): add TEST-13 green evidence (AC-13)

Date (UTC)	Commit	Event
2026-11-07 14:25	`64f05b6`	Review by stickkersz: approved
2026-11-07 14:26	`c508973`	PR #65 merged into sprint-1-staging
2026-11-08 06:07	`12dcd3c`	Release PR #79 merged into main; tag v0.1.0-sprint1 on this commit

Board status and release reference

GitHub Project "Event Registration - Sprint 1" at the time of this build (Issues and sub-tasks; sub-tasks merged into others were closed as not planned and are not on the board). Release: tag `v0.1.0-sprint1` on `12dcd3c`, created after both students agreed on Issue #44.

Column	Count	Issues
Backlog	1	#43
Done	34	#1, #2, #3, #4, #5, #6, #7, #8, #9, #12, #13, #14, #15, #17, #18, #19, #20, #21, #23, #24, #25, #32, #33, #35, #36, #37, #38, #40, #41, #42, #44, #73, #74, #75
Started	1	#10

F6. Contributions

Student	Commits on main (no merges)	PRs authored (merged)	Reviews given	Main work
A: Kirakit Kingkaew, 67070503460 (songt888)	46	20	26	SRS, feature inventory, traceability (Parts A, B); F-01 and F-02 contracts; foundation (Issue 3); F-01 Auth; F-04 implementation; F-05 My registrations; E2E golden journey and login/logout; 73.2 UI screens; PDF Parts A to C; release evidence (10.1); review of every B PR
B: Nattakit Prasertsak, 67070503413 (stickkersz)	50	15	33	System-Level SDS and three-sprint plan (Parts C, D); F-03 to F-06 contracts; F-02 Publish and F-03 Browse; F-04 red tests; F-06 Organizer registrations; E2E full event; QA run and NFR checks (9.4); UI style 73.1 and its fix; release PR and tag (10.4); PDF Parts D to F and assembly; review of every A PR

Counts are from the repository up to the release commit and from GitHub. Commits by "Kirakit Kingkaew" and "songt888" are the same student (two Git identities). Reviews include first-pass reviews by B's supervised autopilot posted under B's account (F7). Equal counts are not required; both students wrote requirements, contracts, tests, code and reviews.

F7. AI-use declaration

Collected from the "AI use" section of every merged Sprint 1 PR (#46 to #82), as `docs/team-workflow.md` section 5 asks, for the AI-use declaration in the PDF. Both students used AI agents for bounded Issues, with the engineering contract as input. No AI output was accepted on its own claim: the author ran the commands, the teammate reviewed the diff and ran the tests again, and both students must be able to explain every PR.

1. Tools

Tool	Used by	For
Claude Code (Anthropic), interactive	A (songt888) and B (stickkersz)	drafting docs and contracts, tests, implementation, UI, reviews, QA runs and evidence capture
Claude Code "autopilot" (Opus, Sonnet, Haiku), a local runner started by B	B	drafting B's contract and feature PRs (#53 to #55, #63, #64, #67 to #69), first-pass reviews of A's PRs posted under B's account after B's setup, and gated merges of A's approved PRs
Codex with document_writer and code_reviewer subagents	A	drafting and checking the SRS, feature inventory, traceability and the F-01 and F-02 contracts (#48, #50, #51, #52, #56)

2. Tasks per Issue

Issue	PRs	AI task	Checked by hand
1 Baseline docs	#46, #47, #48, #49, #50, #51	drafted the workflow, SRS, features, traceability, system SDS and plan	every FR/BR/NFR/F/D/AC/TEST ID present and paired; diagrams render; read in review
2 Contracts	#52 to #57	drafted the six feature contracts and the DoD	IDs, section references and D-xx decisions checked against the baseline; teammate review
3 Foundation	#58 to #61, #71	wrote the skeleton, schema, seed, test runner and README	build, `npm test`, fresh clone by the teammate (which found the NFR-08 defect fixed in #71)
4 to 8 Features	#62 to #67	wrote red tests first, then services, routes and screens from the contracts	`npm test` run by author and reviewer; red then green commits; review against the contract

Issue	PRs	AI task	Checked by hand
9 E2E and QA	#68, #69, #70, #72	wrote the E2E journeys and ran the QA checks	counts copied from the raw output; screenshots looked at
73 UI style	#76, #77, #81	built three design demos, the chosen Y2K style, and its fix	contrast computed, keyboard and reduced-motion checks, E2E unchanged and passing
10 Release and PDF	#78, #79, #80, #82, this PDF	assembled Parts A to F, ran the release checks and the evidence capture	every count checked against the logs; captions checked against screenshots

3. Problems found in AI output, and how they were caught

PR	What the AI got wrong	Caught by	Fix
#59	The seed hashed passwords with bcrypt cost 10, but the contracts fix cost 12; a generated `tsconfig.tsbuildinfo` was committed	B's review	cost 12; file removed and ignored
#63	Two red tests were wrong (a missing key in TEST-06, a TEST-51 leak check that matched the client's own field)	the first green run	test fix commit `259cb84`, recorded in the PR
#65	The TDD evidence file `TEST-13-green.txt` required by the workflow was missing	B's review	added before merge
#71	The autopilot's first review requested changes on a fresh-clone claim it had not proved (an install run with `--prefix`)	B re-ran a real fresh clone	B approved; the autopilot was given a rule never to request changes on an unproven claim
#76	A `client/node_modules` symlink was committed (the `.gitignore` pattern `node_modules/` does not match links)	A's review	removed from the index; pattern changed to `node_modules`
#76	The first two AI UI designs were generic; B rejected them	B	research-based redesign; three demos; the team chose Y2K
#78	The PR text described a one-off edit script, not the committed generator	B's review	sentence removed
#81	Window buttons covered loading and error messages on title-less windows	B's NFR-09 rerun screenshots	one CSS rule; rechecked at two widths
#82	The evidence capture script broke under React's double request in dev	the capture run	fresh page per state; recorded in the PR

4. Limits

- AI reviews posted under B's account were set up and supervised by B; B checked or redid the ones that mattered (for example #71, #77, #82) and every merge into `main` was a human decision (#79).
- No secret, password or real personal data was given to an AI tool; demo data is the synthetic seed.
- Prompt transcripts are not included; the per-PR "AI use" sections on GitHub are the record.

F8. Known limitations, incomplete work and failed or skipped tests

- **Failed or skipped tests:** none. On the release commit `npm test` runs 67 tests with 0 failed and 0 skipped, and `npm run test:e2e` runs 5 with 0 failed and 0 skipped. No test was weakened or removed to get green.
- **NFR-06** (data survives a restart) is checked locally only; the deployed check is Lab 5.
- **Server-side 500:** no automated test makes the server itself fail and checks the 500 error shape; the NFR-09 error state is forced in the browser. This is the one open row in the traceability gaps (Part B).
- **3D desktop layer** (D-16): three.js is loaded from a CDN at run time; without internet or a hardware GPU it is skipped and the page shows the plain sky. The screenshots in this Part were taken in headless Chromium, so they show the sky without the 3D objects.
- **Screenshots** are at 1280 x 800; the 375 px phone layout is in the repository under `docs/evidence/ui/`.
- **Not in Sprint 1** (by plan, Part D): cancellation, registration deadlines, check-in, attendance reports, waitlist, self-registration and password recovery.